Abstract

Real-time multi-object tracking systems commonly rely on fixed detector operating parameters and tracker confidence thresholds, which may become unsuitable when scene conditions change or when detectors with different confidence-score distributions are introduced. This thesis addresses these limitations by proposing two complementary, training-free control approaches that improve tracking performance without retraining the detector or modifying the underlying tracking algorithms.

The first approach, Adaptive Control for Multi-Object Tracking (AC-MOT), dynamically adjusts detector confidence thresholds, non-maximum suppression thresholds, and input resolution according to scene complexity and tracking information. The proposed controller employs three operating states with stability and recovery mechanisms, while its parameters are selected using cross-validation and fixed before final evaluation. Experiments on seven VisDrone2019-MOT sequences demonstrate improvements of 2.937 and 1.325 HOTA points and 17.995 and 6.318 MOTA points with ByteTrack and OATrack, respectively. Additional experiments on UAVDT demonstrate improvements in MOTA of 17.601 and 3.267 points without retuning.

The second approach, the Self-Calibrating Control Layer (SCL), addresses incompatibilities between detector confidence-score distributions and tracker thresholds. It introduces a selective, host-aware adaptation mechanism that analyzes recent detection scores, preserves the original tracker configuration when scores are compatible, and otherwise adjusts detection candidates, scores, and thresholds. Experiments involving seven trackers, four detection sources, and three datasets demonstrate improvements under detector–tracker mismatch conditions. In a controlled stress test involving halved detection scores, SCL restores two trackers from zero to approximately 66 HOTA points. Comparisons with adaptive-threshold baselines and with the published ACT method further demonstrate improvements in the evaluated configurations.

The proposed controllers introduce low additional computational overhead, averaging approximately 4.2 ms/frame for AC-MOT and 2.95 ms/frame for SCL in their respective experimental settings. The findings demonstrate that adaptive detector operating-point control and detector–tracker interface adaptation can improve tracking performance and robustness without additional model training.

Keywords: Real-time object detection, multi-object tracking, adaptive control, detector operating point, confidence-score adaptation, UAV tracking, training-free methods.

Acknowledgments

Special acknowledgment is due to those who have contributed to this work directly or indirectly. I would like to express my deep gratitude to my supervisors, Assoc. Prof. Dr. Mohamed Sobhy Mohamed and Prof. Dr. Tarek Ahmed Mahmoud, for their guidance, their patience, and their careful reading of every result throughout this research. Their advice shaped both the methods and the way they were evaluated.

I am grateful to the Chair of Computing Engineering and Artificial Intelligence of the Military Technical College for its support during my studies, and to the authors of the published detectors and trackers used in this thesis and the maintainers of the VisDrone, UAVDT, MOT17, and KITTI benchmarks and of the TrackEval library, whose released code and data made a reproducible evaluation possible.

Finally, I would like to thank my family for their patience, encouragement, and support.

List of Abbreviations

AC-MOT	Adaptive Control for Multi-Object Tracking
AF-JPDA	Appearance-aided joint probabilistic data association
AOB	Always-on banding
AP	Average precision
AssA	Association accuracy
CI	Confidence interval
CNN	Convolutional neural network
COCO	Common Objects in Context
CPU	Central processing unit
CV	Cross-validation
DetA	Detection accuracy
DETR	Detection transformer
FN	False negatives
FP	False positives
FPS	Frames per second
GBI	Gap-filling post-processing of BoostTrack
GPU	Graphics processing unit
GT	Ground truth
HOTA	Higher order tracking accuracy
ID	Identity number
IDF1	Identity F1 score
IDFN	Identity false negatives
IDFP	Identity false positives
IDS	Identity switches
IDTP	Identity true positives
IoU	Intersection over union
JDE	Joint detection and embedding
mAP	Mean average precision
MOT	Multi-object tracking
MOTA	Multiple object tracking accuracy

NMS	Non-maximum suppression
R-CNN	Region-based convolutional neural network
ReID	Re-identification
RQ	Research question
RT-DETR	Real-time detection transformer
SCI	Scene complexity index
SCL	Self-calibrating control layer
SORT	Simple online and realtime tracking
SSD	Single shot multibox detector
TP	True positives
TPE	Tree-structured Parzen estimator
UAV	Unmanned aerial vehicle
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Overview

Computer vision gives machines the ability to interpret images and video. One of its central tasks is object detection: for each image, a detector returns the objects of interest as bounding boxes, each with a class label and a confidence score that expresses how sure the detector is about the box. Detection has moved from hand-crafted features to convolutional neural networks (CNNs) and, more recently, to transformer models [1, 2, 3, 4], and real-time detectors are now available for a wide range of hardware [5]. Real-time target detection and tracking combines detection with tracking: every target of interest must be found in each frame and followed over time while the video is being processed, with little delay.

Multi-object tracking (MOT) extends detection over time. It finds every object of interest in a video and gives each one an identity number (ID) that stays the same while the object is visible. MOT is a basic function of surveillance, traffic monitoring, driver assistance, and drone systems [6, 7, 8], where the results are usually needed with little delay. Unmanned aerial vehicles (UAVs) are a particularly demanding setting. They are used for traffic monitoring, surveillance, and search, and many of these tasks require every object to be detected in every frame and to keep the same identity over time [7, 6]. Aerial video makes this difficult: objects are small, their number changes quickly, the camera moves, and illumination and blur change within one video [6, 9, 10].

Most MOT systems follow the tracking-by-detection design [11, 12, 13, 5]. In each frame, a detector returns candidate boxes with their scores, and an online tracker links these boxes into tracks, one per object. The two parts are developed separately and combined freely: any detector can feed any tracker. Their interaction is governed by a small number of settings that decide which boxes are used. Figure 1.1 shows this pipeline from its starting point, the input video stream, to its final output, the tracks with identities, and gives the input, the operation, and the output of each processing

block. The detector removes boxes whose score is below its confidence threshold, removes overlapping boxes with non-maximum suppression (NMS), and resizes every frame to one input resolution; these three values form the *detector operating point*. The tracker, in turn, uses a box only if its score passes its own thresholds: one score to match a box to an existing track, one score to start a new track, and, in two-stage trackers such as ByteTrack, a lower score for a second matching step [13]. In almost every system these settings are fixed once, for one detector and one type of scene, and they stay the same for the whole video.

The reliability of a tracking system therefore depends not only on the quality of its detector and tracker, but also on whether these fixed settings fit the scene and the detector that are actually present. Two situations break this fit in practice. Within one aerial video, the scene changes from sparse to crowded, from clear to blurred, and from large to tiny objects. Across the life of a system, the detector is replaced, retrained, or compressed, while the tracker keeps the thresholds that its authors tuned for another detector.

This thesis addresses real-time target detection and tracking and follows one research direction through three studies. A survey of real-time object detection and tracking, published by the author [5], reviewed the detectors, trackers, datasets, and metrics of the field, showed that tracking by detection remains the dominant paradigm, and identified the accuracy–speed trade-off, the hardware, and the deployment environment as the main constraints of real-time operation. Building on this review, the thesis treats the settings that connect the detector and the tracker as control variables of the tracking system rather than as fixed parameters, and it proposes two training-free controllers that adapt them online with the detector and the tracker left unchanged. The first, Adaptive Control for Multi-Object Tracking (AC-MOT), adapts the detector operating point to the changing aerial scene (Chapter 3). The second, a self-calibrating control layer (SCL), adapts the candidates, scores, and thresholds that a tracker receives to the score scale of the detector in use (Chapter 4).

1.2 Motivation and Challenges

The motivation of this thesis comes from the three studies on which it is built. The survey identified the general difficulties of real-time detection and tracking [5]; the AC-MOT and SCL studies then measured, on public benchmarks, how strongly two of these difficulties affect a working tracking pipeline. In the following, statements supported by the literature are separated from effects that are demonstrated by the experiments of this thesis.

Variable scene complexity. Aerial video changes within a single sequence: object density, object size, illumination, and blur vary with the flight path, the altitude, and

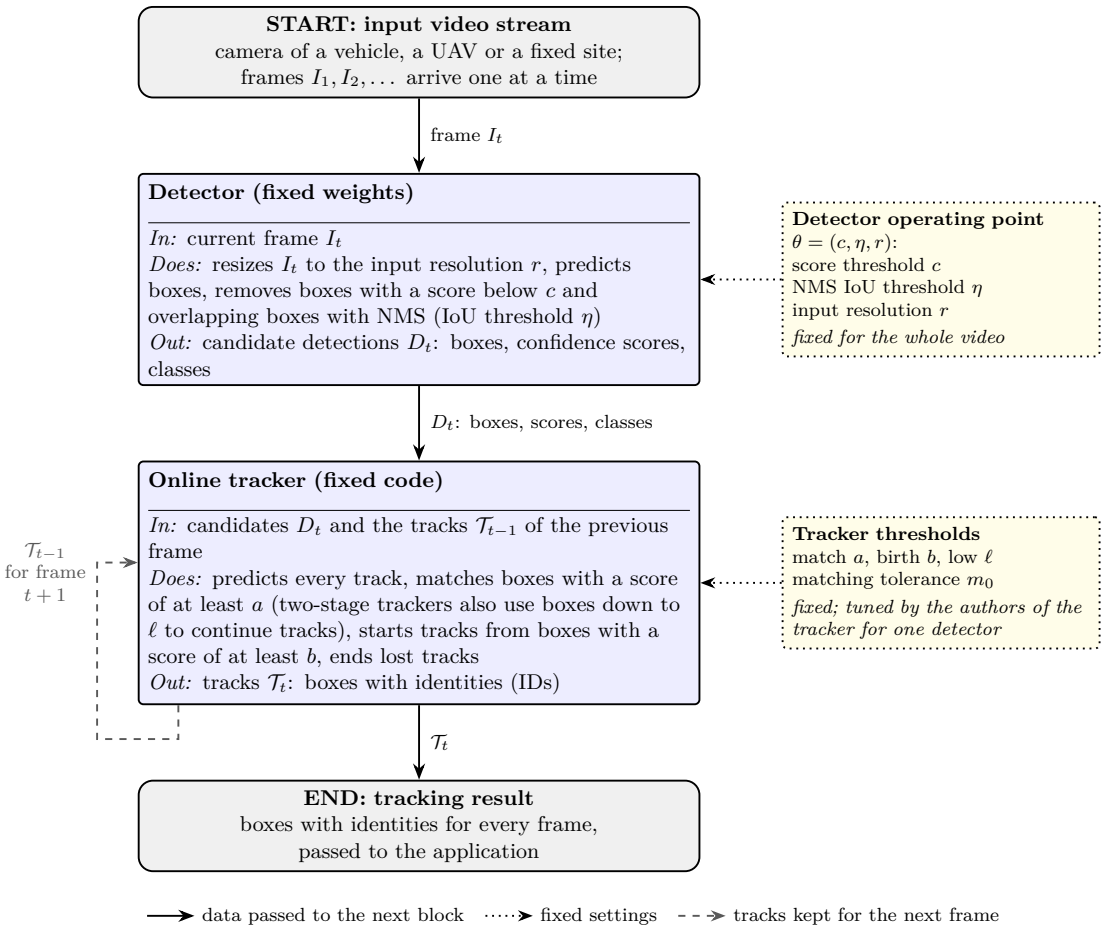


Figure 1.1: Tracking-by-detection pipeline with fixed operating points. The pipeline starts from the input video stream. For every frame I_t the detector returns candidate detections D_t (boxes, confidence scores, and classes), and the online tracker combines them with the tracks $\mathcal{T}_{t-1}$ of the previous frame into the tracks $\mathcal{T}_t$, boxes with identities, which form the tracking result. The detector operating point $\theta = (c, \eta, r)$ decides which boxes reach the tracker, and the tracker thresholds (a, b, ℓ, m_0) decide which of these boxes may match or start a track; in common practice all of these values are fixed for the whole video. Solid arrows: data; dotted arrows: fixed settings; dashed arrow: tracks kept for the next frame.

the viewing angle [6, 9]. A setting that works in a sparse, clear scene can keep too much clutter in a dense scene, and a setting that works in a dense scene can lose small or weak objects in another part of the same video. A fixed operating point is therefore a compromise over all the scenes of a video.

Small and crowded objects. Small objects occupy few pixels and usually receive low detector confidence [10]. In crowded scenes, individual objects are hard to separate, and occlusion causes missed detections and identity switches [1, 5]. A larger input resolution helps small objects, but it costs the same detector time on easy and hard frames.

Motion, occlusion, and image degradation. The motion of a UAV camera shifts the boxes of all objects between frames, a displacement that the motion model of a tracker must absorb [14]. Severe occlusion, extreme illumination changes, blur, and adverse weather remain common causes of missed detections and tracking failures [5, 15].

Sensitivity to the detector operating point. The choice of the confidence threshold, the NMS threshold, and the input resolution moves false detections, missed objects, identity switches, and runtime together. This effect is demonstrated in Chapter 3: with the same frozen detector and the same trackers on the same aerial sequences, replacing the fixed operating point with the operating points selected online by AC-MOT raised the multiple object tracking accuracy (MOTA) by 17.995 points with ByteTrack and by 6.318 points with OATrack.

Tracker dependence on detection scores. Trackers read detector scores through fixed thresholds that their authors tuned for one detector, usually YOLOX [16] on the MOT17 benchmark. Detector scores, however, are not calibrated across models and conditions [17, 18], so a new detector usually gives scores on a different scale, and nothing in the system notices. The effect is demonstrated in Chapter 4: ByteTrack in its official setting starts a track only from a box with a score of at least 0.7; when the scores of a published detector are halved, a car that was scored 0.9 now gets 0.45, ByteTrack never starts a track, and its higher order tracking accuracy (HOTA) [19] falls from 67.70 to 0.

Variation between detectors, trackers, and datasets. The best setting depends on the combination of components. In this thesis, a tracker that uses low-score boxes in a second association step and a tracker that weights detections by quality reacted differently to the same detector control (Chapter 3), and single-stage and two-stage trackers needed different help when the detector changed (Chapter 4). With a replacement detector, the real-time detection transformer RT-DETR-L, ByteTrack in its published setting reached a MOTA of only 3.29 on held-out aerial data (Chapter 4).

Limits of fixed configurations and of existing adaptation. Retuning every detector–tracker pair for every deployment needs labeled data, which a deployed system does not have. Methods that adapt the detector at run time usually learn their policy with

an extra model for one detector [20, 21, 22]; methods that adapt tracker thresholds are each built into one tracker [23, 24, 25]; and score calibration fits a mapping on labeled data [18]. None of these generalizes directly to a new combination of detector, tracker, and scene without training or labels.

These observations lead to the challenges that an online controller of the operating point must meet:

- *No labels at run time.* A deployed system has no ground truth, so the adaptation must be decided from the video and from the system's own outputs.
- *Causality and low cost.* Only the current and earlier frames may be used, and the extra computation must be small compared with the detector, so that the system stays usable in real time.
- *Unchanged components.* Retraining the detector or rewriting the tracker for every new combination is costly and loses the modularity of tracking by detection; the controller should work from the outside.
- *Stability.* Each change of the operating point changes the boxes that reach the tracker, so the settings should follow lasting changes of the scene rather than single frames.
- *No harm when nothing is wrong.* A tracker that already fits its detector should keep its accuracy; an adaptation that always intervenes can lower it (Section 4.4.1).

1.3 Problem Statement

Let a video be a sequence of frames $I_1, I_2, \ldots$, processed online by a frozen detector and a frozen tracker. The weights of the detector, and the code and association algorithm of the tracker, are not changed. At frame t, the detector runs with an operating point $\theta_t = (c_t, \eta_t, r_t)$, its confidence threshold, NMS threshold, and input resolution, and returns candidates D_t, each with a box, a score, and a class. The tracker reads these candidates through its published thresholds and returns the tracks of frame t. A controller may set only values that the pipeline already exposes, and it may base its decision only on the current frame, on earlier frames, and on the outputs that the detector and the tracker have already produced. No ground truth, no future frame, and no training are allowed. The goal is to improve tracking accuracy, measured by HOTA, MOTA, the identity F1 score (IDF1), and the number of identity switches (IDS), relative to the same pipeline with fixed settings, while adding little computation.

One representation of the pipeline. Both controllers of this thesis act on the same tracking-by-detection pipeline, which can be written for frame t as

$$D_t = f_{\text{det}}\left(I_t;\, \theta_t\right), \tag{1.1}$$

$$\mathcal{T}_t = f_{\text{trk}}\left(D_t, \mathcal{T}_{t-1};\, \phi_t\right), \tag{1.2}$$

where f_{det} is the frozen detector, $\theta_t = (c_t, \eta_t, r_t)$ its operating point, f_{trk} the frozen tracker, $\mathcal{T}_t$ the tracks of frame t, and ϕ_t the tracker-side interface: which candidates of D_t are passed to the tracker, with which scores, and the thresholds and matching tolerance with which the tracker reads them. In common practice both θ_t and ϕ_t are constants. AC-MOT controls θ_t in Eq. (1.1) from the current image and the boxes tracked in earlier frames, and SCL controls ϕ_t in Eq. (1.2) from the scores of earlier frames, the current candidates, the previous tracks, and the camera motion. The two controllers are therefore complementary: they act on the two control points of one pipeline, the input of the detector and the interface between the detector and the tracker.

Real-time requirement. In this thesis, real-time operation means that the whole pipeline—detector, controller, and tracker—processes each frame within the frame interval of the video stream. For a stream of 30 frames per second (FPS), this interval is $1000/30 \approx 33.3$ ms per frame. The detector takes most of this budget, so a controller suits real-time use when it adds only a small part of it. The thesis therefore measures the cost of each controller separately from the detector and the tracker, on representative hardware: AC-MOT adds about 4.2 ms per frame on a Tesla T4 GPU (Section 3.6.4), about 13% of the 33.3 ms interval, and SCL adds 2.95 ms per frame on a 4-vCPU processor without a GPU (Section 4.9), about 9% of it. In the first development stages of AC-MOT, candidate settings also had to keep the whole pipeline at 25 FPS or more (Section 3.3.5).

Existing work does not solve this problem as stated. Adaptive inference methods change the detector configuration, but they learn a policy for one model and optimize detection rather than tracking; adaptive-threshold methods adapt one tracker to the current frame; and calibration methods need labels (Section 2.4.5). The thesis therefore addresses three research questions (RQ1–RQ3):

RQ1 Can the detector operating point be adapted online from inexpensive scene cues and the past output of the tracker, without training, so that tracking in aerial video improves over a fixed operating point with the detector and the trackers unchanged? (Chapter 3)

RQ2 Can a label-free and causal layer decide from recent detector scores whether the score scale of a detector fits the published thresholds of a tracker, leave a well-

matched pair as it is, and restore a mismatched pair through the tracker's own published settings? (Chapter 4)

RQ3 Do the effects of the frozen controllers hold on data, trackers, and detectors that were not used to design them, at a cost compatible with real-time operation? (Chapters 3 and 4)

The two research questions RQ1 and RQ2 describe two related but distinct problems. The first concerns one point of the pipeline, the detector call, in aerial video, while the tracker keeps reading the detector scores through its published thresholds. Its controller, AC-MOT, is formulated in Chapter 3 as a control rule $\theta_t = \pi(I_t, \mathcal{B}_{1:t-1})$ for Eq. (1.1) that selects the operating point from the current image and the boxes tracked in earlier frames. The second problem concerns the interface between the detector and the tracker: it asks how one fixed set of rules can make the scores of any detector usable by different published trackers. Its controller, SCL, is formulated in Chapter 4 as a layer for Eq. (1.2) that maps the score logits of earlier frames, the current candidates, the previous tracks, and the camera motion to the candidates, scores, thresholds, and matching tolerance that the tracker receives. The two controllers act at different points of the pipeline and are not two versions of one algorithm: AC-MOT does not adapt the tracker thresholds, and SCL does not choose the detector operating point. Each controller stands on its own, and neither replaces the other.

1.4 Solution Approach

The thesis answers these questions with two rule-based controllers that share the same principles: no learned weights, no labels at run time, use of past information only, an unchanged detector and tracker, values fixed before the final evaluation, and paired bootstrap statistics for every comparison. Figure 1.2 shows where they act in the pipeline of Fig. 1.1, with the input and output of each block: both start from the same input video stream; AC-MOT turns image cues of the current frame and the boxes tracked in the previous frame into the detector operating point θ_t, and SCL turns the candidate detections of the detector into the candidates, scores, thresholds, and matching tolerance that the tracker receives. Table 1.1 compares their scope.

Training-free. In this thesis, a controller is training-free when no neural network or other learned model is trained for it, the detector and the tracker are not retrained, and no parameter of the controller is fitted while the system runs: the online adaptation follows fixed rules. The values of both controllers are fixed before deployment. AC-MOT selects its thresholds and actions once, by cross-validation on separate calibration sequences, and then freezes them (Section 3.3.5); SCL uses standard values and design choices without a parameter search (Section 4.3.9).

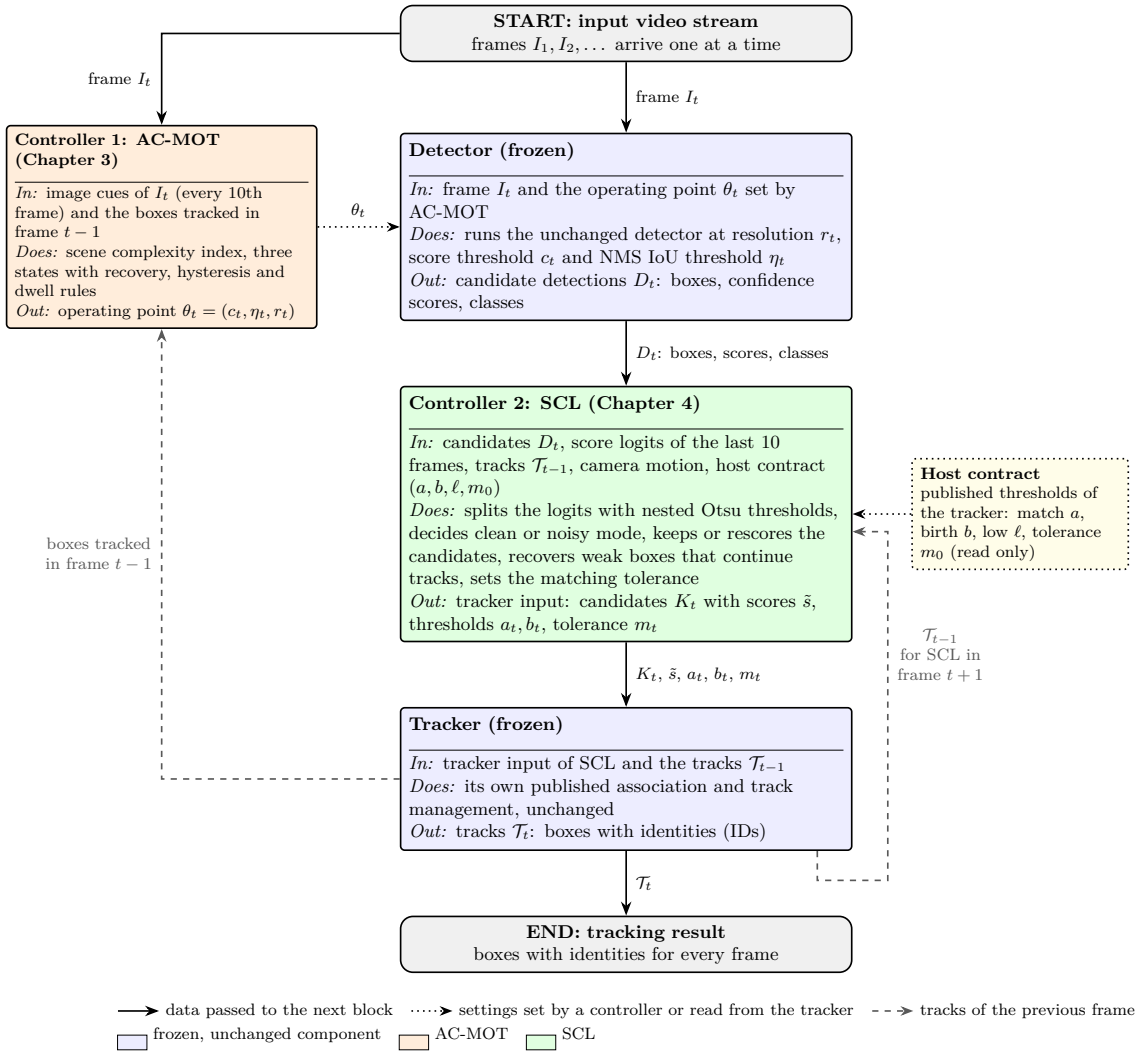


Figure 1.2: Outline of the proposed approach: where the two controllers act in the tracking-by-detection pipeline. Starting from the input video stream, Controller 1 (AC-MOT) reads image cues of the frame and the boxes tracked in the previous frame and sets the detector operating point θ_t. Controller 2 (SCL) receives the candidate detections D_t and, from the recent score logits, the previous tracks, the camera motion, and the host contract of the tracker, sets the candidates K_t and their scores, the thresholds a_t and b_t, and the tolerance m_t with which the tracker runs. The detector and the tracker are not modified: AC-MOT does not change the tracker, and SCL neither retrains the detector nor changes its operating point. Each controller can be used on its own; Chapter 3 evaluates AC-MOT and Chapter 4 evaluates SCL.

A. AC-MOT: online control of the detector operating point. AC-MOT runs next to the detector. On the first frame and then on every tenth frame, it measures inexpensive cues of the current image and of the boxes that the tracker reported in the previous frame, averages the recent crowd level into a scene complexity index (SCI), and selects one of three detector operating points through a three-state controller with recovery, hysteresis, and dwell rules. Its output is the confidence threshold, the NMS threshold, and the input resolution of the detector until the next analysis step. Its thresholds and actions were selected by cross-validation on separate calibration sequences and then frozen. It requires only a detector that can be run at the three selected operating points and a tracker whose boxes can be read after each frame.

B. SCL: a self-calibrating layer between the detector and the tracker. SCL runs once per frame between the detector and the tracker. It splits the score logits of the last ten frames twice with Otsu's method [26] into background, uncertain, and confident groups, and decides from the share of confident candidates whether the score distribution is clean or noisy. On clean scores it keeps the tracker's scores and thresholds and lowers a threshold only if it would reject the confident group; on noisy scores it chooses the candidates, gives them new scores, and sets the thresholds. A track-guided recovery lets a weak box that clearly continues a track reach single-stage trackers, which have no low-score step of their own, and the matching tolerance follows strong camera motion. The layer serves different published trackers through four of their published settings, called the host contract. Its outputs are the candidates, their scores, the thresholds, and the matching tolerance of the tracker for each frame. It requires only that the order of the scores carries information about the objects and that a tracker publishes the four values of its contract, and it was evaluated with seven trackers, four detection sources, and three datasets.

The two controllers form one research narrative. The survey established that tracking by detection is the dominant paradigm and that detection accuracy and association reliability depend on each other [5]. AC-MOT showed that the detector operating point is a first-order variable of aerial MOT that can be controlled online without training. SCL then turned to the relation between the detector's scores and the tracker's thresholds and showed that this compatibility can be checked and restored online for different trackers.

Figure 1.3 shows the effect of each controller on real tracked frames, taken from the examples of Chapters 3 and 4. In a crowded night scene of VisDrone2019-MOT, AC-MOT gives ByteTrack a cleaner detection stream: the tracker follows 25 annotated objects (24 at the fixed operating point) and outputs 23 false tracks instead of 54. On KITTI, where OC-SORT runs with a detector it was not tuned for, SCL lets the tracker follow 9 of 10 labeled objects instead of 6.

The contributions of this research are as follows:

Table 1.1: The two controllers of this thesis: where they act, what they read and set, and how their values were chosen.

Aspect	AC-MOT (Chapter 3)	SCL (Chapter 4)
Acts on	detector operating point: confidence threshold, NMS threshold, input resolution	tracker input: candidates, scores, thresholds, matching tolerance
Reads	image cues of the current frame and tracked boxes of the previous frame	score logits of the last 10 frames, tracks of the previous frame, camera motion, host contract
Update	every 10 frames	every frame
Decision	three states from the scene complexity index, with recovery, hysteresis, and dwell	clean or noisy mode from nested Otsu splits and a whole-stream median
Values chosen by	cross-validation on calibration sequences	standard values and design choices, no search
Unchanged parts	detector weights, tracker	detector, tracker code and association
Evaluated on	VisDrone2019-MOT and UAVDT, ByteTrack and OATrack	MOT17, KITTI, and UAVDT, seven published trackers

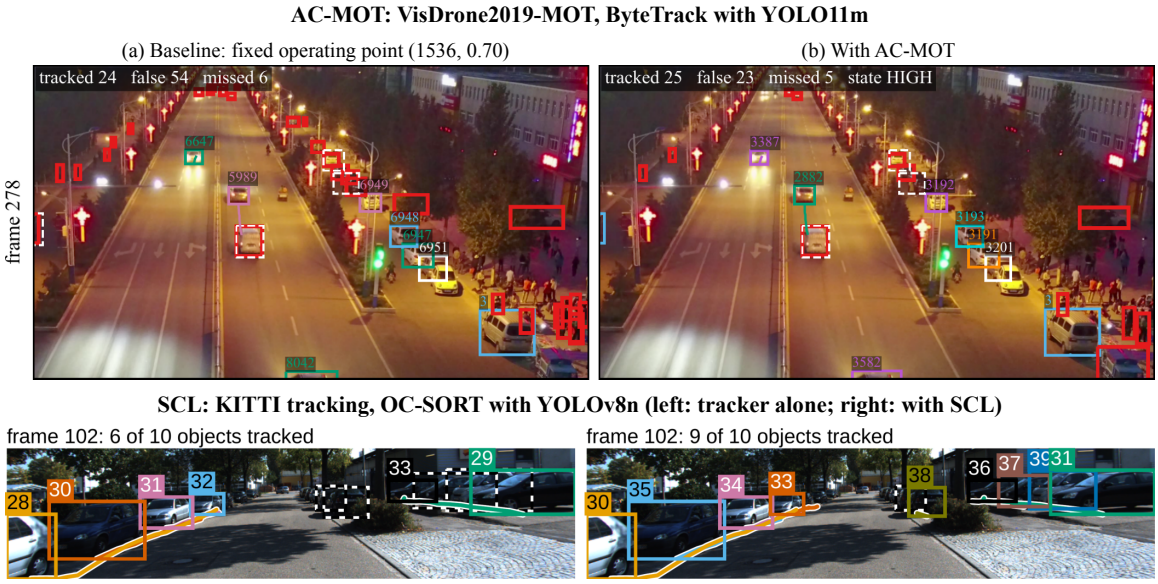


Figure 1.3: The two controllers on real tracked frames. Top: VisDrone2019-MOT sequence uav0000117_02622_v, frame 278, ByteTrack with YOLO11m, (a) at the fixed operating point (1536, 0.70) and (b) with AC-MOT (from Fig. 3.5); red: false tracks; white dashed: missed objects. Bottom: KITTI sequence 0001, frame 102, OC-SORT with YOLOv8n, the tracker alone (left) and with SCL (right) (from Fig. 4.4). Each color and number is one track, and its line shows the last 20 frames.

- A structured, deployment-oriented survey of real-time object detection and tracking, published at IEEE ICMISI 2026 [5], which forms the basis of the technical background in Chapter 2.
- AC-MOT, a training-free and causal controller that treats the detector operating point as part of the MOT system and adapts it online, with the detector and the tracker unchanged, together with an optimization-guided development path whose final values were selected by cross-validation on calibration sequences [27].
- An evaluation of AC-MOT on VisDrone2019-MOT with two trackers, ByteTrack and OATrack, using eight MOT metrics and paired bootstrap intervals, and a transfer to UAVDT without retuning.
- SCL, a training-free and causal layer whose novelty is selective, host-aware adaptation: it keeps the tracker's scores and thresholds when the scores are clean, takes control only when they are noisy, and expresses every action through a host contract that lets one fixed configuration serve single-stage and two-stage trackers without changing them [28].
- An evaluation of one frozen SCL configuration on seven published trackers, four detection sources, and three datasets, including score shifts, predeclared external trackers, a held-out aerial test split, comparisons with simple adaptive thresholds and the published ACT method, an ablation, and a sensitivity analysis.
- Runtime measurements showing that both controllers add only a few milliseconds per frame.

The detectors, the trackers, Otsu's method, and the phase correlation used by SCL are existing methods and are used unchanged; the contributions lie in the control rules, in the way they connect existing components, and in their evaluation. The survey has been published; the AC-MOT and SCL manuscripts have been submitted and are under review (see Publications).

1.5 Thesis Organization

The rest of this thesis is organized as follows.

Chapter 2. Background and Related Work

This chapter first reviews the main applications of real-time object detection and multi-object tracking: surveillance and security, autonomous vehicles and driver assistance, unmanned aerial vehicles, industrial robotics, healthcare and assistive systems, and retail analytics, with emphasis on aerial video. It then provides the technical background needed for the two controllers: object detection, deep learning-based detectors,

MOT and its paradigms, data association and track management, detector confidence and thresholding, aerial tracking and the datasets of the thesis, and the evaluation metrics and protocols. It then reviews the related work on tracking-by-detection systems, adaptive inference, adaptive thresholds, and score calibration, compares the closest methods, and states the research gap.

Chapter 3. AC-MOT: Adaptive Detector Operating-Point Control

This chapter formulates the control of the detector operating point, presents the scene cues, the scene complexity index, the three-state controller, and the action map of AC-MOT, explains how its values were selected through optimization experiments on calibration data, and evaluates the frozen controller on VisDrone2019-MOT and, without retuning, on UAVDT.

Chapter 4. SCL: Training-Free Self-Calibrating Control Layer

This chapter formulates the compatibility between a detector's score scale and a tracker's thresholds, presents the five steps and the host contract of SCL and its development and freeze, and evaluates the frozen layer on seven trackers, four detection sources, and three datasets, with adaptive-threshold baselines, the ACT method, an ablation, a sensitivity analysis, and runtime measurements.

Chapter 5. Conclusions and Future Work

This chapter draws the conclusions of the thesis as a whole and summarizes the findings of the two controllers, states the limitations of the research, and outlines directions for future work that follow from them.

Chapter 2

Background and Related Work

2.1 Introduction

The two controllers of this thesis act on the settings that connect a detector and a tracker. This chapter first places real-time detection and tracking in the applications that need them, and then establishes the technical foundations on which the controllers are built. Section 2.2 reviews six major application domains, surveillance and security, autonomous vehicles and driver assistance, unmanned aerial vehicles, industrial robotics, healthcare and assistive systems, and retail analytics, and gives particular attention to aerial video, the domain of the experiments of this thesis.

To follow the design and evaluation of the controllers, the reader then needs to know how a detector produces its boxes and scores, how a tracker turns these boxes into tracks, how the scores enter the tracker's decisions, what makes aerial video difficult, and how tracking accuracy is measured. Section 2.3 provides this background. Section 2.3.1 introduces object detection and the three settings of the detector operating point, and Section 2.3.2 reviews deep learning-based detectors, including those used in this thesis. Section 2.3.3 defines multi-object tracking and its constraints, Section 2.3.4 describes the tracking paradigms, and Section 2.3.5 reviews data association and track management in the trackers used in this work. Section 2.3.6 explains how trackers use detector scores, why these scores differ between detectors, and introduces the thresholding tools on which SCL builds. Section 2.3.7 describes the difficulties of aerial tracking and the datasets of the thesis, and Section 2.3.8 defines the evaluation metrics and the experimental protocol shared by Chapters 3 and 4. Sections 2.2 and 2.3.1–2.3.7 extend the survey published by the author [5]. Section 2.4 then reviews the related work by theme, compares the closest methods with the proposed controllers, and states the research gap. Section 2.5 summarizes the chapter.

2.2 Applications of Real-Time Object Detection and Multi-Object Tracking

Real-time object detection and tracking are needed wherever a system must perceive a changing environment continuously and act on what it sees with little delay [1, 5]. The same tracking-by-detection pipeline serves very different domains, but the domains differ in the objects of interest, in the camera platform, in the conditions under which the video is recorded, and in how much delay and computation they can accept. This section reviews six major application domains, following the survey published by the author [5], and identifies the difficulties that each of them places on detection and tracking. Table 2.1 summarizes them, and Figures 2.1–2.6 show a typical system or the main difficulties of each domain. In the block diagrams of these figures, gray blocks are sensors, blue blocks process the data close to the sensors, green blocks combine the data of several sources, and orange blocks are the outputs of the system. Aerial video from unmanned aerial vehicles, the domain of the experiments of this thesis, is treated in more detail in Section 2.2.3.

2.2.1 Surveillance and Security

Video surveillance in public spaces, transport hubs, and critical infrastructure is one of the oldest uses of detection and tracking. A detector finds the people and vehicles in each frame, and a tracker keeps their identities so that their paths can be analyzed: a person who enters a restricted area, an object that is left behind, or a vehicle that stops in a prohibited place is recognized from its track rather than from a single frame. Large collections of real surveillance video have also been used to learn to detect anomalous events such as fights, accidents, and robberies [29].

Large sites are covered by many cameras, so tracking is often organized at two levels. Each camera tracks the objects in its own field of view, and a central stage associates the tracks of different cameras with appearance features so that a person keeps one identity across the whole site [30, 31]. Figure 2.1 shows this organization for four cameras. The difficulties of this domain are crowded scenes with frequent occlusion, strong changes of illumination between day and night, and the cost of processing many video streams at the same time, which limits the computation available for each stream.

2.2.2 Autonomous Vehicles and Advanced Driver-Assistance Systems

Autonomous vehicles and advanced driver-assistance systems (ADAS) detect and track vehicles, pedestrians, cyclists, and traffic signs around the vehicle. Tracking adds what

Table 2.1: Application domains of real-time object detection and multi-object tracking: the objects that are typically tracked, the main difficulties, and the operating constraints.

Domain	Typical objects	Main difficulties	Operating constraints
Surveillance and security	people, vehicles, carried objects	crowds, occlusion, day and night lighting, identities across cameras	many continuous streams per server
Autonomous vehicles and ADAS	vehicles, pedestrians, cyclists, traffic signs	fast motion, weather and night, safety-critical missed objects	strict latency on the vehicle
UAVs and drone tracking	small vehicles and people seen from above	small objects, scale variation, camera motion, occlusion, crowds, changing lighting	onboard or ground-station processing with limited power
Industrial robotics	parts, tools, people near the robot	clutter, reflective or textureless parts, precise localization	fixed cycle times, human safety
Healthcare and assistive systems	lesions, surgical tools, patients	reliability, false alarms, privacy, scarce annotations	clinical workflows, home monitoring
Retail and marketing analytics	customers, products on shelves	dense and similar products, occlusion, privacy	continuous store cameras

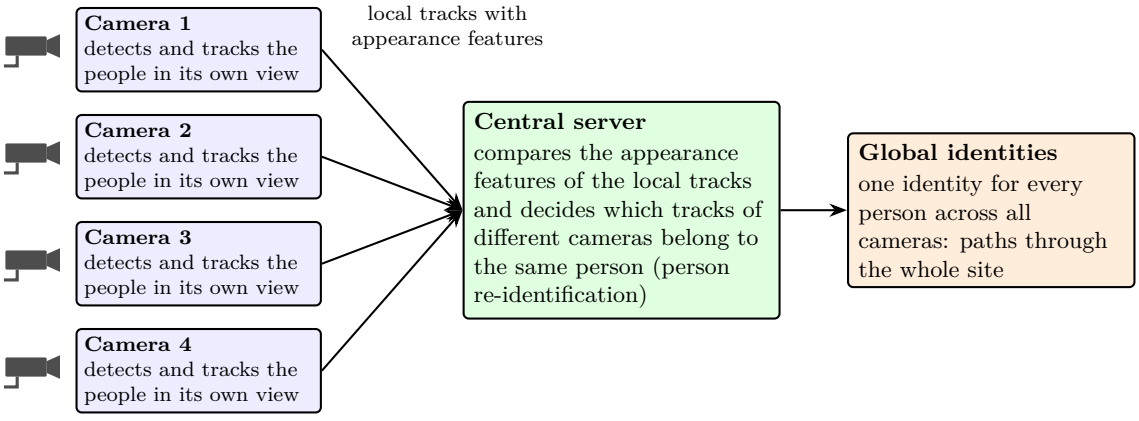


Figure 2.1: Multi-camera surveillance system with one identity per person. Every camera detects and tracks the people in its own view; a central server compares the appearance features of the local tracks and links the tracks of the same person across cameras (person re-identification).

a single frame cannot provide: the velocity and the recent path of every road user, which are needed to predict its motion and to plan the vehicle's own maneuvers. Public benchmarks recorded from cars, such as KITTI [8] and nuScenes [32], provide camera images together with LiDAR and, in nuScenes, radar data. Tracking in this domain is often performed in three dimensions; a simple combination of a Kalman filter and the Hungarian method already gives a strong real-time three-dimensional tracker [33]. Because optical sensors degrade in rain, fog, and snow, the fusion of LiDAR with 4D radar is studied to keep three-dimensional detection reliable in adverse weather [15]. Figure 2.2 shows the processing chain of such a vehicle: the measurements of cameras, LiDAR, and radar are fused, and the fused data are used to detect the surrounding objects, to track them, and to plan the motion of the vehicle. The main constraints of this domain are a strict latency budget on the vehicle and the safety cost of a missed object. KITTI is also one of the datasets of this thesis (Chapter 4).

2.2.3 Unmanned Aerial Vehicles and Drone Tracking

Unmanned aerial vehicles (UAVs) use real-time detection and tracking for traffic monitoring, surveillance, search and rescue, infrastructure inspection, and environmental monitoring [7, 34, 35]. The aerospace literature has studied the tracking of multiple ground targets from a multirotor UAV [36], feature descriptors for tracking moving ground objects [37], real-time onboard detection of urban objects [38], and vision-based guidance for aerial docking and refueling [39, 40]. Two public benchmarks, VisDrone [6] and UAVDT [9], provide annotated drone videos of urban traffic and crowds, and both are used in this thesis. Aerial video combines several difficulties that are less severe in the other domains:

- *Small objects.* Seen from altitude, a car or a pedestrian may cover only a few pixels. Small objects carry little appearance information and usually receive low detector confidence [10, 41].
- *Scale variation.* The altitude, the viewing angle, and the zoom change the size of the same kind of object within one video and between videos [9, 6].
- *Camera motion.* The platform moves and rotates, so every box shifts between frames even when the objects stand still; the motion model of the tracker must absorb this displacement [14].
- *Occlusion and crowded scenes.* Buildings, trees, bridges, and other vehicles hide objects for some frames, and crowded streets and squares contain many similar objects close together, which makes association and identity preservation difficult [6, 1].

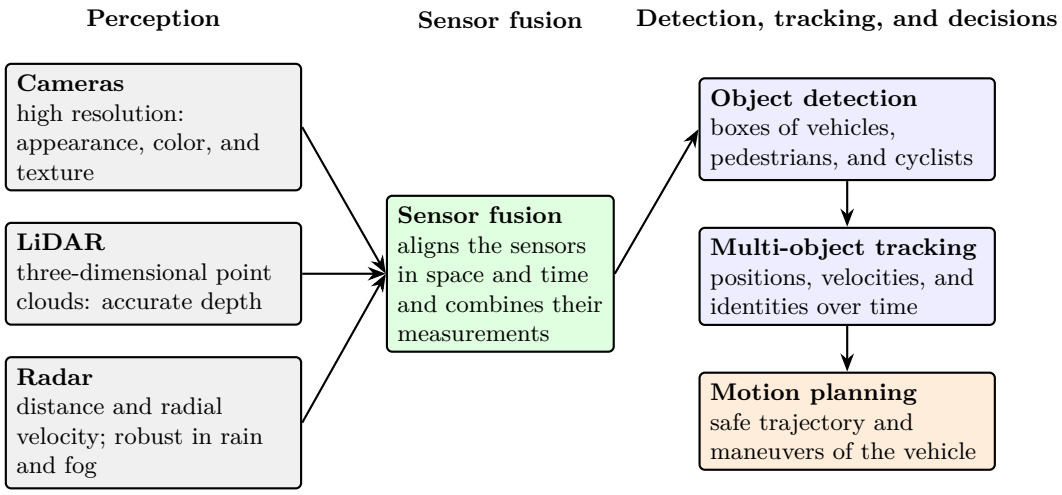


Figure 2.2: Multi-sensor perception of an autonomous vehicle. The measurements of cameras, LiDAR, and radar are fused, and the fused data are used to detect the surrounding objects, to track them over time, and to plan the motion of the vehicle.

- *Variable lighting and weather.* Flights take place by day and at night and under different weather conditions; UAVDT annotates these conditions together with the flying altitude and the camera view [9].
- *Real-time processing constraints.* Processing on board, or on a ground station with a limited link, restricts the size of the detector that can run at the frame rate of the video, which is why lightweight detectors are evaluated for drone platforms [35].

Figure 2.3 illustrates two of these difficulties, scale variation and camera motion. These conditions do not only make aerial tracking difficult; they also change within a single flight. When the scene becomes crowded, the detector passes many weak and false boxes to the tracker; when it becomes sparse or the objects become small, a setting that removed clutter may also remove the few true objects. The quality of the detection stream that reaches the tracker therefore varies over time, and a detector configuration chosen once for the whole video is a compromise over these conditions. In addition, the detector of a UAV system is often replaced by a smaller or compressed model to fit the onboard hardware, which changes the scale of its confidence scores. These two observations motivate the adaptive, training-free control studied in this thesis (Chapters 3 and 4).

2.2.4 Industrial Robotics

In industrial settings, detection and tracking allow robots to locate parts for pick-and-place operations, bin picking, assembly, and quality inspection, and to follow parts that move on conveyors. Learned grasp planning computes robust grasps of objects

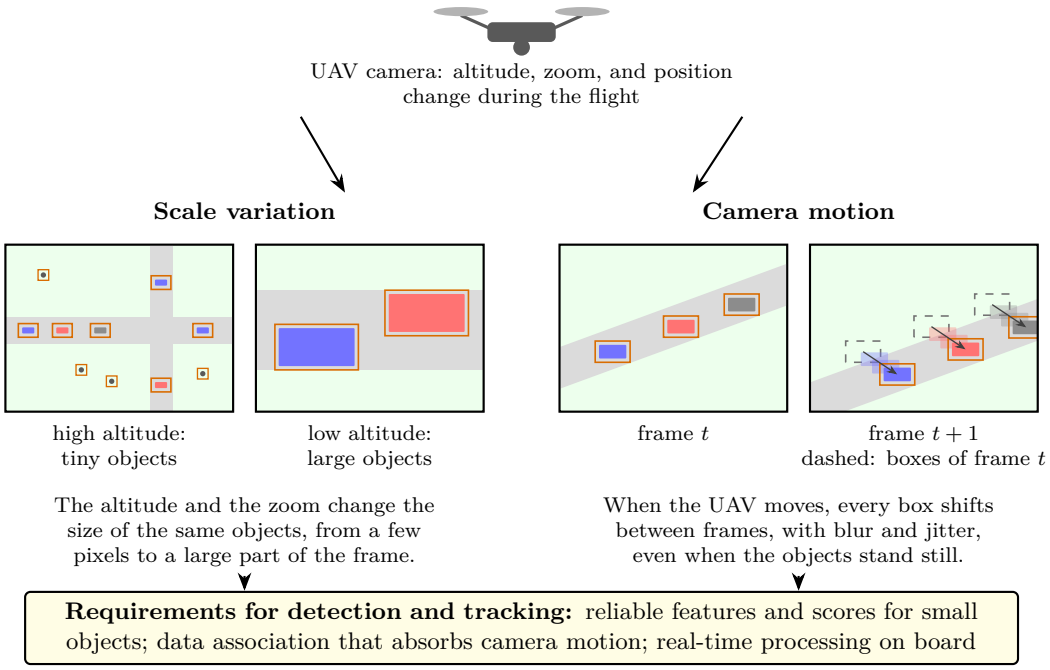


Figure 2.3: Scale variation and camera motion in UAV video. Changes of altitude and zoom change the size of the same objects, and the motion of the UAV shifts every box between frames. Orange boxes: detections.

from depth images [42], and visual simultaneous localization and mapping (SLAM) with ORB features lets a mobile robot estimate its own position from camera images while it moves [43]. Figure 2.4 shows a multi-camera tracking system for automated assembly: a local unit detects and tracks the parts in the view of each camera, a central server fuses the local tracks into one trajectory per part, and the robot controller uses these trajectories to plan grasps and to adjust its path in real time. When people work near robots, the detection and tracking of persons also becomes a safety function. The difficulties of this domain are cluttered bins, reflective or textureless parts, the precise localization that manipulation requires, and fixed cycle times.

2.2.5 Healthcare and Assistive Systems

In medicine, deep detection and segmentation networks localize lesions and anatomical structures in medical images [44]. In surgical video, the presence of instruments and the phases of an operation can be recognized automatically from laparoscopic recordings [45], which supports the monitoring and documentation of procedures. Assistive systems monitor elderly or vulnerable people at home and detect events such as falls with cameras and other sensors [46]. Figure 2.5 shows a monitoring system of this kind: wearable and ambient sensors feed a local unit that recognizes activities and detects falls or signs of distress, alerts reach clinicians and caregivers, and a central server keeps the records and analyzes them over time. This domain requires high reliability

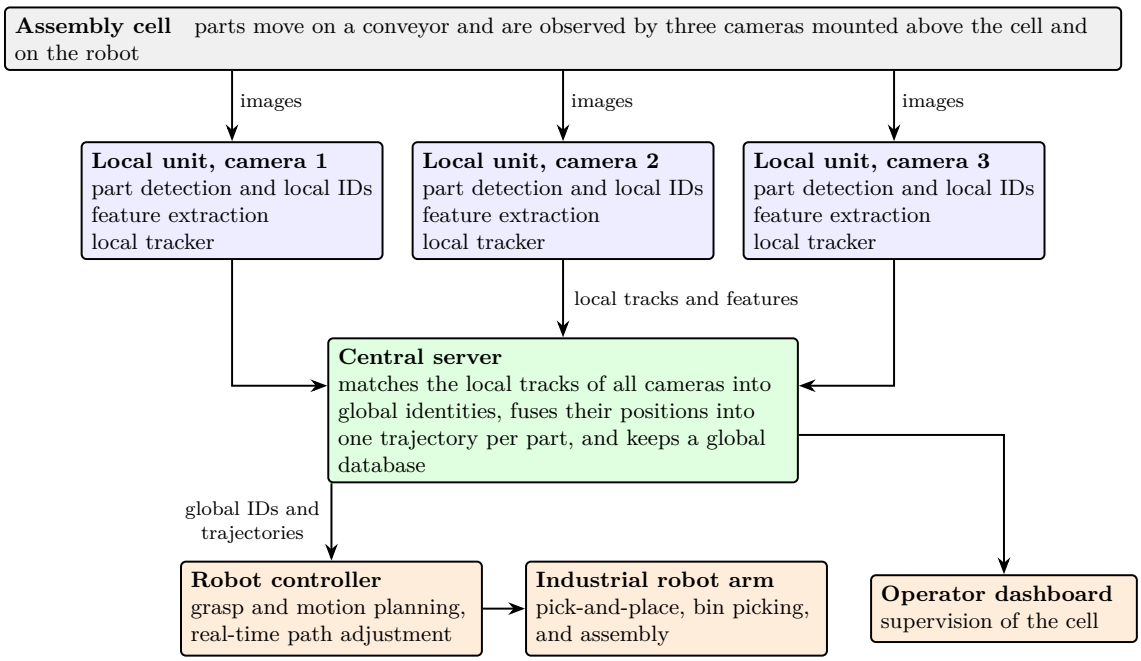


Figure 2.4: Multi-camera tracking of parts for automated assembly. Local units detect and track the parts seen by each camera; a central server fuses the local tracks into global identities and trajectories, which the robot controller uses to plan grasps and to adjust its path.

and few false alarms, must respect the privacy of patients, and often has only small annotated datasets.

2.2.6 Retail and Marketing Analytics

In retail, detection and tracking are used to count customers, to measure how they move through a store and how long they stay in front of displays, and to monitor shelves for missing or misplaced products. Shelf images contain many small, similar, and tightly packed products, which are hard to separate with standard detectors [47], and product recognition must handle thousands of classes with similar packaging that change frequently [48]. Automated checkout systems combine the tracking of customers with the recognition of the products they pick up. Figure 2.6 shows a system of this kind, in which the same local tracking serves both automated checkout and the analytics of the store. Occlusion by other customers and the protection of customers' privacy are the main practical concerns.

From applications to technical requirements. Across these domains, the same pipeline is asked to detect the objects of interest reliably, to keep their identities over time, and to do so within the delay and the computation that the platform allows. The domains differ in which of these demands is the hardest, and aerial video combines several of them: small and crowded objects, a moving camera, changing

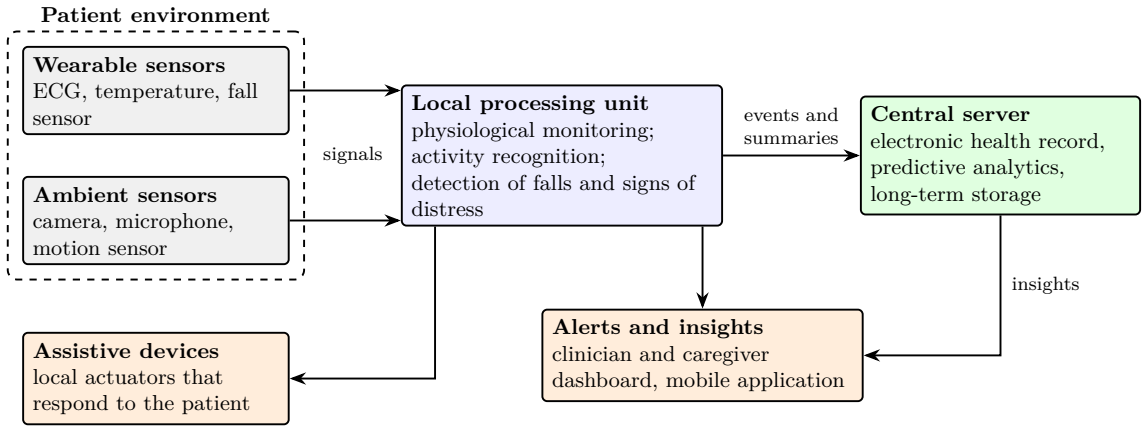


Figure 2.5: Monitoring system for patient safety and assistive care. Wearable and ambient sensors feed a local unit that recognizes activities and detects falls or signs of distress; alerts reach clinicians and caregivers, and a central server stores and analyzes the data.

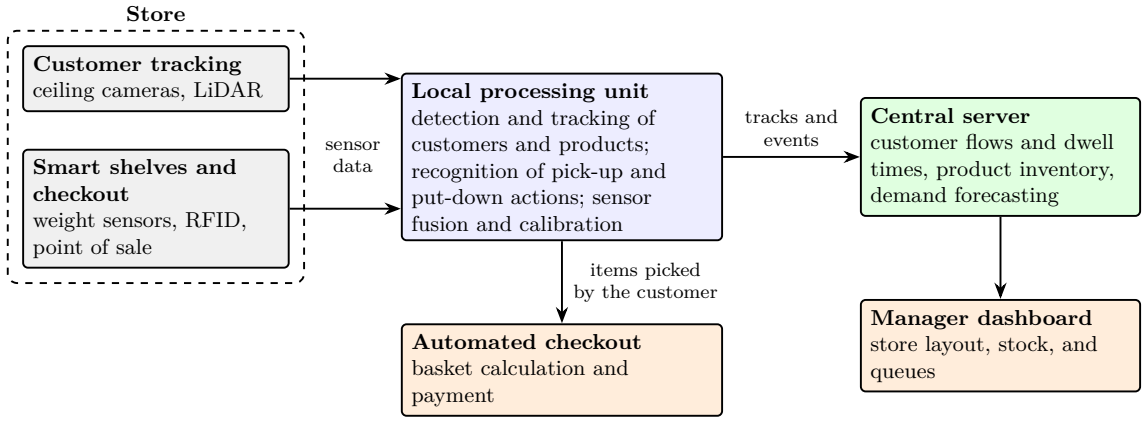


Figure 2.6: Retail analytics system for customer tracking and automated checkout. A local unit tracks customers and products and recognizes pick-up and put-down actions; the results drive the checkout and, through a central server, the analytics of the store.

conditions, and limited onboard computation. The next section introduces the technical background needed to address these demands: how detectors produce boxes and scores, how trackers use them, how the scores are read through thresholds, and how tracking accuracy is measured.

2.3 Technical Background

2.3.1 Object Detection Fundamentals

Object detection and tracking have evolved through three technological phases [1, 5]. Before 2012, detectors relied on hand-engineered features such as the histogram of oriented gradients [49] and scale-invariant keypoints [50], combined with sliding windows and shallow classifiers. The second phase began when CNNs trained on large labeled datasets [51, 52] outperformed hand-crafted features; region-based detectors [53, 2] and single-stage detectors [3, 54] pushed the accuracy–speed frontier, and deep residual networks [55] became standard backbones. The third phase, beginning around 2020, introduced attention-based models [56, 57]: the detection transformer (DETR) [4] removed hand-designed anchors and NMS, and transformer-based trackers integrated detection and association in one network [58, 59, 60].

Whatever its architecture, a detector maps an image I_t to a set of candidates

$$\mathcal{D}_t = \{(\mathbf{x}_{t,i}, s_{t,i}, c_{t,i})\}_{i=1}^{n_t}, \tag{2.1}$$

where $\mathbf{x}_{t,i}$ is a bounding box, $s_{t,i} \in (0, 1)$ a confidence score, and $c_{t,i}$ a class label [5]. The overlap of two boxes A and B is measured by their intersection over union (IoU),

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \tag{2.2}$$

the area of their intersection divided by the area of their union, which is 1 for identical boxes and 0 for disjoint ones.

Three settings, chosen by the user when the detector is run, decide which candidates leave the detector. The *confidence threshold* removes every candidate whose score is below it; a higher threshold passes fewer but more reliable boxes. *Non-maximum suppression* (NMS) removes duplicates: candidates are visited in order of decreasing score, and a candidate is removed when its IoU with a higher-scoring kept candidate exceeds the NMS IoU threshold, so a lower threshold removes more overlapping boxes. The *input resolution* is the size to which every frame is resized before inference; a larger input shows small objects better but increases the computation. Together, these three values form the detector operating point, the variable that AC-MOT controls in

Chapter 3.

Detection quality is commonly reported with the average precision (AP), the area under the precision–recall curve of one class at one IoU threshold, and its mean over classes and thresholds (mAP), together with the speed in frames per second (FPS) [61, 5]. These detection metrics judge every frame on its own; the tracking metrics used in this thesis are introduced in Section 2.3.8.

2.3.2 Deep Learning-Based Object Detectors

Object detectors are broadly divided into two-stage and single-stage architectures, with transformer-based detectors forming a third group [5]. Table 2.2 summarizes the detectors discussed in this section.

Two-stage detectors. The region-based CNN (R-CNN) family first proposes candidate regions and then classifies and refines them [53]. Faster R-CNN introduced a region proposal network that shares convolutional features with the detection head, which made the proposals almost cost-free and improved both speed and accuracy [2]. Mask R-CNN added a parallel branch that predicts a segmentation mask for each detection [62]. Two-stage detectors are accurate, particularly for small objects and cluttered scenes, but they are generally slower than single-stage detectors.

Single-stage detectors. Single-stage detectors predict boxes and class probabilities in one pass over the image. The YOLO (You Only Look Once) family treats detection as a regression problem over a grid [3]; successive versions improved the backbone, the neck, and the training recipe [63, 64, 16, 65], and recent generations distributed as software libraries, such as YOLOv8 [66] and YOLO11 [67], offer a range of model sizes. The Single Shot MultiBox Detector (SSD) predicts boxes from feature maps at several scales [54]. RetinaNet introduced the focal loss to address the imbalance between foreground and background during the training of dense detectors [68]. EfficientDet combined a bidirectional feature pyramid with compound scaling of resolution, depth, and width [69]. Feature pyramid networks [70] are a common architectural response to scale variation.

Transformer-based detectors. DETR formulates detection as set prediction with a transformer encoder–decoder, which removes anchors and NMS [4]. The original DETR converges slowly and is expensive. Real-time transformer detectors such as RT-DETR [71] and RF-DETR [72] keep the set-prediction formulation while reducing latency to the range of single-stage detectors.

Detectors used in this thesis. AC-MOT controls the YOLO11m detector [67] of the Ultralytics YOLO family, in a public version trained on VisDrone2019-DET [6]. SCL is evaluated on four detection sources: the published YOLOX-X detections of the MOT17 benchmark [16], the detections released by the authors of two external

Table 2.2: Detector families discussed in this thesis. The last column gives the thresholds that the user chooses when the detector is run.

Detector	Year	Architecture	Key idea	User thresholds
R-CNN [53]	2014	two-stage	CNN features on region proposals	confidence, NMS
Faster R-CNN [2]	2017	two-stage	shared region proposal network	confidence, NMS
Mask R-CNN [62]	2017	two-stage	parallel mask branch	confidence, NMS
YOLO [3]	2016	single-stage	grid regression of boxes and classes	confidence, NMS
SSD [54]	2016	single-stage	multi-scale default boxes	confidence, NMS
RetinaNet [68]	2017	single-stage	focal loss for class imbalance	confidence, NMS
EfficientDet [69]	2020	single-stage	bidirectional pyramid, compound scaling	confidence, NMS
YOLOX [16]	2021	single-stage	anchor-free, decoupled head	confidence, NMS
DETR [4]	2020	transformer	set prediction, no NMS	confidence
RT-DETR [71]	2024	transformer	real-time hybrid encoder	confidence

trackers, and two detectors that the trackers were not tuned for, YOLOv8n [66] and RT-DETR-L [71]. The three families produce different score distributions, which is the situation that SCL addresses.

2.3.3 Multi-Object Tracking Fundamentals

Multi-object tracking associates detections across time into trajectories $\mathcal{T} = \{\tau_j\}$, each a sequence of boxes of one object between a start frame and an end frame [5]. An online tracker processes the video frame by frame and solves, in each frame, an assignment problem

$$\pi_t^* = \arg\min_{\pi_t} \sum_{(i,j)\in\pi_t} C\left(\mathbf{x}_{t,i}, \tau_j\right), \tag{2.3}$$

where π_t is a one-to-one matching between the detections of frame t and the existing tracks, and C is a cost based on predicted motion, appearance, or learned embeddings. The Hungarian method [73] solves this assignment exactly, and a Kalman filter [74] is the usual motion predictor. The quality of $\mathcal{T}$ therefore depends on the detections that enter Eq. (2.3) as much as on the cost and the assignment: missed detections break trajectories, and false detections cause wrong associations and identity switches.

Core problems. In all the applications of Section 2.2, MOT faces three coupled problems: detecting the objects, associating detections across frames, and keeping identities through occlusion [1, 5].

Real-time constraints. Real-time performance is limited by three factors [35, 5]. The first is the accuracy–speed trade-off: more accurate models generally need more computation and longer inference. The second is the hardware, which ranges from data-center graphics processing units (GPUs) to low-power embedded processors on board a vehicle or an aircraft. The third is the deployment context: bandwidth, power supply, and the need for onboard rather than remote processing. In a tracking pipeline these constraints act mainly through the detector, whose input resolution sets most of the cost, while the tracker itself is usually inexpensive. A controller that changes the detector's resolution therefore also changes the cost of the pipeline, and a controller placed between the detector and the tracker must itself be inexpensive.

2.3.4 Tracking-by-Detection Architectures

Three paradigms organize the MOT literature [5, 11].

Tracking by detection. A detector is applied independently to every frame, and an online tracker associates the detections over time using motion and appearance cues [12, 75]. The paradigm benefits directly from advances in detectors and remains modular: any detector can feed any tracker. Its interface is the set of candidates of Eq. (2.1), filtered first by the detector operating point and then by the tracker's score

thresholds. It is the paradigm used throughout this thesis, because it is the one in which detectors and trackers are combined and exchanged independently, and therefore the one in which their settings must be reconciled.

Joint detection and tracking. Joint detection and embedding (JDE) learns detection and appearance embeddings in a shared model [76]; FairMOT treats detection and re-identification (ReID) as equally important tasks with an anchor-free design [77]; and CenterTrack predicts object centers and their displacements between frames [78]. These methods reduce the cost of a separate appearance model, but they couple the detector and the tracker, so that neither can be exchanged independently.

Transformer-based tracking. TransTrack propagates queries between frames [58]; TrackFormer separates object queries for new objects from track queries for existing ones [59]; MOTR carries track queries across frames and updates them by attention [60]; and MOTIP predicts identities directly [79]. These methods are fully differentiable, but they remain computationally expensive, and their thresholds are internal to the network.

2.3.5 Data Association and Track Management

A tracking-by-detection tracker repeats three operations in every frame: it predicts where each existing track should be, associates the new detections with these predictions, and manages the life of the tracks. A track is *born* from a detection that matches no track and whose score passes the birth threshold; it is *continued* by matched detections; and it is *lost* and finally removed when no detection has matched it for a given number of frames, called the track buffer. This section reviews the trackers used or discussed in the thesis, with attention to how each of them uses detector scores; Table 2.3 summarizes them.

Motion-based trackers. Simple online and realtime tracking (SORT) combines a constant-velocity Kalman filter with Hungarian assignment on box overlap [12]. It is simple and fast, but it relies on motion alone, so occlusion and irregular motion cause many identity switches. The IOU tracker shows how far pure overlap association can go at high frame rates [80]. ByteTrack introduced a second association step: detections above a high threshold are associated first, and the tracks that remain unmatched are then associated with low-confidence detections, which recovers partially occluded objects that other trackers discard [13]. OC-SORT refines the Kalman filter with an observation-centric re-update and a velocity-direction consistency term, which improves robustness to occlusion and non-linear motion [81]; it uses a single confidence threshold. Buffered IoU matching enlarges the boxes during association to handle irregular motion [82].

Motion and appearance trackers. DeepSORT adds deep appearance embeddings to

SORT and combines the Kalman prediction with the cosine distance between embeddings, which reduces identity switches [75]. BoT-SORT adds camera-motion compensation, an improved Kalman state, and ReID features to the ByteTrack association [83]. StrongSORT revisits DeepSORT with an exponential moving average of appearance features, camera-motion compensation by the enhanced correlation coefficient [84], and other refinements [85]. Deep OC-SORT adds adaptive appearance re-identification to OC-SORT [86], and SMILEtrack learns similarity for occlusion-aware association [87].

Recent trackers used in this thesis. Hybrid-SORT uses weak cues, such as the confidence state and the height of a box, alongside strong cues [88]. SparseTrack decomposes crowded scenes by pseudo-depth before association [89]. BoostTrack boosts the confidence of likely true detections and augments the similarity measure [90]. PD-SORT adds pseudo-depth cues to the motion model to handle occlusion [91]. U2MOT is an uncertainty-aware tracker trained without identity labels [92]. OATrack [93] is a recent tracker for small objects in UAV video; it weights association by detection quality, uses a YOLO detection cache, and has a fixed tracker-side confidence gate.

2.3.6 Detector Confidence Scores and Thresholding

How trackers use detector scores. All of the trackers above read detector scores through fixed thresholds that their authors tuned for one detector, usually YOLOX [16] on MOT17. Two kinds of trackers can be distinguished (Fig. 2.7). A *single-stage* tracker, such as OC-SORT, ignores every box below one threshold. A *two-stage* tracker, such as ByteTrack, uses boxes above a high threshold to match tracks and boxes above a higher threshold to start tracks, and it uses lower-score boxes, down to a low threshold, in a second step that can only continue tracks left unmatched by the first step. If a detector gives scores on a different scale from the one for which the thresholds were tuned, a single-stage tracker can lose most of its boxes, and a two-stage tracker can lose its ability to start tracks. Published comparisons show the steady progress of tracking accuracy from motion-only methods to trackers that combine motion, appearance, and low-score association [5]; they compare trackers on one fixed set of detections, however, and do not show how a tracker behaves when its detector changes.

Confidence and calibration. A classifier is calibrated when its confidence equals its probability of being correct. Modern neural networks are often poorly calibrated [17], and object detectors are no exception: their confidence depends on the position and size of the box as well as on the class, and it shifts with the data [18]. Detectors of different families [3, 2, 4, 71, 65] produce different score distributions, and soft suppression changes the scores of overlapping boxes [94]. The same object may therefore receive 0.9 from one detector and a much lower score from another. Calibration methods fit a mapping from scores to probabilities on labeled data [18], which is not possible when

Table 2.3: Tracking-by-detection trackers discussed in this thesis and how they use detector scores. The last column names the chapter in which a tracker is used.

Tracker	Year	Main cues	Score use	Used with
SORT [12]	2016	Kalman motion, IoU	one threshold	background
DeepSORT [75]	2017	motion, appearance	one threshold	background
ByteTrack [13]	2022	motion, IoU, low-score recovery	two stages	AC-MOT, SCL
BoT-SORT [83]	2022	motion, camera motion, ReID	two stages	SCL
OC-SORT [81]	2023	observation-centric motion	one threshold	SCL
StrongSORT [85]	2023	motion, appearance, camera motion	one threshold	background
U2MOT [92]	2023	uncertainty-aware association	published static setting	AC-MOT development
Hybrid-SORT [88]	2024	strong and weak cues	two stages	SCL
BoostTrack [90]	2024	confidence boost, similarity	two stages	SCL
SparseTrack [89]	2025	pseudo-depth decomposition	two stages	AC-MOT development, SCL
PD-SORT [91]	2025	pseudo-depth motion	one threshold	SCL
OATrack [93]	2026	quality-weighted association	confidence gate	AC-MOT

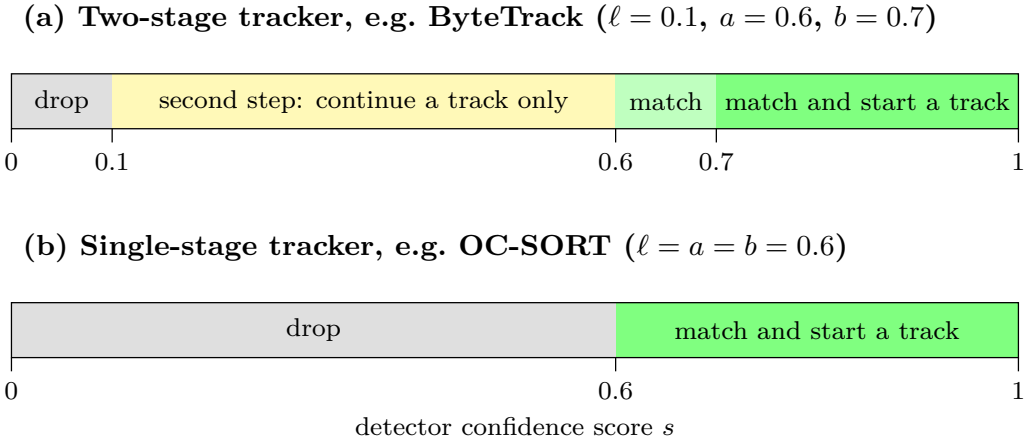


Figure 2.7: How a two-stage and a single-stage tracker use the detector score, with the published thresholds of ByteTrack (official setting) and OC-SORT. ℓ: lowest score used; a: score needed to match a track; b: score needed to start a track.

a detector is exchanged in a deployed pipeline without labels. Test-time adaptation methods adapt a model to unlabeled data [95], but they change the model's weights, whereas this thesis keeps both the detector and the tracker frozen.

Logits. A score $s \in (0, 1)$ can be written as its logit, the log-odds

$$z = \log \frac{s}{1 - s}, \tag{2.4}$$

which keeps the order of the scores and spreads out the regions near 0 and 1 where detector scores crowd; the logistic function $\sigma(z) = 1/(1 + e^{-z})$ maps a logit back to a score. A common change of a detector's scores—a change of temperature—divides all logits by the same number.

Label-free thresholding. Classical image thresholding offers a tool that needs no labels. Otsu's method chooses the threshold that best splits a histogram into two groups [26]. For a candidate threshold k, let $\omega_0(k)$ and $\omega_1(k)$ be the fractions of values below and above k, and $\mu_0(k)$ and $\mu_1(k)$ their means. The method selects

$$k^* = \arg\max_k \; \omega_0(k)\,\omega_1(k)\,\Big(\mu_0(k) - \mu_1(k)\Big)^2, \tag{2.5}$$

the threshold that maximizes the between-class variance. It adapts to the shape of the distribution, and it moves with any stretch or shift of the values. SCL applies it to the logits of recent detector scores (Chapter 4).

2.3.7 Aerial Tracking Challenges and Datasets

Difficulties of aerial video. Aerial video adds small targets, changing object counts, camera motion, and clutter to the general difficulties of MOT [6, 9]. Small objects cover few pixels and receive low detector confidence [10]; the number of objects changes quickly in traffic; the moving camera changes position, altitude, and viewing angle; and illumination, background, and occlusion vary within one video. These properties make the detector operating point especially important: a larger input helps small objects, while the confidence and NMS thresholds change how many detections reach the tracker.

Datasets of the field. Detection models are commonly trained on COCO (Common Objects in Context), which contains more than 330,000 images with annotations for 80 object categories [61], and ImageNet provides the large-scale classification data on which most backbones are pretrained [52]. For tracking, the MOTChallenge benchmarks MOT16 and MOT17 [96, 97] contain pedestrian sequences from static and moving cameras, and MOT20 contains very crowded scenes [98]. DanceTrack contains dancers with uniform appearance and diverse motion, which makes association by appearance difficult [99]. The four datasets used in this thesis are described below, and

Table 2.4 lists the splits and the role of each one.

VisDrone2019-MOT. VisDrone2019-MOT is a widely used benchmark for MOT in UAV videos [6]. It contains video sequences recorded by various drone-mounted cameras in 14 cities in China, in urban and rural regions and in both sparse and crowded scenes. The VisDrone benchmark annotates ten object categories, such as pedestrians, cars, buses, and bicycles, with boxes that give the location and identity of each tracked object. Its MOT track uses 96 video clips: 56 for training (24,198 frames), 7 for validation (2,846 frames), 16 for challenge testing (6,322 frames), and 17 for development testing (6,635 frames). Although ten categories are annotated, the official MOT evaluation scores only five of them: pedestrian, car, van, bus, and truck [6].

UAVDT. UAVDT is a benchmark designed for object detection and MOT in UAV videos, with a particular focus on vehicles [9]. It consists of 100 video sequences selected from more than 10 hours of video recorded at urban locations such as squares, arterial streets, toll stations, highways, crossings, and T-junctions, at 30 frames per second and a resolution of 1080×540 pixels. About 80,000 frames are annotated with 0.84 million boxes of vehicles in three categories: car, truck, and bus. The sequences are recorded under different weather conditions, flying altitudes, and camera viewing angles, which change object size, visibility, and the number of vehicles per frame. The test set contains 20 sequences for the MOT task (16,592 frames) [9].

MOT17. MOT17 [96] contains 14 sequences of people filmed with static and moving cameras. As is common, the second half of each of the seven training sequences, called val-half (2,652 frames), is used for evaluation [13]. The benchmark distributes detections of a published YOLOX-X detector [16] that are used by many trackers.

KITTI tracking. KITTI tracking [8] has 21 training sequences (8,008 frames) recorded from a car and is scored on cars and pedestrians.

2.3.8 Evaluation Metrics and Experimental Protocols

Eight commonly used MOT metrics are considered: HOTA, MOTA, IDF1, IDS, false positives (FP), false negatives (FN), precision, and recall. They measure different aspects of tracking performance, including detection accuracy, the consistency of object identities, and the ability to follow objects across consecutive frames [97, 19]. They are defined below, starting from the errors counted in each frame and ending with the scores that combine them.

Detection errors. In each frame t, output boxes are matched one to one to ground-truth boxes with an IoU of at least 0.5. Let GT_t be the number of ground-truth boxes, TP_t the number of matched pairs (true positives), FP_t the number of output boxes without a match, and FN_t the number of ground-truth boxes without a match (missed

Table 2.4: Datasets and splits used in this thesis and the role of each one.

Dataset	Split	Controller	Role
VisDrone2019-MOT	training, 8 sequences (3,282 frames)	AC-MOT	calibration (tuning) data
VisDrone2019-MOT	validation, 7 sequences (2,846 frames)	AC-MOT	frozen evaluation
UAVDT	test, 20 sequences (16,592 frames)	AC-MOT	transfer without retuning
MOT17	val-half, 7 sequences (2,652 frames)	SCL	development; predeclared external trackers
KITTI tracking	training, 21 sequences (8,008 frames)	SCL	development under a detector change
UAVDT	test, 20 sequences (16,592 frames)	SCL	held-out evaluation, run once

objects). Over all frames,

$$\mathrm{GT} = \sum_t \mathrm{GT}_t, \qquad \mathrm{TP} = \sum_t \mathrm{TP}_t, \tag{2.6}$$

$$\mathrm{FP} = \sum_t \mathrm{FP}_t, \qquad \mathrm{FN} = \sum_t \mathrm{FN}_t, \tag{2.7}$$

so that $\mathrm{TP} = \mathrm{GT} - \mathrm{FN}$. Precision is the fraction of output boxes that are correct, and recall is the fraction of ground-truth boxes that are found:

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad \mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{GT}}. \tag{2.8}$$

Identity switches. The counts above treat each frame separately, so they do not show whether an object keeps the same track over time. Let $\phi_t(i)$ be the output track matched to ground-truth object i in frame t, and $\phi_{\mathrm{prev}}(i)$ the output track to which object i was last matched before t. An identity switch is counted whenever these two tracks differ:

$$\mathrm{IDS} = \sum_t \sum_i \mathbf{1}[\phi_t(i) \neq \phi_{\mathrm{prev}}(i)]\,, \tag{2.9}$$

where the inner sum runs over the objects that are matched in frame t and were matched before, and the indicator $\mathbf{1}[\cdot]$ is 1 when its condition holds and 0 otherwise.

Figure 2.8 shows these outcomes on two consecutive frames of an aerial video of VisDrone2019-MOT, tracked by ByteTrack with the YOLO11m detector at a fixed operating point. Each track box carries the ID of its track, and the line behind it shows where the track was in the last 20 frames. A red box is a false positive, a dashed box is a missed object, and the ringed pedestrian is matched to track 1497 in frame 47 and to the new track 1693 in frame 48, which counts as one identity switch.

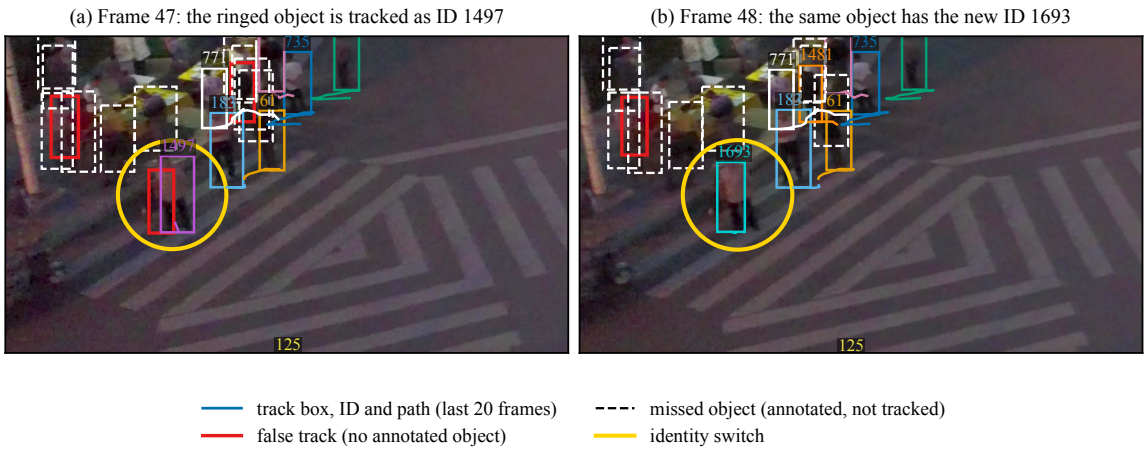


Figure 2.8: Outcomes counted by the MOT metrics, on frames 47 and 48 of the VisDrone2019-MOT sequence uav0000117_02622_v, tracked by ByteTrack with YOLO11m at the fixed operating point (1536, 0.70). Colored boxes: tracks with their IDs and their paths over the last 20 frames; red: false positives; white dashed: missed objects; yellow ring: an object whose track ID changes (identity switch). Both panels show the same 960×540 window.

MOTA and IDF1. These two scores summarize the errors above in one number each [100, 30]. MOTA combines the three error types, and IDF1 measures how long each object keeps the correct identity:

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}, \tag{2.10}$$

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}. \tag{2.11}$$

For IDF1, each ground-truth track is first matched to at most one output track over the whole video; IDTP, IDFP, and IDFN are then the true-positive, false-positive, and false-negative boxes under this track-level matching. Because MOTA counts every false positive and every missed object, it reacts strongly to how many boxes the detector passes on, and it becomes negative when the errors outnumber the ground-truth boxes. IDF1 depends more on whether each object keeps one identity.

HOTA. MOTA depends mostly on detection errors, and IDF1 mostly on identity errors. HOTA balances detection and association in one score [19]. At a localization threshold α, let TP_α, FN_α, and FP_α be the matched pairs, missed ground-truth boxes, and unmatched output boxes. For a matched pair c, $\mathrm{TPA}(c)$ is the set of matched pairs with the same ground-truth identity and the same output identity as c; $\mathrm{FNA}(c)$ is the set of boxes of the same ground-truth object that are matched to another output identity or not matched; and $\mathrm{FPA}(c)$ is the set of boxes of the same output track that

are matched to another object or not matched. Then

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}_\alpha|}{|\mathrm{TP}_\alpha| + |\mathrm{FN}_\alpha| + |\mathrm{FP}_\alpha|}, \tag{2.12}$$

$$\mathcal{A}(c) = \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}, \tag{2.13}$$

$$\mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}_\alpha|} \sum_{c \in \mathrm{TP}_\alpha} \mathcal{A}(c), \tag{2.14}$$

$$\mathrm{HOTA} = \frac{1}{19} \sum_{\alpha} \sqrt{\mathrm{DetA}_\alpha \, \mathrm{AssA}_\alpha}, \tag{2.15}$$

where α runs over $0.05, 0.10, \ldots, 0.95$; DetA is the detection accuracy and AssA the association accuracy.

Units and direction. HOTA, MOTA, IDF1, precision, and recall are given in percent, and differences in points; IDS, FP, and FN are counts; speed is given in FPS, and runtime in milliseconds (ms) per frame. Higher values are better for HOTA, MOTA, IDF1, precision, recall, and FPS, and lower values are better for IDS, FP, FN, and runtime.

Paired comparison and bootstrap intervals. Every comparison in this thesis is paired: the same tracker runs with fixed settings and with a controller on the same sequences and detections, and nothing else changes. To judge whether the difference between two systems is reliable, paired bootstrap resampling is applied at the sequence level [101], which keeps the dependence within each sequence. Each resample draws the evaluation sequences with replacement; the same draw is used for both systems, the pooled metrics are recomputed, and their difference is recorded. The 95% confidence interval (CI) is the 2.5–97.5 percentile range, and a difference is called reliable (significant) when its interval does not include zero. Because whole sequences are resampled, the intervals reflect the variation between videos, not only between frames. The number of resamples is given in each chapter.

Development, calibration, and evaluation data. Both controllers follow the same protocol. Their values are chosen only on development or calibration data, the configuration is then frozen, and only the frozen configuration is run on the evaluation data. The UAVDT test split, used by both controllers, and the predeclared external trackers of SCL were run once with the frozen configuration. Detector and tracker parameters affect accuracy, identity consistency, and speed in conflicting ways, so the development of AC-MOT used systematic search with Optuna and its tree-structured Parzen estimator (TPE) sampler [102], on calibration data only.

2.4 Related Work

This section reviews the work closest to the two controllers, organized by theme rather than by publication: the tracking-by-detection systems that the controllers serve, methods that adapt the detector, methods that adapt the thresholds of a tracker, and methods that calibrate scores or connect different trackers to a detector. It ends with a comparison of the closest methods and the research gap that this thesis addresses.

2.4.1 Tracking-by-Detection Systems and Aerial MOT

In tracking by detection, objects are detected in each frame and associated across frames, and most methods improve motion prediction or association for a given detector. SORT builds a compact online pipeline from motion prediction and assignment [12]; StrongSORT adds appearance features to reduce identity fragmentation [85]; ByteTrack associates high-confidence detections first and then uses low-confidence boxes to recover objects that would otherwise be lost [13]; and OC-SORT uses one score threshold with an observation-centric motion model [81]. Many recent trackers improve association further with trained or domain-specific parts [103, 104, 105, 106, 107, 108]. These methods change or retrain the tracker; the thresholds through which they read the detector remain fixed values tuned for one detector, usually YOLOX [16].

The VisDrone [6] and UAVDT [9] benchmarks established large annotated drone video collections and documented the difficulties of the aerial domain, and surveys of deep learning for drone-based detection and tracking [7] and of computer vision for UAVs [34] summarize the resulting methods. UAVMOT addresses the motion of the UAV camera inside the tracker [14], and appearance-aided joint probabilistic data association (AF-JPDA) has been used for multiclass MOT in cluttered aerial images, mainly to reduce identity switches and track fragmentation [109]. Small objects are a central difficulty: detectors that keep features at high resolution have been designed for small objects on the airport surface [110] and for tiny objects in remote sensing imagery [111], and LSO-YOLO designs a lightweight real-time detector for small UAV objects [41]. OATrack weights association by detection quality for small UAV objects [93]. All of these works improve the tracker or the detector itself. The input resolution, the confidence threshold, and the NMS threshold of the detector, and the score thresholds of the tracker, remain fixed settings, which is where the controllers of this thesis act.

2.4.2 Adaptive Inference and Detector Adaptation

Dynamic neural networks change their structure or parameters for each input [20], and resolution-adaptive networks spend less computation on easy inputs [112]. AdaScale selects the input scale of each video frame with a scale regressor trained on detector

features [21]. ApproxDet switches detection configurations at run time with accuracy and latency models trained offline [22]. Chameleon re-profiles the configuration of video analytics pipelines over time [113]. Test-time adaptation updates the weights of a model on unlabeled test data [95]. These methods trade detection accuracy against cost or adapt the model itself, and they select a configuration with additional trained models, periodic profiling, or weight updates. They are designed for detection or video analytics, not for MOT, where a detection error can also break an association or switch an identity. AC-MOT differs in three respects: it adjusts the confidence threshold, the NMS threshold, and the input resolution together for better tracking rather than better detection; it needs no learned model; and it is evaluated with tracking metrics on two trackers.

2.4.3 Adaptive Confidence and Threshold Methods in MOT

Fixed tracker thresholds decide which detections are accepted and how they are associated, but their reliability changes with object density, occlusion, and detection confidence, so several methods adapt them. Adaptive ByteTrack sets ByteTrack's confidence threshold from the detections of each frame [23], and AdapTrack adapts the matching thresholds of its data association [24]. ACT sets the high and low thresholds of the tracker from the first quartile and the number of the current scores, choosing between preset levels [25]. ConfTrack weights the Kalman update and the matching cost by the score [114], UTrack uses the spread of the boxes merged by NMS as a measure of the uncertainty of each detection [115], and PlugTrack blends motion predictors with a trained network [116]. BoostTrack, one of the trackers used in this thesis, boosts the confidence of likely true detections inside its own association [90]. These methods are built into one tracker, read only the current frame or each box, act in every frame, or need trained parts. AC-MOT leaves the tracker's thresholds unchanged and adapts the detector instead; SCL adapts the tracker's input and thresholds from outside the tracker, first decides from past scores whether to act at all, and serves several unchanged trackers.

Threshold rules compared. The threshold controllers compared in Chapter 4 differ in the information from which they derive the thresholds. Let S_t be the scores of the candidates of frame t, Z_t^W the logits of the candidates of the last W frames, t_1 and t_2 the two nested Otsu splits of Z_t^W (Eq. (2.5)), σ the logistic function, and a, b the published thresholds of the tracker for matching and for starting a track. ACT derives the high and low thresholds from the first quartile Q_1 and the number of the current scores, with a rule $\mathcal{A}$ that chooses between preset levels [25]:

$$\left(\tau_t^{\text{high}}, \tau_t^{\text{low}}\right) = \mathcal{A}\left(Q_1(S_t),\, |S_t|\right). \tag{2.16}$$

A simple sliding-window controller sets both thresholds to one Otsu split of the recent logits and passes the scores unchanged:

$$a_t = b_t = \sigma(t_k), \qquad k \in \{1, 2\}. \tag{2.17}$$

SCL first decides the mode of the stream from the median of the confident share ρ over all frames so far (Section 4.3.4). In the clean mode it keeps the published thresholds unless they would reject the confident group, $a_t = \min(a, \sigma(t_2))$ and $b_t = \min(b, \sigma(t_2))$ (Eq. (4.4)); in the noisy mode it drops the background candidates, remaps the remaining scores into fixed ranges, and sets $a_t = b_t = 0.5$ (Eq. (4.5)). ACT thus reads only the current frame and acts in every frame; the window controller reads the last W frames and also acts in every frame; SCL reads the last W frames for its splits, the whole stream for its mode, and the tracks of the previous frame for the recovery of weak detections (Section 4.3.6), and it leaves a well-matched tracker at its own thresholds. SCL pools the candidates of all classes in its splits and uses the class only to remove cross-class duplicates.

2.4.4 Score Calibration and Heterogeneous Tracker Interfaces

Score calibration maps detector confidence to the probability that a detection is correct. Temperature scaling and related methods calibrate classifiers [17], and multivariate calibration extends this to object detectors by using the position and size of each box [18]. Both fit the mapping on labeled data for one detector, and their goal is a correct probability, not a score that a particular tracker can use. Soft-NMS lowers the scores of overlapping boxes instead of removing them [94], which itself changes the score distribution that the tracker receives.

Trackers also differ in how they read scores. A single-stage tracker uses one threshold, a two-stage tracker uses separate thresholds for matching, birth, and a low-score step, and published configurations set these values for one detector (Section 2.3.6). There is no shared interface through which an external component could adapt the thresholds of different trackers in the same way: each adaptive-threshold method of Section 2.4.3 is written for one tracker. Label-free tools that follow the distribution of the data, such as Otsu's method [26], offer a way to read the score scale of a detector without labels. SCL combines the two observations: it reads the score distribution with nested Otsu splits and acts through four published values of each tracker, its host contract (Chapter 4). Its remapping of scores is not probability calibration: it places the scores in fixed ranges that a tracker's own thresholds read, without labels.

2.4.5 Comparison and Research Gap

Table 2.5 compares the methods closest to the proposed controllers by what they adapt, where they act, and the requirement of this thesis that they do not meet. The review points to a gap between how tracking methods are developed and how they are deployed. Detectors and trackers are developed and benchmarked with fixed settings tuned for one detector and one kind of scene. In deployment, scenes change within a video, and detectors are exchanged more often than trackers are retuned. Adaptive inference systems learn or profile a policy for each model and are designed for detection, adaptive-threshold trackers adapt one tracker to the current frame, and calibration needs labels. What is missing is (i) a training-free way to control the detector operating point online in changing aerial scenes, judged with tracking metrics, and (ii) a training-free and tracker-agnostic way to keep published trackers working when the detector's score scale changes and that leaves a tracker unchanged when no change is needed. AC-MOT (Chapter 3) addresses the first point and SCL (Chapter 4) the second.

Table 2.6 groups the related methods by family and answers, for each family, four questions that matter for the proposed controllers: whether it adapts the detector settings, whether it needs trained parts or labeled data, whether it changes the tracker, and whether it can be used when the detector or the tracker is exchanged. Only the two proposed controllers adapt part of the operating point without trained parts or labels while leaving both the detector and the tracker unchanged, and both are judged with tracking metrics.

Novelty of the proposed controllers. The contribution of the two controllers can be separated into three levels. (i) *Established methods used as components:* Otsu's method [26], the logit transform, the variance of the Laplacian [117], phase correlation [118], the published trackers with their published thresholds, the TPE search [102], cross-validation, and the paired bootstrap [101]. (ii) *New combinations of existing methods:* AC-MOT joins inexpensive image and track cues with a three-state controller that sets the confidence threshold, the NMS threshold, and the input resolution of the detector together, for tracking rather than for detection; SCL joins nested Otsu splits of past logits with the published thresholds of each tracker through its host contract. (iii) *New control rules and experimental findings:* for AC-MOT, a causal state machine on a temporal scene complexity index, with recovery, hysteresis, and dwell rules, whose thresholds and actions were selected with tracking metrics under cross-validation and then frozen, together with the finding that the detector operating point raises MOTA by 17.995 points with the same detector and tracker; for SCL, the clean or noisy decision by a whole-stream majority rule that leaves a well-matched pair unchanged, the rank remap that places scores in the ranges read by the tracker's own thresholds, the track-guided recovery that gives single-stage trackers a continuation step, and the motion-

Table 2.5: Methods closest to the proposed controllers: what each method contributes, the part of the pipeline it adapts, and the requirement of this thesis that it does not meet (training-free, label-free, external to unchanged components, judged by tracking metrics); –: all requirements met.

Year	Method	Contribution	Adapted part	Requirement not met
2018	Chameleon [113]	re-profiles the configuration of video analytics pipelines over time	pipeline configuration	periodic profiling; video analytics, not MOT
2019	AdaScale [21]	selects the input scale of each frame with a scale regressor trained on detector features	detector input scale	trained regressor; detection, not tracking
2020	ApproxDet [22]	switches detection configurations with accuracy and latency models trained offline	detector configuration	models trained offline; detection, not tracking
2020	RANet [112]	resolution-adaptive network that spends less computation on easy inputs	network resolution	changes and trains the network
2020	Multivariate calibration [18]	calibrates detector confidence with box position and size	detector scores	needs labeled data
2022	UAVMOT [14]	models the motion of the UAV camera inside the tracker	tracker motion model	changes the tracker; detector settings fixed
2023	Adaptive ByteTrack [23]	sets ByteTrack's confidence threshold from the detections of each frame	tracker confidence threshold	built into one tracker; current frame only
2024	AdapTrack [24]	adaptive thresholding in the matching step	matching thresholds	built into its own tracker; ReID network
2024	ConfTrack [114]	weights the Kalman update and the matching cost by the score	Kalman update, matching cost	built into its own tracker; ReID network
2024	AF-JPDA [109]	appearance-aided probabilistic association for multiclass aerial MOT	data association	changes the tracker; detector settings fixed
2025	UTrack [115]	uses the spread of the boxes merged by NMS as detection uncertainty	association	built into one tracker
2026	ACT [25]	sets high and low thresholds from the first quartile and the number of the current scores	tracker high and low thresholds	current frame only; preset score levels
2026	PlugTrack [116]	blends motion predictors with a trained network	motion prediction	needs trained parts
2026	OATrack [93]	quality-weighted association for small UAV objects	association weights	fixed confidence gate; detector settings fixed
–	AC-MOT (Chapter 3)	adapts confidence, NMS, and input resolution online from scene cues	detector operating point	–
–	SCL (Chapter 4)	checks and restores score–threshold compatibility from past score logits	candidates, scores, tracker thresholds	–

adaptive matching tolerance, together with the finding that one frozen configuration restores two trackers from 0 to about 66 HOTA under halved scores and serves seven published trackers.

The algorithmic differences from the closest methods are specific. AdaScale predicts a scale for every frame with a regressor trained on detector features [21], ApproxDet chooses configurations with accuracy and latency models trained offline [22], and Chameleon re-profiles configurations periodically [113]; AC-MOT has no learned model, decides every tenth frame from an index smoothed over time with hysteresis and dwell, and selects joint operating points by tracking metrics. Adaptive ByteTrack and ACT compute thresholds from the scores of the current frame, in every frame, inside one tracker [23, 25]; SCL pools the logits of the last ten frames, decides from the whole stream whether to act at all, acts through the published settings of each host, and recovers weak detections from the tracks of the previous frame (Eqs. (2.16)–(2.17) and Section 2.4.3).

2.5 Summary

This chapter first reviewed the main application domains of real-time object detection and tracking, with emphasis on aerial video, and then introduced the technical foundation of the thesis. A detector returns boxes with confidence scores, and three settings—the confidence threshold, the NMS threshold, and the input resolution—decide which of them leave the detector. A tracking-by-detection tracker then reads these boxes through fixed score thresholds, in one stage or in two, and builds tracks by prediction, association, and track management. Detector scores are not calibrated across detectors, and aerial video changes the scene within a video, so both families of settings can stop fitting the situation for which they were tuned. The chapter also defined the datasets, the eight evaluation metrics, and the paired bootstrap protocol used in the following chapters. The review of related work showed that existing adaptive methods either learn a policy for one detector, adapt one tracker from inside, or need labels. The next chapter presents the first controller of the thesis, AC-MOT, which adapts the detector operating point online in aerial video.

Table 2.6: Families of related methods and the four questions that matter for the proposed controllers: does the family adapt the detector settings, does it need trained parts or labeled data, does it change the tracker, and can it be used when the detector or the tracker is exchanged.

Family	Adapts detector settings	Trained parts or labels	Changes the tracker	Detector or tracker exchanged
Adaptive inference [21, 22, 113, 112]	yes: input scale, network resolution, or configuration	trained models or periodic profiling	no tracker	policy built for one model
Test-time adaptation [95]	no: updates the weights	weight updates on test data	no tracker	one model
Adaptive tracker thresholds [23, 24, 25]	no	none, except the ReID network of AdapTrack	yes: built into one tracker	one tracker
Score-aware association [114, 115, 90, 93]	no	none, except the ReID network of ConfTrack	yes: inside the association	one tracker
Score calibration [17, 18]	no: maps the scores	labeled data	no	one mapping per detector
Aerial tracker designs [14, 109, 116]	no: detector settings fixed	trained network in PlugTrack	yes: new tracker parts	one tracker
AC-MOT (Chapter 3)	yes: confidence, NMS, input resolution	none: values fixed before deployment	no	unchanged trackers; detector exposing the three settings
SCL (Chapter 4)	no: acts on the tracker input	none: standard values	no: published settings only	one configuration for seven trackers and four detection sources

Chapter 3

AC-MOT: Adaptive Detector Operating-Point Control

3.1 Introduction

Multi-object tracking in UAV video is difficult because object density, size, illumination, and blur change over time, while most tracking-by-detection systems run the detector with fixed settings (Chapter 1). This chapter presents the first controller of the thesis, Adaptive Control for Multi-Object Tracking (AC-MOT), a training-free controller that adjusts the detector operating point online: the confidence threshold, the IoU threshold of NMS, and the input resolution. Using only the current frame and the past output of the tracker, it selects one of three operating points without modifying or retraining the detector or the tracker. The chapter is based on the manuscript submitted to the Journal of Aerospace Information Systems [27].

The evaluation shows that this simple control has a large effect. On VisDrone2019-MOT with ByteTrack, the controller raised, relative to a fixed operating point, HOTA by 7.1%, MOTA by 182%, and IDF1 by 11.0%, and it reduced identity switches by 24.2%. With OATrack, the same three scores rose by 2.8%, 22.4%, and 4.0%, and identity switches fell by 21.9%. Paired bootstrap analysis supports these improvements. On a second aerial benchmark, UAVDT, the controller raised MOTA with both trackers without retuning, including a 16.7% gain with OATrack, and it adds about 4.2 ms per frame on a Tesla T4 GPU.

Section 3.2 formulates the control problem and the requirements of the controller. Section 3.3 presents the controller: its scene cues, the scene complexity index, the three-state controller, the action map, and the selection of its values. Section 3.4 explains how optimization experiments on calibration data led to the frozen controller. Section 3.5 describes the experimental setup, Section 3.6 gives the results on VisDrone2019-MOT and UAVDT, Section 3.7 discusses them, and Section 3.8 summarizes the chapter.

3.2 Problem Formulation and Design Requirements

The detector settings at frame t are called the operating point

$$\theta_t = (c_t, \eta_t, r_t), \tag{3.1}$$

which has three parts (Section 2.3.1):

- r_t is the input resolution, the long side of the resized image in pixels. A larger input shows small objects better but takes more time.
- η_t is the NMS IoU threshold. When two boxes overlap with an IoU above η_t, the box with the lower score is removed. A lower η_t removes more overlapping boxes.
- c_t is the confidence threshold, also called the score threshold. Boxes with a detector score below c_t are removed before they reach the tracker. A higher c_t passes fewer but more reliable boxes.

The detector weights and the tracker are frozen; the controller changes only θ_t. It uses only information that is available at run time:

$$\theta_t = \pi\left(I_t, \mathcal{B}_{1:t-1}\right), \tag{3.2}$$

where π is the control rule, I_t is the image of the current frame, and $\mathcal{B}_{1:t-1}$ are the boxes that the tracker reported for the earlier frames 1 to $t-1$. The tracker output of frame t is used only from frame $t+1$ on. No ground truth, no future frame, and no detector score enter the decision.

The rule π belongs to a family of rules with a small number of values (Section 3.3.5), and the operating points it may select come from a finite set of supported values. Its quality is judged by tracking, not by detection: in the final development stage, each candidate rule was scored by the cross-validated mean change in HOTA over a fixed operating point on separate calibration sequences,

$$J(\pi) = \frac{1}{F}\sum_{f=1}^{F}\left(\mathrm{HOTA}_f(\pi) - \mathrm{HOTA}_f(\theta_0)\right), \tag{3.3}$$

where f runs over the $F = 4$ cross-validation folds, HOTA_f is measured on the sequences held out in fold f, and θ_0 is the fixed baseline operating point. The frozen rule was then chosen among the candidates that really adapt within the video (Section 3.4.4).

The controller must meet five requirements, which follow from the challenges of Section 1.2:

1. *Training-free and label-free:* no learned model and no ground truth at run time.
2. *Causal:* the decision for frame t uses only frames up to t and tracker output up to $t-1$ (Eq. (3.2)).
3. *External:* the detector and the tracker are used as they are; only three arguments of the detector call change.
4. *Stable:* the operating point follows lasting changes of the scene, not single frames.
5. *Inexpensive:* the cost of the controller is small compared with the detector.

3.3 The AC-MOT Controller

AC-MOT is a small, rule-based controller that runs next to the detector. On frame 1 and then on every tenth frame, it measures inexpensive cues of the current image and of the boxes tracked in the previous frame (Section 3.3.1), averages the recent crowd level into a scene complexity index (SCI) (Section 3.3.2), chooses a target state (LOW, MEDIUM, or HIGH), adjusts it with a recovery probe and with hysteresis and dwell rules (Section 3.3.3), and maps the state to one of three fixed detector operating points (Section 3.3.4). Section 3.3.5 explains how each value is chosen. Figure 3.1 shows where AC-MOT sits in the pipeline: the controller chooses the detector settings before the detector runs, and the detector and the tracker then run as usual.

3.3.1 Scene Cues

The controller needs an inexpensive description of the scene. Scene analysis therefore does not run on every frame. It runs on frame 1 and then once every 10 frames ($t = 1$ or $t \bmod 10 = 1$, that is, frames 1, 11, 21, ...). Each of these frames is an analysis step

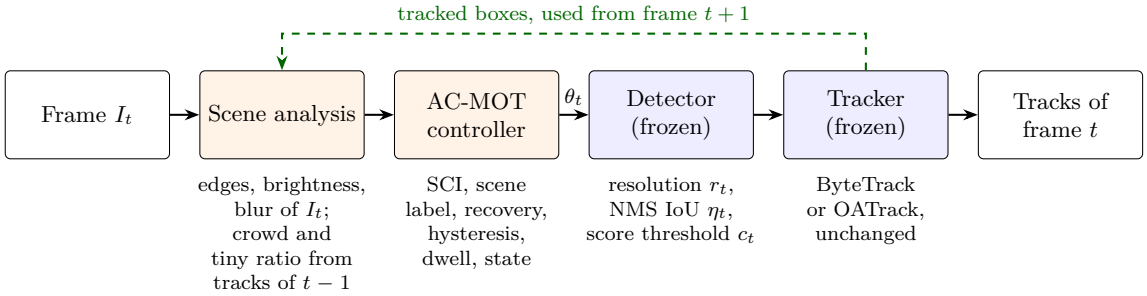


Figure 3.1: AC-MOT in a tracking-by-detection pipeline. Scene analysis and the controller choose the operating point $\theta_t = (c_t, \eta_t, r_t)$ before the detector runs. The frozen detector and tracker are not modified. Tracked boxes of frame t feed the scene analysis of later frames (dashed arrow).

j. Between two analysis steps the controller keeps its last decision. Two kinds of cues are computed at each step: image cues, which describe how the current frame looks, and box cues, which describe the objects that the tracker found in the previous frame.

Image cues of the current frame. The frame is reduced to a quarter of its width and height and converted to grayscale, so that the cues are inexpensive to compute. Three cues are then measured:

- the edge density e_t, the fraction of pixels marked as edges by the Canny detector (lower and upper thresholds 50 and 120); many edges indicate a cluttered scene;
- the brightness β_t, the mean gray level from 0 (black) to 255 (white); a low value indicates a dark or night scene;
- the sharpness ν_t, the variance of the Laplacian of the grayscale image [117]; a low value indicates a blurred frame.

Box cues of the previous frame. Let N_j be the number of boxes that the tracker reported for frame $t-1$, and let a_i be the area of box i in pixels of the original image. The crowd cue and the tiny-object ratio are

$$C_j = \min\left(\frac{N_j}{30}, 1\right), \tag{3.4}$$

$$T_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \mathbf{1}\left[a_i < 32^2\right]. \tag{3.5}$$

C_j is the number of tracked objects divided by 30 and capped at 1, so every frame with 30 or more objects has $C_j = 1$. T_j is the fraction of tracked boxes smaller than 32×32 pixels, which is the size limit of small objects in the COCO benchmark [61]. The indicator $\mathbf{1}[\cdot]$ is 1 when its condition holds and 0 otherwise, and $T_j = 0$ when $N_j = 0$.

3.3.2 Scene Complexity Index and Scene Label

The SCI measures how crowded the recent frames are. A single frame is a noisy measurement, because the tracker may miss or add a few objects in any frame, so the SCI averages over time. It is the mean crowd cue of the last seven analysis steps:

$$S_j = \frac{1}{|H_j|} \sum_{k \in H_j} C_k, \tag{3.6}$$

where H_j holds the steps $j-6, \ldots, j$ (fewer at the start of a video) and $|H_j|$ is their number. With a stride of 10 frames, the seven steps cover about the last 60 frames, so a single unusual frame changes S_j only a little. Because $C_j \leq 1$, S_j lies between 0 and 1.

The SCI uses the crowd cue alone, selected by the cue audit of Section 3.4.4. The other four cues are still used as checks on the state, mainly through the scene label

$$\ell_j = \begin{cases} \text{night} & \text{if } \beta_t < 80, \\ \text{blur} & \text{else if } \nu_t < 180, \\ \text{tiny} & \text{else if } T_j > 0.50, \\ \text{crowded} & \text{else if } C_j > 0.65 \text{ or } e_t > 0.13, \\ \text{clear} & \text{otherwise.} \end{cases} \tag{3.7}$$

The conditions are checked from top to bottom, and the first one that holds gives the label. For example, $T_j > 0.50$ means that more than half of the tracked boxes are tiny, and $C_j > 0.65$ means that at least 20 objects are tracked.

Only the crowd cue enters the SCI, and it also enters the crowded label. The tiny-object ratio T_j is used by the state rules of Section 3.3.3 and gives the tiny label, and the edge density e_t is used only through the crowded label. The brightness β_t and the sharpness ν_t act only through the night and blur labels, which do not lead to MEDIUM or HIGH by themselves: because they are checked first, a dark or blurred frame cannot receive the crowded or tiny label, while the SCI conditions and the tiny-object guard of Eq. (3.8) still apply.

3.3.3 Three-State Controller

The controller is always in one of three states: LOW, MEDIUM, or HIGH. The names describe the scene: LOW for a sparse scene, HIGH for a crowded scene or a scene of mostly tiny objects. They do not rank the input size: in the final action map the HIGH state uses the smallest input (Table 3.1). The state is updated only at analysis steps, and each update has four steps (Algorithm 3.1). Step 1 proposes a target state from the cues. Steps 2–4 adjust this target so that the controller reacts to a sudden drop in the object count but does not switch back and forth. Figure 3.2 summarizes the three states, the operating point of each state, and the conditions under which the controller moves between them.

Step 1: target state. The target is the state that the current cues ask for. It is given by the SCI, the tiny-object ratio, and the scene label:

$$g_j = \begin{cases} \text{HIGH} & \text{if } S_j > \tau_h \text{ or } T_j > 0.50, \\ \text{MEDIUM} & \text{else if } S_j > \tau_m \text{ or } \ell_j \in \{\text{crowded}, \text{tiny}\}, \\ \text{LOW} & \text{otherwise,} \end{cases} \tag{3.8}$$

where $\tau_m = 0.297$ and $\tau_h = 0.511$ are the two SCI thresholds (Table 3.2). Since S_j is

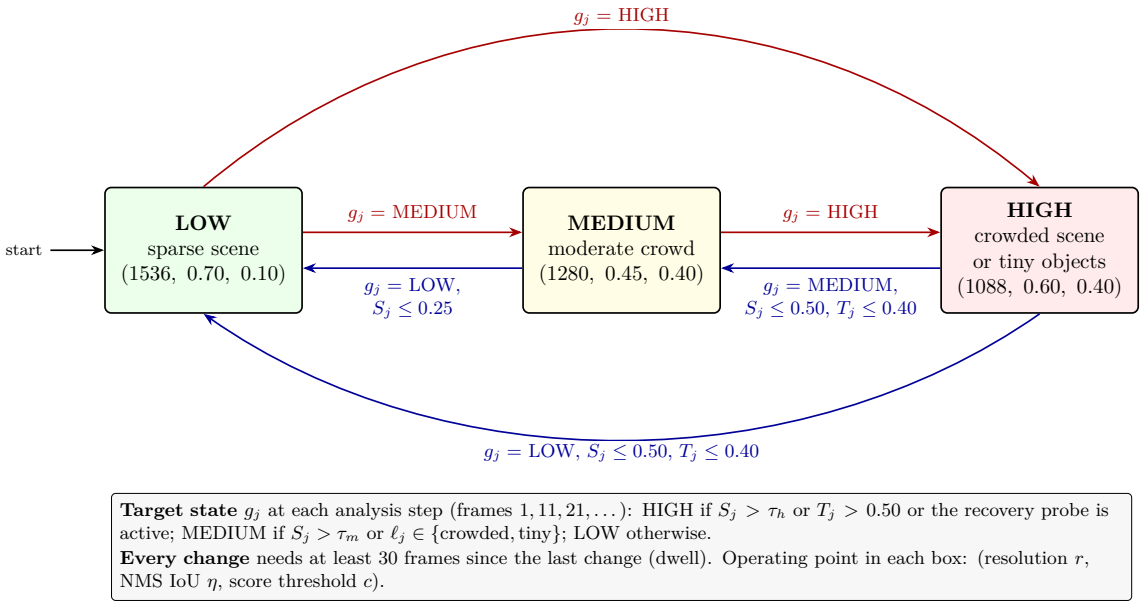


Figure 3.2: The three states of AC-MOT with the operating point of each state (Table 3.1) and the conditions of each change. Red arrows: moves to a more difficult state, taken as soon as the target state g_j asks for them; blue arrows: moves to an easier state, taken when the hold conditions of Eq. (3.11) are released. Every change also follows the dwell rule of Eq. (3.12).

the recent average number of tracked objects divided by 30, the SCI conditions give MEDIUM above about 9 objects ($30\tau_m = 8.9$) and HIGH above about 15 objects ($30\tau_h = 15.3$).

Step 2: recovery probe. The SCI averages seven steps, so it reacts slowly when the tracker suddenly reports far fewer objects than a few steps earlier. The recovery probe responds to such a collapse of the object count. Instead of letting the falling count move the controller toward a lower state, it sets the target to HIGH for a short time. To detect a collapse, it keeps a slowly decaying peak P_j of the object count and a drop counter d_j:

$$d_j = \begin{cases} d_{j-1} + 1 & \text{if } P_{j-1} \ge 5 \text{ and } N_j < 0.40\, P_{j-1}, \\ 0 & \text{otherwise,} \end{cases} \tag{3.9}$$

$$P_j = \max\left(N_j, \lfloor 0.8\, P_{j-1} \rfloor\right), \tag{3.10}$$

with $P_0 = d_0 = 0$. A collapse is detected when at least 5 objects have been seen before and fewer than 40% of them remain. The peak loses 20% at each step, so an old peak is forgotten after a few steps. While $1 \le d_j \le 3$, the target g_j is set to HIGH, so the probe lasts at most three analysis steps (30 frames); the dwell rule of Step 4 still applies.

Step 3: hysteresis. Without a margin, a scene whose SCI stays close to a threshold

would make the controller switch at almost every step. Hysteresis prevents this: a move to a lower state is held while the scene remains nearly as difficult:

$$g_j \leftarrow \begin{cases} \text{HIGH} & \text{if } q_{j-1} = \text{HIGH and } (S_j > 0.50 \text{ or } T_j > 0.40), \\ \text{MEDIUM} & \text{if } q_{j-1} = \text{MEDIUM},\ g_j = \text{LOW and } S_j > 0.25, \\ g_j & \text{otherwise.} \end{cases} \tag{3.11}$$

The hold values lie just below the values that lead into each state ($0.50 < \tau_h$, 0.40 below the tiny guard of 0.50, and $0.25 < \tau_m$). Upward moves are not held.

Step 4: dwell. The dwell rule sets a minimum time between two changes. After a change, the state is kept for at least 30 frames, that is, three analysis steps:

$$q_j = \begin{cases} g_j & \text{if } t - t_c \geq 30, \\ q_{j-1} & \text{otherwise,} \end{cases} \tag{3.12}$$

where t_c is the frame of the last state change. The controller starts in LOW with $t_c = 1$, so the first change is possible at frame 31. Whenever $q_j \neq q_{j-1}$, t_c is set to t.

Why changes are kept rare. Each change of state changes the boxes that reach the tracker, so the temporal rules let the state follow lasting changes of the scene, not single frames: the seven-step window smooths the SCI over about 60 frames, the holds add a margin below each threshold, and the dwell rule allows at most one change every 30 frames.

Example. Consider a clear, sharp video in which the tracker reports 20 objects of normal size in every frame ($C_j = 0.67$), and only 10 objects from frame 71 on. The controller starts in LOW. The first step sees no earlier boxes ($C_j = 0$), so S_j grows as the window fills: 0.33 at frame 11, 0.44 at frame 21, and 0.50 at frame 31. The target is MEDIUM from frame 11, but the dwell rule keeps LOW until frame 31, where the state becomes MEDIUM. From frame 41, S_j exceeds τ_h and the target is HIGH; the dwell rule delays this change to frame 61. When the scene thins, S_j falls slowly because the window still holds the earlier steps, so the state stays HIGH until frame 101. At frame 111, $S_j = 0.48$ is below both τ_h and the hold value of 0.50, and the state returns to MEDIUM. It then stays in MEDIUM, because 10 tracked objects keep S_j at about 0.33, above τ_m.

The state q_j then selects the operating point from the action map, and the detector uses it until the next analysis step.

3.3.4 Action Map

Table 3.1 gives the operating point of each state. The actions are not set by hand: they are selected together with τ_m and τ_h by the joint search on the calibration se-

Algorithm 3.1 AC-MOT controller for one video.

Input: frames $I_1, I_2, \ldots$; frozen detector and tracker; constants of Table 3.2
Output: tracks of every frame

$j \leftarrow 0$; $q_0 \leftarrow$ LOW; $P_0 \leftarrow 0$; $d_0 \leftarrow 0$; $t_c \leftarrow 1$
for each frame $t = 1, 2, \ldots$ **do**
    if $t = 1$ **or** $t \bmod 10 = 1$ **then** ▷ analysis step, every 10 frames
        $j \leftarrow j + 1$
        $e_t, \beta_t, \nu_t \leftarrow$ image cues of I_t ▷ Section 3.3.1
        $N_j, C_j, T_j \leftarrow$ cues of the tracked boxes of frame $t - 1$ ▷ no boxes at $t = 1$
        $S_j \leftarrow$ mean of C_k over the last seven steps $k \in H_j$ ▷ SCI, Eq. (3.6)
        $\ell_j \leftarrow$ label, Eq. (3.7); $g_j \leftarrow$ target, Eq. (3.8) ▷ Step 1: target
        update d_j and P_j, Eqs. (3.9)–(3.10)
        if $1 \leq d_j \leq 3$ **then** $g_j \leftarrow$ HIGH ▷ Step 2: recovery probe
        end if
        apply hysteresis to g_j, Eq. (3.11) ▷ Step 3: hysteresis
        if $t - t_c \geq 30$ **and** $g_j \neq q_{j-1}$ **then** ▷ Step 4: dwell
            $q_j \leftarrow g_j$; $t_c \leftarrow t$
        else
            $q_j \leftarrow q_{j-1}$
        end if
    end if
    $(r_t, \eta_t, c_t) \leftarrow$ action of state q_j, Table 3.1 ▷ kept until the next analysis step
    detect with r_t and η_t; drop boxes with score $< c_t$
    update the tracker; store its boxes of frame t ▷ used from frame $t + 1$
end for

quences, whose objective is the change in HOTA, a metric that balances detection and association (Section 3.3.5).

In LOW (the sparse state), the detector uses the largest input, loose NMS, and a low score threshold, so it passes more weak boxes. In MEDIUM and HIGH it passes only boxes with a score of at least 0.40. The HIGH state, used for crowded scenes and for scenes of mostly tiny objects, has the smallest input, because the search optimizes tracking quality: a score threshold of 0.40 gives the tracker fewer but more reliable boxes (Section 3.4.4), and the smaller input costs less detector time. Section 3.6 shows that the main effect of AC-MOT is a more selective detection stream: fewer false positives and identity switches.

The score threshold c_t is applied on the detector side before the tracker. It is separate from any confidence gate inside the tracker, such as the fixed tracker-side gate of OATrack.

3.3.5 Selection of Parameter Values

All values of Tables 3.1 and 3.2 were fixed before the final evaluation and were not changed after it. The last column of Table 3.2 gives the source of each value. There are four sources.

Joint search on the calibration sequences. The SCI thresholds τ_m and τ_h and the action map are selected together by an Optuna TPE search with 60 trials [102]; the search ranges are given in Section 3.4.4. The calibration sequences are eight sequences of the VisDrone2019-MOT training split. They serve as the validation (tuning) data of the final stage and are disjoint from the evaluation sequences. Each trial is scored by four-fold cross-validation (CV), with two sequences held out in each fold, as the mean change in HOTA against the baseline. The candidate values come from existing settings. The input sizes (1088, 1280, and 1536 pixels) and NMS values (0.45, 0.60, and 0.70) are the operating points for which detector outputs are cached; 1536 pixels with NMS 0.70 is the baseline point, and 0.70 is the default NMS value of the Ultralytics detector. The three score thresholds are the low and high thresholds of ByteTrack (0.10 and 0.25) and the confidence gate of OATrack (0.40).

Cue audit on the calibration sequences. The choice of the crowd cue as the only

Table 3.1: Action map of AC-MOT. State names describe scene difficulty, not a resolution rank. The actions were selected by the joint search on the calibration sequences.

State	Resolution r (pixels)	NMS IoU η	Score threshold c
LOW	1536	0.70	0.10
MEDIUM	1280	0.45	0.40
HIGH	1088	0.60	0.40

SCI input comes from testing all 31 non-empty subsets of the five cues with the same cross-validation (Section 3.4.4).

Temporal sweep on the validation split. The analysis stride and the SCI window are selected in the first development stage, with the YOLOv8n pipeline, on the VisDrone2019-MOT validation sequences. The sweep tests 25 combinations: windows of 1, 3, 5, 7, and 9 analysis steps and strides of 1, 5, 10, 15, and 20 frames. Five of the 25 settings meet both constraints of that stage, at least 25 frames per second (FPS) and at most 271 identity switches (the number reached by the original controller on the same data). Among them, a window of 7 and a stride of 10 give the highest MOTA, 18.165 against 18.044 for the next setting. Operating-point sweeps on the same data cover input resolutions from 512 to 960 pixels, confidence thresholds from 0.05 to 0.50, and NMS thresholds from 0.30 to 0.80, and define the supported values of the first optimized profiles (Section 3.4).

Fixed design values. The tiny-object area follows the COCO definition of small objects [61]. The remaining values (the crowd normalizer, the Canny thresholds, the label thresholds, the recovery, hold, and dwell values, and the initial state) come from the original AC-MOT controller and are kept unchanged in every search of the final stage, so the searched values are matched to them. For example, the crowd normalizer of 30 only sets the scale of S_j: τ_m and τ_h are searched on this scale and correspond to about 9 and 15 tracked objects.

3.4 Development of the Frozen Controller

The controller of Section 3.3 is the result of four development stages. The quality-oriented profile (Section 3.4.1) asked whether a scene index that steers the detector settings can beat a fixed default; the identity-aware profile (Section 3.4.2) made identity stability and speed explicit objectives; the transfer studies (Section 3.4.3) tested other data and pipelines; and the modern stage (Section 3.4.4) built the frozen controller for a modern detector with separate calibration data. The first three stages are historical: they used a weighted five-cue index with interpolated confidence and NMS values and three resolution levels (the SparseTrack study used a single edge-driven NMS rule), other detectors, and other evaluation protocols. Their numbers are reported in their own tables, each against its own baseline, and are not pooled with the modern results. Only the frozen controller of the final stage is evaluated in Section 3.6. Within the final stage, the cue audit and the score-threshold sweep play the role of an ablation: they test which scene cues and which parts of the operating point the controller needs.

Table 3.2: Values of the AC-MOT controller and how each value is chosen. Calibration sequences: eight VisDrone2019-MOT training sequences used as tuning data, disjoint from the evaluation sequences. CV: four-fold cross-validation.

Symbol	Meaning	Value	How the value is chosen
τ_m	SCI threshold for MEDIUM, Eq. (3.8)	0.297	joint TPE search on the calibration sequences, 60 trials, CV
τ_h	SCI threshold for HIGH, Eq. (3.8)	0.511	same joint search
(r, η, c)	operating point of each state	Table 3.1	same joint search over $\{1088, 1280, 1536\} \times \{0.45, 0.60, 0.70\} \times \{0.10, 0.25, 0.40\}$
–	SCI input cue	crowd only	cue audit of all 31 cue subsets on the calibration sequences, CV
–	analysis stride	10 frames	temporal sweep of 25 settings on the validation split; highest MOTA with $\geq$25 FPS and $\leq$271 IDS
$\lvert H_j \rvert$	SCI window	7 steps	same temporal sweep
–	tiny-object area	32×32 pixels	COCO small-object limit [61]
–	crowd normalizer, Eq. (3.4)	30 boxes	original controller; sets the scale on which τ_m, τ_h are searched
–	Canny thresholds (lower, upper)	50, 120	original controller; fixed in all final-stage searches
–	night and blur labels, Eq. (3.7)	$\beta_t < 80$, $\nu_t < 180$	original controller; fixed in all final-stage searches
–	crowded label, Eq. (3.7)	$C_j > 0.65$ or $e_t > 0.13$	original controller; fixed in all final-stage searches
–	tiny label and HIGH guard, Eqs. (3.7), (3.8)	$T_j > 0.50$	original controller; fixed in all final-stage searches
–	recovery: minimum peak, drop ratio, peak decay, steps	5, 0.40, 0.80, 3	original controller; fixed in all final-stage searches
–	holds in HIGH (S_j, T_j) and MEDIUM (S_j), Eq. (3.11)	(0.50, 0.40), 0.25	original controller; fixed in all final-stage searches
–	dwell, Eq. (3.12)	30 frames	original controller; fixed in all final-stage searches
–	initial state	LOW	original controller; fixed in all final-stage searches

3.4.1 Quality-Oriented Profile

Baseline. The reference pipeline is the pretrained YOLOv8n detector [66] followed by ByteTrack [13], both at their default settings: confidence threshold 0.25, NMS IoU threshold 0.45, input size 640 pixels, and the default ByteTrack parameters listed in Section 3.5. It runs with one fixed operating point for the whole video.

Method. The first optimization-derived profile kept the same detector, used a tuned ByteTrack profile, and searched only on the VisDrone2019-MOT validation split. Each of the five cues was normalized with validation statistics, and the index was their weighted sum

$$S_j = \sum_{k \in \{c,t,e,n,b\}} w_k \, x_{k,j}, \tag{3.13}$$

where k runs over the cues (c: crowd count, t: fraction of earlier boxes below 32×32 pixels, e: edge density, n: low brightness, b: low Laplacian variance), $x_{k,j}$ is the normalized value of cue k at analysis step j, and w_k is its weight, with $w_k \geq 0$ and $\sum_k w_k = 1$. The index was averaged over seven analyses taken every ten frames. Confidence and NMS were interpolated linearly between the values at $S = 0$ and $S = 1$, and two sorted thresholds selected one of three resolutions (512, 928, 960 pixels).

An Optuna TPE study [102] jointly searched the five weights, the confidence and NMS end points, and the two thresholds. The resolution levels, the supported confidence values $\{0.25, \ldots, 0.45\}$, the supported NMS values from 0.30 to 0.70, and the temporal settings were fixed beforehand by validation sweeps (Section 3.3.5). The selection rule kept trials with at least 25 FPS and at most 271 IDS (the IDS of the original controller on the same validation data) and then took the highest MOTA, with lower IDS, HOTA, IDF1, and FPS as tie-breaks. The frozen profile (archived as Trial 24) has $w_c = 0.1294928$, $w_t = 0.2217477$, $w_e = 0.4337134$, $w_n = 0.0535577$, $w_b = 0.1614885$, confidence end points $(0.30, 0.40)$, NMS end points $(0.35, 0.35)$, and thresholds $(0.1353494, 0.2872868)$. On validation it reached MOTA 23.038, HOTA 36.110, IDF1 40.758, and IDS 270 at 37.169 FPS.

Comparison. Table 3.3 compares the profile with the baseline on the historical VisDrone test-dev split, with paired bootstrap intervals (5,000 resamples, seed 42). A scene index that steers the detector settings improved tracking quality over the fixed default point, with HOTA, MOTA, and IDF1 intervals above zero. The next stage made identity stability an explicit objective. The final controller keeps two parts of this stage: the stride of 10 frames and the window of seven steps from its temporal sweep (Section 3.3.5), and the five kinds of scene cues, which were audited again in the modern stage.

Table 3.3: Quality-oriented profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol. Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Quality	Gain [95% CI]
HOTA	28.430	33.835	+5.4051 [4.1273, 6.9022]
MOTA	19.729	26.948	+7.2195 [5.4362, 9.2934]
IDF1	32.724	41.546	+8.8220 [6.9005, 11.0537]
IDS	1235	1184	51 [−114, 219]
FPS	36.528	38.985	–

3.4.2 Identity-Aware Pareto Profile

Baseline. The baseline is the same YOLOv8n and ByteTrack pipeline at default settings (Section 3.4.1).

Method. The second study kept the same cues, operating levels, temporal settings, and validation data, but changed the question. It maximized MOTA and minimized IDS subject only to FPS ≥ 25, without the IDS gate. A seeded TPE sampler ran 49 completed trials. A trial dominated another when it had no lower MOTA and no higher IDS, with at least one strict improvement, and the feasible Pareto front was built from these relations. The balanced candidate maximized an equal-weight score of normalized MOTA and normalized IDS reduction, with HOTA, IDF1, FPS, and trial index as tie-breaks. The selected profile (archived as Trial 22) has weights $w_c = 0.1646453$, $w_t = 0.1746265$, $w_e = 0.5076113$, $w_n = 0.1206956$, $w_b = 0.0324213$, confidence end points $(0.40, 0.40)$, NMS end points $(0.60, 0.35)$, and thresholds $(0.2927136, 0.6661672)$. On validation it reached MOTA 19.330, HOTA 31.651, IDF1 34.226, and IDS 168 at 51.406 FPS.

Comparison. Table 3.4 compares the profile with the baseline on test-dev. It removes 316 IDS and raises throughput from 36.528 to 46.024 FPS, while HOTA, MOTA, and IDF1 also rise. Against the quality-oriented profile it has 265 fewer IDS $[82, 502]$. The two profiles therefore answer different questions: one favors overall quality and the other favors identity stability and speed. The detector operating point controls a trade-off between quality, identity stability, and speed, and the objective of the search decides which side is favored.

3.4.3 Transfer and Cross-Pipeline Studies

This stage applied the historical controllers outside their development setting: to a second dataset and to two other detector–tracker pipelines.

UAVDT. The baseline is again YOLOv8n with ByteTrack at default settings (Section 3.4.1). Both frozen profiles were applied to UAVDT [9] without retuning (Ta-

Table 3.4: Identity-aware profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol. Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Identity	Gain [95% CI]
HOTA	28.430	31.218	+2.788 [0.747, 4.660]
MOTA	19.729	23.792	+4.063 [0.952, 6.886]
IDF1	32.724	37.870	+5.146 [2.028, 8.080]
IDS	1235	919	316 [175, 470]
FPS	36.528	46.024	–

ble 3.5). The quality-oriented profile raised HOTA by 4.305 points $[2.740, 5.709]$ and the identity-aware profile by 2.845 points $[1.238, 4.348]$; their IDS reductions are 237 $[104, 377]$ and 250 $[110, 404]$.

U2MOT on VisDrone. The baseline is the published U2MOT tracker [92] with its released YOLOX-X detector, run at its own static setting. Seven VisDrone validation sequences served for development and 17 test-dev sequences for evaluation. The cue weights and transforms of the quality-oriented profile were kept, the tracker was not modified, and the action boundaries were set to the 33rd and 67th percentiles of the index on the U2MOT validation run, without using ground truth. Table 3.6 (top) gives the comparison: the controlled pipeline has 87 fewer IDS and 1,230 fewer false positives.

SparseTrack on MOT17. The baseline is the published SparseTrack tracker [89] with a fixed NMS IoU threshold of 0.75, on the seven MOT17 val-half sequences (2,652 frames) [97]. A minimal edge-driven rule switched only the NMS threshold, between 0.70 and 0.80; its threshold came from a leave-one-sequence-out study on the same sequences. Table 3.6 (bottom) gives the comparison: the adaptive rule reduces IDS from 136 to 123.

The frozen profiles kept their direction on UAVDT without retuning, and with U2MOT and SparseTrack the control reduced IDS. Together these studies show that the preferred profile depends on the objective (quality or identity) and on the tracker host. This motivated a modern stage with a stronger detector, explicit calibration

Table 3.5: Historical UAVDT transfer without retuning. Baseline: YOLOv8n with ByteTrack at default settings.

Metric	Baseline	Quality	Identity
HOTA	24.085	28.390	26.930
MOTA	13.841	17.399	16.118
IDF1	27.887	34.453	32.014
IDS	558	321	308
FPS	65.015	58.353	61.462

Table 3.6: Cross-pipeline studies. Top: U2MOT with its YOLOX-X detector at its static setting on VisDrone test-dev. Bottom: SparseTrack with a fixed NMS IoU threshold of 0.75 on MOT17 val-half.

Pipeline	Metric	Baseline	With control
U2MOT	MOTA	53.873	53.767
	IDF1	69.775	69.849
	IDS	1239	1152
	FP	41385	40155
	FN	63241	64801
SparseTrack	MOTA	76.8881	76.9252
	IDF1	81.4943	81.5299
	IDS	136	123

data, and two tracker hosts.

3.4.4 Modern Stage

The modern stage decided which cues enter the SCI, which operating points each state uses, and where the SCI thresholds lie.

Baseline. The modern baseline is the YOLO11m detector [67], in a public version trained on VisDrone2019-DET [6], run at a fixed operating point of 1536 pixels and NMS IoU 0.70 with no extra score threshold. It feeds ByteTrack [13] during development and both ByteTrack and OATrack [93] in the final evaluation. A large fixed input is a common choice for small aerial objects when no adaptation is used.

Data and protocol. Eight sequences of the VisDrone2019-MOT training split (3,282 frames) were declared as calibration data. They are disjoint from the seven evaluation sequences. The searches used the same four folds, each holding out two calibration sequences. Detector outputs were cached for every resolution and NMS value; a score threshold applied to the cache was checked to reproduce direct inference exactly. The steps are described below in the order in which they were carried out.

Resolution and NMS maps. Two exhaustive grid searches, archived as Stages 1 and 2, searched the state-to-resolution and state-to-NMS maps. At this point the action of a state contained only the input resolution and the NMS value.

Cue audit. The next search, archived as Stage 3A, asked which cues should form the SCI. It searched all 31 non-empty subsets of the five cues and selected {crowd}, which defines the crowd-selected SCI of Eq. (3.6). The other cues were kept as guard and auxiliary cues of the state logic (Section 3.3.2).

Score threshold. A separate sweep then tested the score threshold. At 1088 pixels and NMS 0.45, a fixed score threshold of 0.40 raised pooled calibration HOTA by +0.881 over 0.01 and improved seven of the eight sequences. The score threshold was

therefore added as the third action dimension.

Final joint search. The final search then selected the thresholds and the three-part actions together. It used 60 TPE trials (seed 43) and searched $\tau_m \in [0.10, 0.50]$, $\tau_h \in [\tau_m + 0.05, \min(\tau_m + 0.40, 0.90)]$, and for each state a resolution in $\{1088, 1280, 1536\}$, an NMS value in $\{0.45, 0.60, 0.70\}$, and a score threshold in $\{0.10, 0.25, 0.40\}$. The objective was the cross-validated mean change in HOTA against the baseline, with ByteTrack as host. A candidate was eligible only if it really adapted within the video rather than acting as one fixed operating point: it had to realize at least two distinct actions, each on at least 10% of calibration frames, and switch within sequences. The frozen policy is the eligible candidate whose three states use three different actions that together vary all three parts of the operating point (Table 3.1). On calibration data it used LOW, MEDIUM, and HIGH on 7.6%, 10.1%, and 82.3% of frames, with 0.607 state changes per 100 frames on average.

Comparison. Table 3.7 compares the frozen policy with the baseline on each calibration fold. All four folds are positive, with a CV mean of +2.447 HOTA. After these steps the thresholds, cue subset, and action map were frozen. The final comparison on the evaluation sequences is given in Section 3.6.

3.5 Experimental Setup

The experiments test whether replacing the fixed operating point with the frozen AC-MOT controller improves tracking with the same detector and tracker, on VisDrone2019-MOT and, without retuning, on UAVDT. Table 3.8 summarizes the setup.

Data and protocol. The datasets are described in Section 2.3.7. Eight sequences of the VisDrone2019-MOT training split (3,282 frames) are the calibration data on which all values of the frozen controller were selected (Section 3.4.4). The seven sequences of the VisDrone2019-MOT validation split (2,846 frames, 71,830 ground-truth boxes) are the evaluation data; they are disjoint from the calibration sequences. The 20 sequences of the UAVDT test split (16,592 frames) test whether the frozen controller still helps on a different dataset with none of its values changed; detector outputs were produced at the three frozen operating points (49,776 detector forward passes), with a single frozen run per host.

Table 3.7: Frozen AC-MOT policy versus the modern baseline (YOLO11m at 1536 pixels and NMS 0.70, ByteTrack) on the four calibration folds: change in HOTA.

	Fold 1	Fold 2	Fold 3	Fold 4	Mean
ΔHOTA	+0.592	+4.164	+2.029	+3.004	+2.447

Detector, trackers, and baseline. All experiments use the YOLO11m detector [67] in the public version trained on VisDrone2019-DET (Section 2.3.2). It runs in half precision, and detections of the classes pedestrian, car, van, truck, and bus are passed on. ByteTrack uses the Ultralytics implementation with its default parameters: high threshold 0.25, low threshold 0.10, new-track threshold 0.25, buffer 30 frames, and match threshold 0.80. OATrack is a re-implementation of the published method [93] in its main configuration, with its published tracker-side confidence gate of 0.40. This gate belongs to the tracker and is never changed by AC-MOT. For each host, two systems are compared. The baseline runs the detector at the fixed operating point of 1536 pixels and NMS IoU 0.70, with a minimal score threshold of 0.01. AC-MOT runs the frozen controller of Section 3.3 on the same detector, so the two systems differ only in the detector operating point.

Evaluation and statistics. On VisDrone, results are scored with a Python port of the VisDrone2019-MOT Task 4b evaluation: ignored regions are removed, the five classes above are evaluated separately, CLEAR-MOT and identity measures use IoU 0.5, and counts are summed over classes and sequences; HOTA is added with the same class-aware scheme using TrackEval. On UAVDT, all vehicles are scored as one class. Uncertainty is estimated with the paired sequence-level bootstrap of Section 2.3.8, with 5,000 resamples (seed 42); the bootstrap draws seven sequences with replacement on VisDrone and 20 on UAVDT. On UAVDT it draws from the stored per-sequence counts and therefore covers the count-based metrics MOTA, IDF1, IDS, FP, and FN.

Runtime. Latency is measured on one Tesla T4 GPU over the first 200 frames of sequence uav0000086_00000_v, excluding five warm-up frames. For every frame, the time spent in the controller (image statistics, scene analysis, and state update) is recorded separately from the detector and tracker time.

3.6 Results

3.6.1 Frozen VisDrone Evaluation

The cue subset, thresholds, and action map of the frozen controller were selected on the separate calibration sequences only (Section 3.4.4). Table 3.9 lists the aggregate results on the seven VisDrone sequences, and Fig. 3.3 shows HOTA, MOTA, and IDF1.

ByteTrack host. AC-MOT raises HOTA, MOTA, and IDF1 and lowers IDS and FP. The detector output becomes more selective: clutter and identity-confusing boxes are removed. The very low baseline MOTA shows how many FP the fixed operating point passes to this tracker; because MOTA counts every FP, the removal of more than a third of them is the main source of the MOTA gain.

OATrack host. OATrack starts from a stronger baseline, and its gains are smaller,

Table 3.8: Experimental setup of AC-MOT.

Item	Setting
Detector	YOLO11m, VisDrone2019-DET weights, half precision; classes pedestrian, car, van, truck, bus
Trackers (hosts)	ByteTrack (Ultralytics defaults); OATrack (published main configuration, tracker-side gate 0.40)
Baseline	fixed operating point: 1536 pixels, NMS IoU 0.70, score threshold 0.01
Calibration data	VisDrone2019-MOT training, 8 sequences, four-fold cross-validation
Evaluation data	VisDrone2019-MOT validation, 7 sequences; UAVDT test, 20 sequences, no retuning
Scoring	VisDrone Task 4b port with TrackEval HOTA; UAVDT one vehicle class
Statistics	paired sequence-level bootstrap, 5,000 resamples, seed 42
Runtime	Tesla T4 GPU, 200 frames, 5 warm-up frames excluded

Table 3.9: Frozen VisDrone evaluation on seven VisDrone2019-MOT sequences (71,830 ground-truth boxes). Baseline: fixed operating point (1536, 0.70).

	ByteTrack		OATrack	
Metric	Baseline	AC-MOT	Baseline	AC-MOT
HOTA	41.586	44.523	46.770	48.095
MOTA	9.876	27.871	28.165	34.483
IDF1	48.569	53.929	57.107	59.364
IDS	1341	1016	773	604
FP	39274	24892	25394	21400
FN	24121	25902	25432	25057
Precision	54.849	64.852	64.628	68.609
Recall	66.419	63.940	64.594	65.116

but every metric of Table 3.9 moves in the direction of improvement, including FN (25,432 to 25,057) and recall (64.594 to 65.116).

Paired bootstrap. In the VisDrone columns of Table 3.10, the ByteTrack intervals of HOTA, MOTA, IDF1, precision, IDS, and FP lie entirely on the side of improvement. With OATrack, HOTA, MOTA, IDF1, and precision rise and IDS and FP fall, all with intervals on one side of zero.

3.6.2 Controller Behavior

The controller uses all three states with both hosts. With ByteTrack, LOW, MEDIUM, and HIGH cover 7.4%, 30.1%, and 62.5% of the frames; with OATrack they cover 7.4%, 33.3%, and 59.3%. For most of the video, the detector therefore runs with the HIGH action (the smallest input and a score threshold of 0.40). The two hosts diverge through their different tracked boxes. State changes are rare, between 0.10 and 0.54 per 100 frames depending on the sequence, so the detector input stays stable over long stretches of video.

3.6.3 Frozen UAVDT Transfer

The UAVDT transfer applies the frozen controller, with none of its values changed, to the 20 test sequences of a dataset that was not used to build it. Table 3.11 and Fig. 3.4 report the results.

With ByteTrack, HOTA rises by 1.081 points, MOTA by 17.601, and IDF1 by 3.367; with OATrack, MOTA rises by 3.267 and IDF1 by 0.831. With both hosts, FP and IDS fall and precision rises, the same pattern as with ByteTrack on VisDrone: the controller changes the detector output in the same way on a dataset that was not used to build it.

In the UAVDT columns of Table 3.10, the ByteTrack intervals of MOTA, IDF1, IDS, and FP lie on the side of improvement; with OATrack, MOTA and IDF1 rise and FP fall with intervals on one side of zero.

3.6.4 Runtime

AC-MOT needs no training and no access to detector or tracker internals; it changes three arguments of the detector call and analyzes a quarter-scale image every ten frames. On a Tesla T4 the controller takes 4.26 ms per frame with ByteTrack and 4.20 ms with OATrack, which is small compared with detector inference and suitable for real-time use.

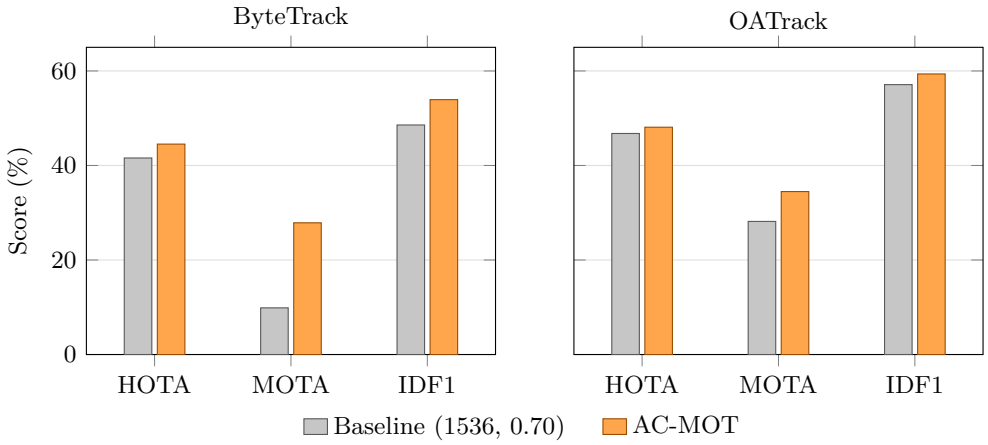


Figure 3.3: HOTA, MOTA, and IDF1 on the seven VisDrone sequences for both tracker hosts (values in Table 3.9).

Table 3.10: Paired sequence-level bootstrap of AC-MOT: AC-MOT minus baseline, 5,000 resamples, seed 42, on VisDrone (7 sequences) and UAVDT (20 sequences). HOTA, MOTA, IDF1, precision, and recall in points; IDS, FP, and FN in counts. On UAVDT the bootstrap covers the count-based metrics only.

		VisDrone		UAVDT	
Host	Metric	Difference	95% interval	Difference	95% interval
ByteTrack	HOTA	+2.937	[+2.356, +3.375]	–	
	MOTA	+17.995	[+13.541, +23.282]	+17.601	[+11.134, +26.767]
	IDF1	+5.360	[+4.490, +6.178]	+3.367	[+1.676, +5.013]
	IDS	−325	[−498, −164]	−115	[−211.025, −26.975]
	FP	−14382	[−19176, −9620]	−67539	[−104215.350, −37352.950]
	FN	+1781	[+288, +3594.075]	+7652	[+1542.925, +18359.400]
	Precision	+10.003	[+8.712, +11.895]	–	
	Recall	−2.479	[−4.507, −0.469]	–	
OATrack	HOTA	+1.325	[+0.200, +2.264]	–	
	MOTA	+6.318	[+4.741, +7.879]	+3.267	[+0.607, +5.881]
	IDF1	+2.258	[+0.548, +3.474]	+0.831	[+0.125, +1.271]
	IDS	−169	[−292, −42]	−31	[−101, +39]
	FP	−3994	[−6190, −2494.900]	−14268	[−33958.075, −2188.950]
	FN	−375	[−1462.575, +670]	+3162	[−211.100, +8959.050]
	Precision	+3.981	[+2.957, +5.511]	–	
	Recall	+0.522	[−0.869, +2.111]	–	

Table 3.11: Frozen UAVDT transfer of AC-MOT without retuning (20 sequences, 16,592 frames, one vehicle class). Baseline: fixed operating point (1536, 0.70).

	ByteTrack		OATrack	
Metric	Baseline	AC-MOT	Baseline	AC-MOT
HOTA	45.923	47.004	47.728	47.640
MOTA	−1.573	16.028	19.607	22.873
IDF1	58.400	61.767	62.101	62.931
IDS	601	486	517	486
FP	279616	212077	197937	183669
FN	66050	73702	75612	78774
Precision	49.571	55.751	57.270	58.800
Recall	80.625	78.381	77.820	76.893

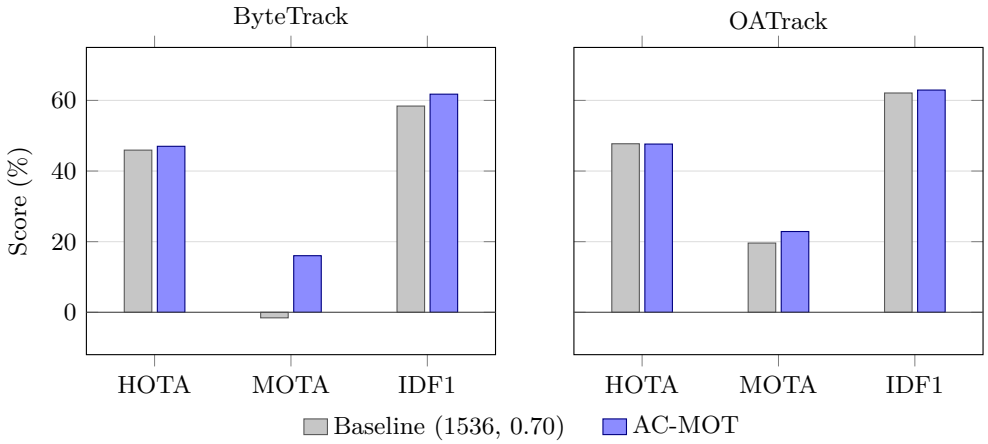


Figure 3.4: HOTA, MOTA, and IDF1 of the frozen UAVDT transfer of AC-MOT for both tracker hosts (values in Table 3.11). The ByteTrack baseline MOTA is negative (−1.573); the solid line marks zero.

3.6.5 Qualitative Evaluation of Tracking Performance on Selected Video Frames

Two qualitative experiments show the effect of AC-MOT directly on the frames of a video. Both run the same detector, YOLO11m, and the same tracker, ByteTrack, on the same video. The baseline pipeline keeps the fixed operating point (1536, 0.70), and the AC-MOT pipeline lets the frozen controller set the operating point. The tracked targets of both pipelines are drawn on the same frames.

Video. The video is sequence uav0000117_02622_v of VisDrone2019-MOT (349 frames of 2720 × 1530 pixels), one of the seven evaluation sequences of Table 3.9. It shows a busy street at night, recorded from a UAV whose camera turns from a view across an intersection to a view along the street. It contains cars, vans, trucks, and many small pedestrians; vehicles pass behind poles and trees and partly hide each other, and the number of annotated objects per frame changes between 18 and 68. Among the seven sequences, it has the median HOTA gain with ByteTrack, so it shows a typical effect of the controller. For the figures, the video was processed again with the same frozen detector weights, controller, and tracker, and this run reproduces the gain of the sequence in the frozen evaluation: HOTA rose from 34.21 to 37.42 (+3.21 points), IDF1 from 37.50 to 42.01, and MOTA from −12.71 to 13.25, while the identity switches fell from 439 to 400 and the false positives from 9,958 to 6,479.

Experiment 1: a cleaner detection stream. Figure 3.5 shows frames 268, 273, and 278. Frame 278 is the frame with the largest reduction in false tracks, and the frames 5 and 10 earlier show how the tracks develop. At the fixed operating point, 47 to 54 output boxes per frame match no annotated object (red), many of them on the street lamps and lanterns along the road. With AC-MOT, which is in its HIGH state here, only 22 to 28 boxes per frame are false, while the number of correctly tracked objects is the same or higher (27, 26, and 25 against 27, 26, and 24). The tracked vehicles keep one ID and a smooth path over the three frames. Because fewer false boxes reach the tracker, fewer tracks are started: over the whole video, the AC-MOT pipeline output 1,255 tracks against 1,732, which is also why its ID numbers are lower.

Figure 3.6 follows the whole video and shows the effect of the controller's decision itself. The SCI rises above τ_h within the first analysis steps, and the controller moves from LOW to HIGH at frame 31, the first frame that the dwell rule allows (Section 3.3.3). The change takes effect at once: in the first 30 frames, while the controller is in LOW, both pipelines output the same number of false tracks; from frame 31 on, with the HIGH operating point, the AC-MOT pipeline has fewer false tracks in 306 of the 319 frames, 18.0 per frame on average against 28.9. The controller then holds HIGH while the scene stays crowded (SCI between 0.72 and 1.00), so the detector input stays stable. The same frozen rules choose other states in sparser scenes: over



Figure 3.5: Experiment 1: VisDrone2019-MOT sequence uav0000117_02622_v tracked by ByteTrack with YOLO11m at frames 268, 273, and 278 (rows), (a) at the fixed operating point (1536, 0.70) and (b) with AC-MOT. Each color and number is one track, and its line shows the last 20 frames; red: false tracks (no annotated object); white dashed: missed objects. The text in each panel gives the correctly tracked objects, the false tracks, and the missed objects of the whole frame, and the AC-MOT state. Frame 278 is the frame with the largest reduction in false tracks; all panels show the same 1280×720 window. IDs are numbered separately in each run.

the seven evaluation sequences, LOW, MEDIUM, and HIGH cover 7.4%, 30.1%, and 62.5% of the frames with ByteTrack (Section 3.6.2), so the controller follows each scene with the operating point that suits it.

Experiment 2: identities of single objects. Figure 3.7 follows single annotated objects over their whole time in the video. The line is the path of the track that is matched to the object, and its color changes whenever the object receives a new track ID. The objects were selected by a fixed rule: objects present in at least 40 frames that AC-MOT follows in at least as many frames as the baseline, ranked by the ratio of their identity counts. A pedestrian on a crosswalk, the object of Fig. 2.8, receives five identities at the fixed operating point and two with AC-MOT, which follows it in 70 of its 71 frames. A van that is partly hidden behind a pole receives six identities in 46 tracked frames at the fixed operating point and three identities in 75 tracked frames with AC-MOT. A truck at the roadside keeps one identity over 198 frames with AC-MOT, against two identities over 149 frames at the fixed operating point. Over the whole video, 31 annotated objects receive fewer identities with AC-MOT, which is the identity gain behind the lower IDS and the higher IDF1 of this sequence.

Together, the two experiments show on real frames the effect measured in Section 3.6.1: AC-MOT gives the unchanged tracker a cleaner detection stream, so fewer false tracks are started, objects keep their identities for longer, and the operating point stays stable while the scene remains crowded.

3.7 Discussion

The operating point is part of the MOT system. The same frozen detector and trackers give very different MOT results at different operating points: moving away from the fixed 1536/0.70 point raises MOTA by 17.995 points with ByteTrack and by 6.318 points with OATrack on VisDrone. On both datasets FP and IDS drop sharply and precision rises, so MOTA, which counts every FP, reacts most strongly. The detector operating point is therefore a first-order variable of aerial MOT and deserves the same attention as the association method. This answers RQ1: the operating point can be adapted online from scene cues and past tracker output, without training, and the adaptation improves tracking over a fixed operating point.

A more selective detection stream. Most of the time the controller selects the HIGH action, with the smallest input and a score threshold of 0.40 (Section 3.6.2). The development studies explain why: a score threshold of 0.40 gave the tracker fewer but more reliable boxes and raised calibration HOTA on seven of eight sequences (Section 3.4.4), and the smaller input costs less detector time. The gain comes mainly from removing clutter and identity-confusing boxes, as the tracked frames of Section 3.6.5 show.

Tracker dependence. ByteTrack, which starts from a weaker baseline and uses low-

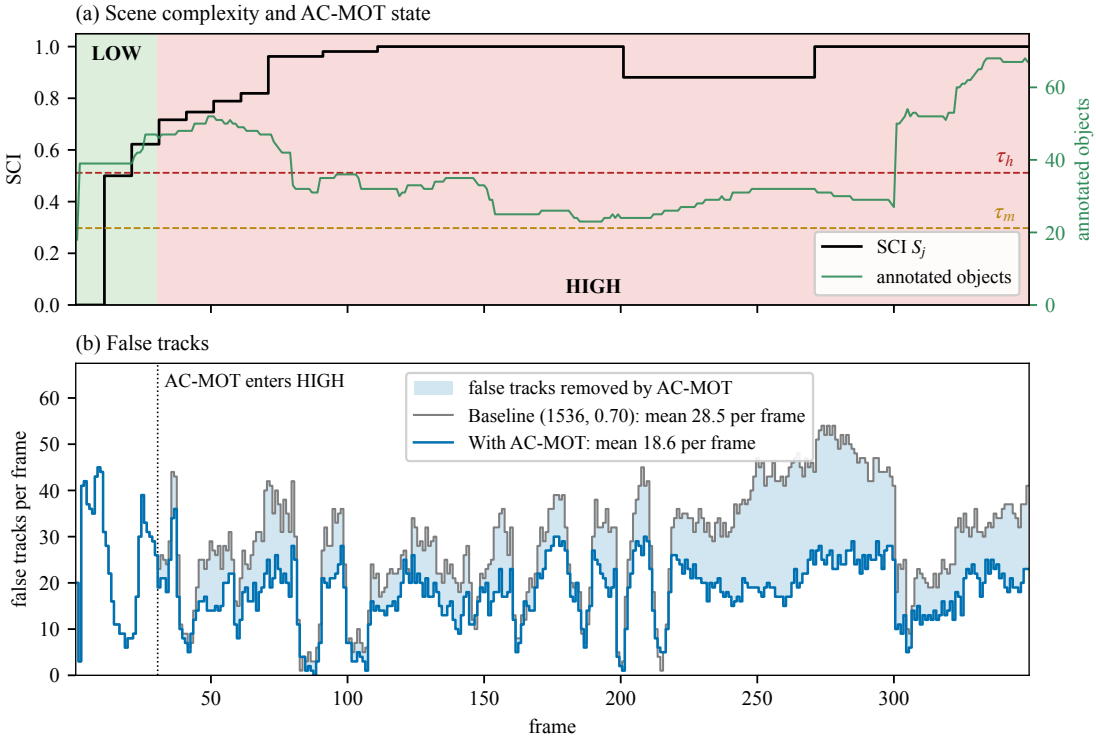


Figure 3.6: Experiment 1 over the whole video of Fig. 3.5: (a) the SCI computed by AC-MOT (black), its thresholds τ_m and τ_h, the state chosen by the controller (shaded), and the number of annotated objects per frame (green); (b) the false tracks per frame at the fixed operating point (gray) and with AC-MOT (blue); the shaded area marks the false tracks that AC-MOT removes.

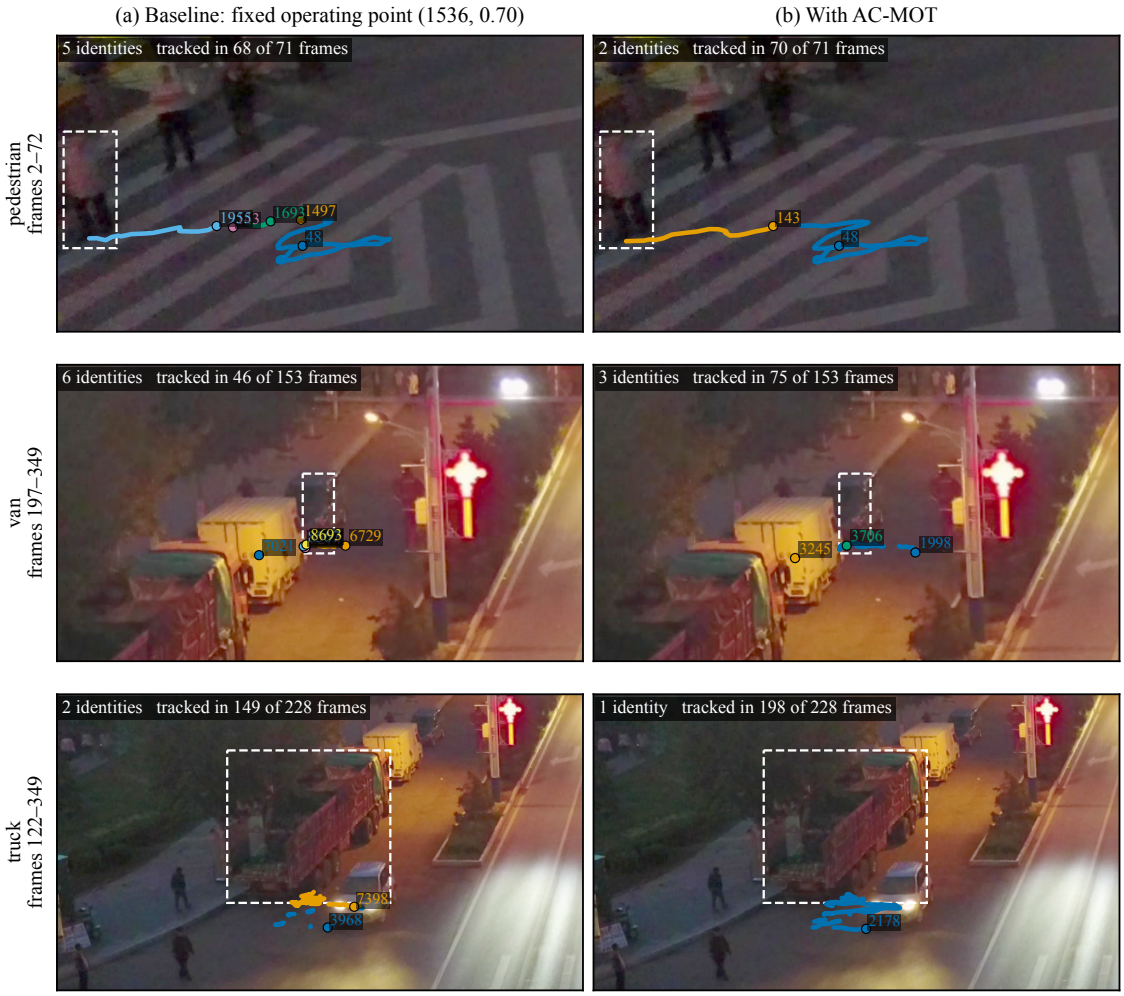


Figure 3.7: Experiment 2: single annotated objects of the video of Fig. 3.5, (a) at the fixed operating point (1536, 0.70) and (b) with AC-MOT, same detector and tracker. The line is the path of the track matched to the object (bottom center of its box); its color changes, and a dot with the new ID marks the start, whenever the object receives a new track ID. Dashed box: the object in the frame shown (its last frame). The text gives the number of identities and the number of frames in which a track follows the object. Objects: present in at least 40 frames and followed by AC-MOT in at least as many frames as by the baseline, ranked by the ratio of their identity counts.

score boxes in its second association step, gains more than OATrack, which already weights detections by quality and gains mainly in MOTA, IDF1, IDS, and FP. The preferred operating point thus depends on the tracker host, as the historical profiles of Section 3.4 also showed. AC-MOT serves both hosts without changing them.

Transfer. The frozen controller kept its effect on UAVDT, a dataset of vehicles recorded under other conditions, with none of its values changed (RQ3). The same pattern of fewer FP and IDS and higher precision appeared on both datasets.

Deployment. AC-MOT needs no training and no access to detector or tracker internals, and it adds about 4.2 ms per frame on a Tesla T4. Its state changes are rare, so the detector input stays stable over long stretches of video.

3.8 Summary

This chapter presented AC-MOT, a training-free controller that adjusts the confidence threshold, the NMS IoU threshold, and the input resolution of the detector online to improve MOT in UAV video, using only the current frame and previous tracking results and without changing or retraining the detector or the tracker. A scene complexity index formed from the crowd cue, checks from four further scene cues, and a three-state controller with recovery, hysteresis, and dwell rules let the settings follow the scene without frequent changes. The controller was developed through optimization experiments, and its final values were selected by cross-validation on separate calibration sequences and then kept fixed. On seven VisDrone2019-MOT sequences, AC-MOT improved HOTA by 2.937 points with ByteTrack and 1.325 points with OATrack, and MOTA by 17.995 and 6.318 points, respectively; paired bootstrap analysis supports these gains. On UAVDT it improved MOTA by 17.601 points with ByteTrack and 3.267 points with OATrack without retuning, at about 4.2 ms per frame. Qualitative experiments on a VisDrone video showed the same effect on the tracked frames: fewer false tracks, objects kept under fewer identities, and a stable operating point in a crowded scene. The results show that the detector operating point should be treated as part of the tracking system rather than as a fixed setting. The next chapter turns to the other control point of the pipeline: the relation between the detector's score scale and the tracker's thresholds.

Chapter 4

SCL: Training-Free Self-Calibrating Control Layer

4.1 Introduction

Multi-object trackers rely on score thresholds tuned for one detector. When the detector is replaced, retrained, or compressed, its scores shift, and the tuned thresholds silently stop fitting. AC-MOT, the controller of Chapter 3, adapts the detector to the scene and reads the detector's scores through the tracker's published thresholds. This chapter turns to these thresholds and to their compatibility with the detector's score scale. It presents the second controller of the thesis, a training-free self-calibrating control layer (SCL) between an unchanged detector and an unchanged tracker. The chapter is based on the manuscript submitted to The Visual Computer [28].

SCL splits the logits of recent scores twice with Otsu's method and classifies the stream as clean or noisy, without labels or future frames. Its novelty is selective, host-aware adaptation. On clean streams it keeps the tracker's scores and lowers a threshold only if it would reject the confident detections; on noisy streams it selects and rescores the detections and sets the thresholds. It acts through four published tracker settings and passes on weak detections that continue tracks, so one frozen configuration serves single-stage and two-stage trackers.

The layer was evaluated on seven trackers, four detection sources, and three datasets, and all changes below are relative to the tracker alone. When the detector scores were halved, two trackers fell to zero HOTA; with SCL both recovered at least 97% of their original HOTA. With a replacement detector, RT-DETR-L, SCL raised the MOTA of ByteTrack and BoT-SORT by 12.6% and 16.3% and cut their identity switches by 43% and 47%, respectively. It also raised the HOTA of PD-SORT, a tracker not seen during design, by 0.9%, and it kept four well-matched trackers within 0.1% of their own accuracy. SCL adds on average no more than 2.95 ms per frame on

an ordinary processor, so a detector can be replaced or retrained without retuning the tracker.

Section 4.2 states the problem and the requirements of the layer. Section 4.3 presents the five steps of SCL, its host contract, and its fixed values. Section 4.4 describes how it was developed and frozen. Section 4.5 gives the experimental setup, Section 4.6 the results on MOT17, KITTI, and UAVDT, Section 4.7 the comparison with adaptive-threshold methods, Section 4.8 the ablation and sensitivity analysis, and Section 4.9 the runtime. Section 4.10 discusses the results, and Section 4.11 summarizes the chapter.

4.2 Problem Setting and Requirements

In this chapter the tracker that SCL serves is called the *host*. A host reads detector scores through its published thresholds (Section 2.3.6): a score a needed to match a track, a score b needed to start one, and the lowest score ℓ it uses at all. These values were tuned for one detector. The detector and the host are *compatible* when the confident detections of the objects pass the host's thresholds and the background clutter does not. Compatibility is a property of the pair, not of either component: the same tracker can be compatible with one detector and incompatible with another, and the same detector can serve one tracker well and another badly.

When compatibility is lost, single-stage and two-stage hosts fail in different ways. A single-stage host ignores every box below one threshold, so a lower score scale removes most of its boxes, and it has no step that could continue a track through a weak detection. A two-stage host keeps continuing tracks through its low-score step, but it may stop starting new tracks, as in the example of ByteTrack with halved scores (Section 1.2). A noisy detector, in turn, can pass many false positives above the host's thresholds, as RT-DETR-L did on the UAVDT test split (Section 4.6.5).

The problem is to restore compatibility online, frame by frame, without changing either component. The layer must meet six requirements:

1. *Label-free:* it decides from the detector scores alone, without ground truth or a calibration set.

2. *Causal:* the decision for frame t uses only the scores of earlier frames, the current candidates, and the tracks of the previous frame.

3. *Training-free and external:* the detector and the host are frozen; their code, weights, and association are unchanged, and only the values of the host's thresholds and its input are set for each frame.

4. *Selective:* a well-matched pair must keep its own output, because an adaptation that always acts can lower the accuracy of strong trackers (Section 4.4.1).

5. *Host-aware and tracker-agnostic:* the same rules must serve single-stage and two-stage hosts, through values that each tracker already publishes.

6. *Inexpensive:* the layer must run on an ordinary processor in a few milliseconds per frame.

4.3 The SCL Framework

SCL is a lightweight, rule-based layer operating between a frozen detector and a frozen host without modifying their underlying algorithms. The host gives SCL four of its published settings, the host contract (Section 4.3.8), and SCL returns, for every frame, the candidates, scores, thresholds, and matching tolerance with which the host runs its own update.

4.3.1 Overview

Figure 4.1 illustrates the five processing steps of SCL for frame t. First, Otsu-based logit partitioning (Step 1) divides recent detection scores into background, uncertain, and confident groups. Next, detection stream classification (Step 2) determines whether the score distribution is clean or noisy based on historical confidence statistics. Mode-adaptive candidate selection (Step 3) then retains, removes, or remaps detection scores and adjusts the tracker's confidence thresholds according to the operating mode. Track-guided weak detection recovery (Step 4) allows eligible low-confidence detections to support track continuity. Finally, camera motion-adaptive matching tolerance (Step 5) adjusts the association criterion according to estimated camera motion. The host contract provides the tracker's published parameters, while the tracker performs its original association and track-management operations unchanged.

Table 4.1 lists every symbol in plain words, with its value in the clean stream of Fig. 4.2a, which the subsections below follow from input to output.

4.3.2 Inputs and Outputs

In each frame t, the detector gives candidates $D_t = \{(\mathbf{x}_i, s_i, c_i)\}$, each with a box $\mathbf{x}_i$, a score $s_i \in (0, 1)$, and a class c_i. The layer works on the logit of each score (Eq. (2.4)),

$$z_i = \log \frac{s_i}{1 - s_i}, \tag{4.1}$$

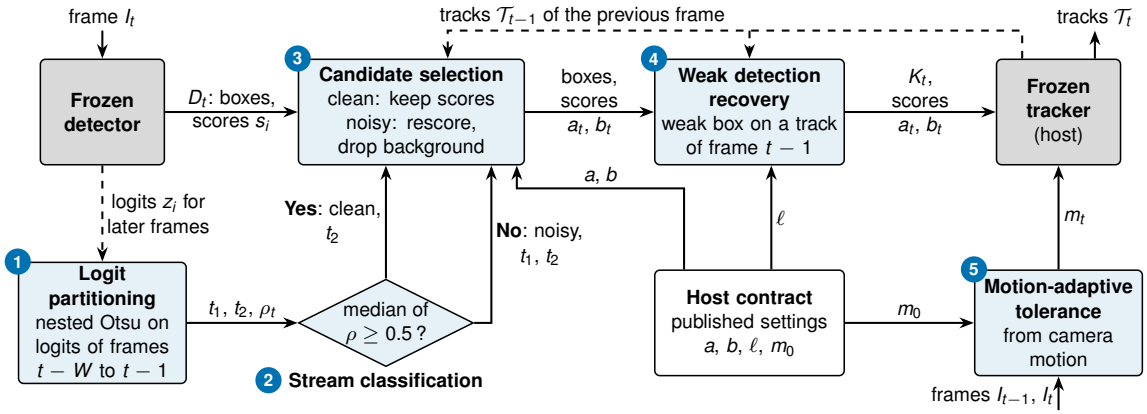


Figure 4.1: SCL between a frozen detector and a frozen tracker (the host), for frame t. Arrows are labeled with the data they carry; dashed arrows carry data from earlier frames. The diamond (Step 2) is a decision: clean (Yes) if the median of ρ is at least 0.5, noisy (No) otherwise. Steps 1–5: Sections 4.3.3–4.3.7; host contract: Section 4.3.8; symbols: Table 4.1.

Table 4.1: The symbols of SCL in plain words. Example: the value in the clean stream of Fig. 4.2a (MOT17-02, frame 101, ByteTrack in its official setting), as recorded by the frozen SCL; "e.g." marks a worked example from the text.

Symbol	Meaning	Example
s_i, z_i	detector score of candidate i, between 0 and 1, and its logit, Eq. (4.1)	e.g. 0.9 and $\log 9 = 2.2$
W, Z_t^W	number of past frames whose logits are pooled; the logits of frames $t-W$ to $t-1$	10; 916 logits
t_1, t_2	first split (background lies below it); second split (confident lies above it)	-0.78 (0.32); 0.95 (0.72)
ρ_t	share of confident candidates in the foreground, Eq. (4.3)	$175/263 = 0.67$
median ρ	median of ρ over the stream so far; clean if ≥ 0.5	0.63: clean
a, b, ℓ	the tracker's published scores to match a track and to start one, and the lowest score it uses	0.6, 0.7, 0.1
m_0	the tracker's matching tolerance: 1 minus the smallest IoU that matches	0.8 (IoU ≥ 0.2)
a_t, b_t, m_t	the thresholds and tolerance SCL passes for frame t, Eqs. (4.4) and (4.6)	0.6, 0.7 (kept); 0.8
u_i, $\tilde{s}_i$	rank of s_i among stored scores; the new score of the noisy mode, Eq. (4.5)	e.g. $u_i = 0.8 \to \tilde{s}_i = 0.9$
$\mathcal{T}_{t-1}$, K_t	the tracks of the previous frame; the candidates passed to the tracker	–
g_t, r_t	camera shift between two frames; its ratio to the usual shift	e.g. $r_t = 2$

so that 0.5 gives $z_i = 0$, 0.9 gives $\log 9 = 2.2$, and 0.1 gives -2.2. Because the logit spreads out the regions near 0 and 1, where detector scores crowd, the splits of Step 1 separate the groups more clearly, and a change of temperature, which divides all logits by the same number, is a simple stretch that the splits follow.

The host gives its contract $h = (a, b, \ell, m_0)$: the score a needed to match a track, the score b needed to start one, the lowest score ℓ it uses, and its matching tolerance m_0, one minus the smallest intersection over union (IoU) that counts as a match. A single-stage tracker has $\ell = \min(a, b)$; a two-stage tracker has $\ell < \min(a, b)$. For ByteTrack in its official setting, $a = 0.6$, $b = 0.7$, $\ell = 0.1$, and $m_0 = 0.8$ (Table 4.2). In every frame the layer returns

$$(K_t, \tilde{s}, a_t, b_t, m_t) = \mathcal{L}\left(Z_{<t}, D_t, \mathcal{T}_{t-1}, g_t;\, h\right) : \tag{4.2}$$

the candidates K_t, their scores, and the thresholds and tolerance for the tracker, from the logits $Z_{<t}$ of earlier frames, the tracks $\mathcal{T}_{t-1}$, and the camera motion g_t between frames I_{t-1} and I_t; no future frame is used.

4.3.3 Otsu-Based Logit Partitioning and Confidence Estimation

Step 1 in Fig. 4.1. Input: the logits Z_t^W of the candidates of the last $W = 10$ frames. *Output:* the splits t_1, t_2 and the confident share ρ_t.

The logits are split twice with Otsu's method [26] (Eq. (2.5)), which picks from the data alone the threshold that best separates two groups: t_1 separates background (low logits) from foreground, and t_2, found on the foreground only, divides it into uncertain and confident candidates. Otsu's method is used because it picks a split without labels or tuning, and applying it twice gives the three groups that one threshold cannot. The share of confident candidates in the foreground,

$$\rho_t = \frac{\left|\{z \in Z_t^W : z \geq t_2\}\right|}{\left|\{z \in Z_t^W : z \geq t_1\}\right|}, \tag{4.3}$$

describes the shape of the score distribution and needs no labels. For example, for the scores 0.02, 0.03, 0.05, 0.4, 0.5, 0.9, and 0.95, the splits fall at $t_1 = -1.67$ (score 0.16) and $t_2 = 1.10$ (score 0.75): three scores are background, two uncertain, and two confident, so $\rho_t = 2/4 = 0.5$. In the clean stream of Fig. 4.2a, the window holds 916 candidates: 653 below $t_1 = -0.78$ (score 0.32), 88 between the splits, and 175 above $t_2 = 0.95$ (score 0.72), so $\rho_t = 175/(88 + 175) = 0.67$; in the noisy stream of Fig. 4.2b, 1,037 are background, 99 uncertain, and 83 confident, so $\rho_t = 83/(99 + 83) = 0.46$. As clutter usually gives more candidates than objects do, a window with fewer candidates

Table 4.2: Host contracts: the four published settings (a, b, ℓ, m_0) of each tracker that SCL reads (Section 4.3.2), and where each value comes from; θ: the published IoU threshold, $m_0 = 1 - \theta$.

Tracker (setting)	Type	a	b	ℓ	m_0	Published settings behind the values
ByteTrack (official)	two-stage	0.6	0.7	0.1	0.8	track_thresh 0.6; new tracks +0.1; second step from 0.1; match_thresh 0.8
ByteTrack (library)	two-stage	0.25	0.25	0.1	0.8	Ultralytics bytetrack.yaml: high 0.25, new 0.25, low 0.1, match 0.8
BoT-SORT (library)	two-stage	0.25	0.25	0.1	0.8	Ultralytics botsort.yaml: the same four values
SparseTrack	two-stage	0.6	0.7	0.1	0.85	track_thresh 0.6; new tracks +0.1; second step from 0.1; match_thresh 0.85
BoostTrack	two-stage	0.6	0.6	0.1	0.7	det_thresh 0.6; every detection from 0.1 can be boosted; $\theta = 0.3$
Hybrid-SORT	two-stage	0.6	0.6	0.1	0.75	det_thresh 0.6; low-score step from 0.1; $\theta = 0.25$
OC-SORT	single-stage	0.6	0.6	0.6	0.7	det_thresh 0.6, no low-score step; $\theta = 0.3$
PD-SORT	single-stage	0.6	0.6	0.6	0.7	det_thresh 0.6, no low-score step; $\theta = 0.3$

below t_1 than above it counts as clean, with $\rho_t = 1$ (background check); this check does not apply in Fig. 4.2.

4.3.4 Median-Based Detection Stream Classification

Step 2 in Fig. 4.1. Input: the values ρ of all frames so far. *Output:* the mode of frame t.

The mode describes the distribution of the scores, not whether a single detection is correct: the stream is clean at frame t if the median of all ρ values so far is at least one half (a majority rule) and noisy otherwise; before the first valid split it is cold. In Fig. 4.2a, frames 2 to 101 give a median of 0.63, so frame 101 is clean; in Fig. 4.2b the median is 0.42, so it is noisy. The whole-stream median keeps the mode stable against a few frames of blur or occlusion, and during development this rule raised the HOTA of ByteTrack and OC-SORT (Section 4.4.2). The groups, and in the clean mode also the thresholds of Step 3, still follow a new score scale within W frames.

4.3.5 Mode-Adaptive Detection Candidate Selection and Threshold Adjustment

Step 3 in Fig. 4.1. Input: D_t, the mode, t_1, t_2, the thresholds a, b, and the tracks $\mathcal{T}_{t-1}$. *Output:* the kept boxes, their scores, and the thresholds a_t, b_t.

Cold and clean modes. The tracker keeps its own scores, and only cross-class duplicates are removed. In the clean mode, a threshold that would reject the confident group is lowered to the score $\sigma(t_2)$ at which that group begins:

$$a_t = \min(a, \sigma(t_2)) \,, \qquad b_t = \min(b, \sigma(t_2)) \,. \tag{4.4}$$

In Fig. 4.2a, $\sigma(t_2) = 0.72$ is above both $a = 0.6$ and $b = 0.7$, so the published thresholds are kept and the output equals that of the host alone, as for BoostTrack, ByteTrack at floor 0.1, and Hybrid-SORT on MOT17 (Section 4.6.1). When the scores are halved, no score is above 0.5 and ByteTrack in its official setting never starts a track; $\sigma(t_2)$ falls with the scores below 0.5, Eq. (4.4) lowers both thresholds to it, and the confident candidates reach the tracker again.

Noisy mode. Background candidates ($z < t_1$) are dropped. Each remaining score is replaced by its mid-rank $u_i \in (0, 1)$ among the scores of a remap memory, which stores the scores kept in the first frame and in every 10th frame, keeping the last 20 such frames (equal scores count half):

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\, u_i, & z_i \geq t_2, \\ 0.1 + 0.4\, u_i, & t_1 \leq z_i < t_2, \end{cases} \tag{4.5}$$

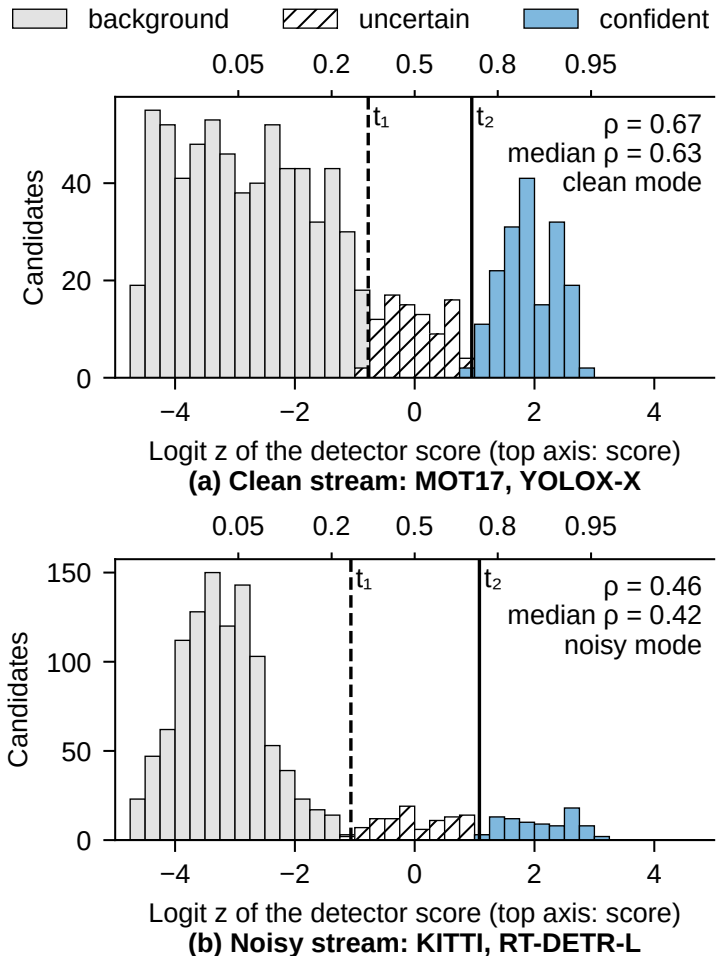


Figure 4.2: What SCL sees before its decision for frame 101: the logits of the candidates of the last 10 frames (top axis: the same position as a score). Dashed line: t_1; solid line: t_2; light gray: background; hatched: uncertain; blue: confident. (a) MOT17 sequence MOT17-02, YOLOX-X, ByteTrack (official): median $\rho = 0.63 \geq 0.5$, clean. (b) KITTI sequence 0001, RT-DETR-L, ByteTrack (library): median $\rho = 0.42 < 0.5$, noisy. Values recorded by the frozen SCL.

and both thresholds are set to $a_t = b_t = 0.5$. A confident candidate with $u_i = 0.8$ gets $0.5 + 0.5 \cdot 0.8 = 0.9$ and may start and continue tracks; an uncertain one with $u_i = 0.3$ gets $0.1 + 0.4 \cdot 0.3 = 0.22$ and may only continue tracks through a tracker's low-score step. A weaker candidate that overlaps a stronger one with IoU above one half is removed, unless a different track of the previous frame claims it, which keeps people who stand close together in a crowd.

Score remapping is not probability calibration. Calibration learns from labeled data the probability that a detection is correct [18]. The remap needs no labels: it keeps the order of the scores within each group and places the groups in fixed ranges, so that the tracker's own thresholds read the layer's decision (above 0.5: "may start a track"; 0.1 to 0.5: "may only continue one"), and it is unchanged by any order-preserving change of the scores.

4.3.6 Track-Guided Weak Detection Recovery

Step 4 in Fig. 4.1. Input: the output of Step 3, the tracks $\mathcal{T}_{t-1}$, and ℓ. *Output:* K_t, their scores, and a_t, b_t for the tracker.

A track can be lost when its object gets low scores for a few frames, above all in a single-stage tracker, which has no low-score step. The rescue passes such a candidate, with its score raised just above the lowest threshold of the frame (ℓ for a two-stage tracker, the lower of a_t and b_t for a single-stage one), when four conditions hold: it is in the foreground (or, for a single-stage tracker, there is no background group), it overlaps a track of the previous frame with an IoU of at least one half, no other usable candidate covers that track, and the tracker could not see it otherwise. For ByteTrack in its official setting, a rescued candidate gets 0.101 and enters its low-score step; for OC-SORT in a noisy frame it gets 0.501. The rescue thus gives single-stage trackers the continuation step they lack, and the tracker itself still decides, with its own matching, whether and to which track the box is assigned.

4.3.7 Camera Motion-Adaptive Matching Tolerance

Step 5 in Fig. 4.1. Input: the frames I_{t-1}, I_t and the tolerance m_0. *Output:* the tolerance m_t of frame t.

In noisy frames, the matching tolerance grows with camera motion, which moves boxes far from their tracks; otherwise $m_t = m_0$. The motion g_t is the shift between consecutive grayscale frames at quarter resolution, found by phase correlation [118], and r_t is its ratio to its median over the last 100 frames ($r_t = 1$ during a warm-up of

5 frames or if this median is zero):

$$m_t = \min\left(0.95,\ 1 - \frac{1 - m_0}{\max(1, r_t)}\right). \tag{4.6}$$

With $m_0 = 0.8$, a match needs an IoU of at least 0.2; if the camera moves twice as much as usual ($r_t = 2$), then $m_t = \min(0.95, 1 - 0.2/2) = 0.9$, and an IoU of 0.1 is enough. The cap of 0.95 keeps a minimum overlap of 0.05.

4.3.8 SCL Integration with Existing Trackers

Each contract value in Table 4.2 comes from the tracker's published configuration; none is fitted. For each tracker family, a small adapter writes the layer's candidates, scores, thresholds, and tolerance into the tracker's existing parameters before each update and then calls the tracker's own update. The matching steps, motion model, track management, and any post-processing of the authors run unchanged, so the host stays frozen: its code, weights, and association algorithm do not change; only the values of its thresholds are set for each frame, and the rules are the same for every tracker.

4.3.9 Fixed SCL Parameters

Table 4.3 lists all values of SCL. They were fixed before the external evaluation, are the same for every tracker, detector, and dataset, and none was chosen by searching over a range of values: they are standard values (the IoU of 0.5 used in MOT evaluation and the common thresholds 0.5 and 0.1), definitions, or design choices kept fixed in all development stages. Algorithm 4.1 gives Steps 1–5 for one frame.

4.4 Development and Freezing of SCL

SCL was developed in three stages on development data: MOT17 with four trackers, KITTI, and the score transforms of Section 4.5. Section 4.4.1 introduces the initial always-on banding (AOB) design (Table 4.4). Section 4.4.2 describes the incremental improvements that led to the final layer, supported by the experimental comparisons in Table 4.5. Finally, Section 4.4.3 explains the configuration freezing and validation performed before external evaluation, to ensure reproducibility and prevent changes based on later results.

4.4.1 Initial Design

The earlier design AOB computed the same nested Otsu groups but treated every stream as noisy: in every frame it applied the groups, removed every weaker candidate

Table 4.3: The fixed values of SCL, their settings, and their sources; the same values are used for every tracker, detector, and dataset.

Value	Setting	Source
Window W	10 frames	design choice
Mode rule	clean if median $\rho \geq 0.5$	majority rule
Background check	$\rho = 1$ if fewer below t_1 than above	rule: no background group
Noisy score ranges	$[0.5, 1)$, $[0.1, 0.5)$	common thresholds
Noisy a_t, b_t	0.5	common threshold
Remap memory	20 frames, one every 10	design choice
Duplicate, rescue IoU	0.5	MOT evaluation IoU
Rescued score	lowest threshold $+\,0.001$	just above the threshold
Motion cue	phase correlation, quarter resolution	standard method [118]
Motion history	100 frames, warm-up 5	design choice
Tolerance cap	0.95	minimum overlap 0.05

Algorithm 4.1 SCL for frame t; the comments name the steps of Fig. 4.1. The window holds the logits of frames $t - W$ to $t - 1$, so the decision for frame t uses no future frame.

Input: candidates D_t, tracks $\mathcal{T}_{t-1}$, motion cue g_t, contract (a, b, ℓ, m_0), constants of Table 4.3
Output: candidates K_t, scores $\tilde{s}$, thresholds a_t, b_t, tolerance m_t

$z_i \leftarrow$ logit of s_i for all i ▷ Eq. (4.1)
if no valid split exists **then** ▷ e.g., first frame
    mode $\leftarrow$ cold; $(a_t, b_t) \leftarrow (a, b)$
else
    $(t_1, t_2) \leftarrow$ nested Otsu on the window ▷ Step 1
    $\rho_t \leftarrow$ Eq. (4.3) with the background check
    mode $\leftarrow$ clean if median$(\rho) \geq 0.5$, else noisy ▷ Step 2
end if
if mode is cold or clean **then** ▷ Step 3
    remove cross-class duplicates; keep raw scores
    if clean **then** set (a_t, b_t) by Eq. (4.4)
    end if
else ▷ noisy mode
    remove duplicates with track context
    drop $z < t_1$; remap scores by Eq. (4.5)
    $a_t \leftarrow 0.5$; $b_t \leftarrow 0.5$
end if
rescue continuations (not cold) ▷ Step 4
$m_t \leftarrow$ Eq. (4.6) if noisy, else m_0 ▷ Step 5
pass K_t, $\tilde{s}$, a_t, b_t, m_t to the tracker
add the logits left after duplicate removal to the window
add g_t to the motion history
store the tracks of frame t after the tracker update

that overlapped a stronger one with IoU above one half, and rescored all candidates by rank. On strong MOT17 trackers (Table 4.4), AOB lowered HOTA by 4.15 to 5.89 points, as the overlap rule deleted people hidden behind others in crowds and its thresholds were stricter than the trackers' own; with the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left BoostTrack, online and with its gap-filling post-processing (GBI), identical to the host. The next steps therefore let the layer decide from the scores whether to act at all.

4.4.2 Incremental Design Improvements

Each step was compared with the step before it (Table 4.5). The *mode gate* added the mode check, the clean mode that keeps the tracker's scores and thresholds, and the crowd-safe duplicate rule; it restored SparseTrack from 64.72 to 68.93 HOTA and BoostTrack from 62.61 to 68.37. The *whole-stream mode* removed mode changes in the middle of streams and raised ByteTrack from 67.561 to 67.702 and OC-SORT from 65.697 to 66.071. The *continuation rescue* added 0.2 to 0.4 points for OC-SORT. The *background check* completed SCL: ByteTrack and BoostTrack returned to output identical to the host, and OC-SORT rose to 67.04 and 66.91 at the two floors.

4.4.3 Configuration Freezing and Validation

The final configuration was frozen before any external test as a version-control tag, with a lock file holding a cryptographic hash of the layer, its configuration, and the evaluation scripts; sixty-one automated tests check, among other things, that the layer uses no future data and passes clean streams unchanged. All SCL results of this chapter come from this frozen configuration; the comparisons, the ablation, and the sensitivity analysis were run after the freeze, through run-time options of the evaluation scripts, and changed none of its values.

4.5 Experimental Setup

The experiments ask whether the frozen layer helps a tracker whose detector does not fit its thresholds, whether it leaves a well-matched tracker unchanged, and whether both effects hold on trackers and data that were not used to design it. Table 4.6 summarizes the setup.

Data. The datasets are described in Section 2.3.7. On MOT17 val-half (2,652 frames), the trackers use the published YOLOX-X detections [16] at the two published emission floors (the lowest scores emitted) of 0.01 and 0.1. To study score changes directly, these scores were also changed before the tracker by three transforms that keep their order: cubing (s^3), halving ($0.5\,s$), and logit temperature $T = 2$, which

Table 4.4: MOT17 val-half, published YOLOX-X detections: the earlier design AOB and the final SCL, each added to the same strong tracker (Host). Change: the run with the layer minus Host, for AOB with its 95% paired-bootstrap interval (10,000 resamples, seed 42); bold: the interval does not include zero (a reliable change).

Metric	Host	Host+AOB	AOB change [95% CI]	Host+SCL	SCL change
SparseTrack [89]					
HOTA	68.88	64.72	−4.15 [−5.54, −1.66]	68.93	+0.06
MOTA	77.85	71.71	−6.14 [−8.10, −2.74]	77.93	+0.08
IDF1	81.97	77.49	−4.49 [−6.19, −1.96]	82.13	+0.16
BoostTrack, online [90]					
HOTA	68.49	62.61	−5.89 [−7.76, −2.87]	68.49	0 (identical)
MOTA	75.50	66.64	−8.87 [−12.37, −7.14]	75.50	0 (identical)
IDF1	81.41	75.19	−6.22 [−9.33, −2.85]	81.41	0 (identical)
BoostTrack, with GBI [90]					
HOTA	71.73	66.35	−5.37 [−7.30, −2.75]	71.73	0 (identical)
MOTA	81.03	72.30	−8.73 [−12.59, −6.50]	81.03	0 (identical)
IDF1	84.16	78.51	−5.65 [−8.96, −2.77]	84.16	0 (identical)

Table 4.5: How SCL was built from AOB on development data. Each row adds one change to the row above; Effect gives the HOTA (%) before and after that change ("A to B") or compared with the tracker alone (host). The number in parentheses after ByteTrack and OC-SORT is the lowest emitted detector score.

Step	Change	Effect
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ mode gate	mode statistic, host-anchored clean mode, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream mode	mode from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)

divides the logit of each score by T. On KITTI tracking (21 training sequences, 8,008 frames), the trackers run with two other detectors, YOLOv8n [66] and RT-DETR-L [71], and keep thresholds that were set for other detectors. On the UAVDT test split (20 sequences, 16,592 frames), ByteTrack runs with YOLOv8n and RT-DETR-L.

Development and external data. MOT17 with four trackers, KITTI, and the score transforms were development data, looked at while the layer was designed. Two predeclared external trackers, PD-SORT and Hybrid-SORT, were chosen in advance and run once with the frozen configuration on the detections released by their authors on MOT17 val-half. The held-out UAVDT test split was also run once, after a written plan. Each external host was first checked against its authors' number: EXACT within 0.2 HOTA points, CLOSE within 1.0.

Trackers and baseline. Seven published trackers serve as hosts, each with the authors' code and published configuration: ByteTrack [13], BoT-SORT [83], OC-SORT [81], SparseTrack [89], BoostTrack [90], PD-SORT [91], and Hybrid-SORT [88]. ByteTrack is used with the thresholds that its authors used on MOT17 (official setting) and with the defaults of the Ultralytics library, which also provides BoT-SORT (library setting). For each host, the baseline is the host alone (Host in the tables), run with its published configuration and the same detections; the second run adds the frozen SCL (Host+SCL) and changes nothing else.

Evaluation and statistics. All metrics are computed with TrackEval (commit 12c8791) and the official ground truth [97]. KITTI HOTA is averaged over cars and pedestrians; on UAVDT all vehicles form one class and predictions in ignored regions are removed. Change is Host+SCL minus Host, and its 95% interval comes from the paired sequence-level bootstrap of Section 2.3.8 with 10,000 resamples (seed 42).

Runtime. The runtime of SCL is measured on a central processing unit (CPU) alone—a 4-vCPU Intel Xeon processor at 2.10 GHz without a GPU—with YOLOv8n and ByteTrack on three KITTI sequences (2,279 timed frames).

4.6 Results

This section evaluates the effect and the generalization of SCL against the host trackers, using identical detections and published tracker configurations. Section 4.6.1 examines tracking performance across different trackers on MOT17, Section 4.6.2 evaluates detector–tracker compatibility on KITTI, and Section 4.6.3 investigates robustness to shifts of the detector scores. Sections 4.6.4 and 4.6.5 assess the generalization to predeclared trackers not seen during design and to the held-out UAVDT test split. In all comparisons, the baseline is the host tracker running without SCL, and Fig. 4.3 summarizes the HOTA differences across the evaluated configurations.

How to read the result tables. Each row gives one metric of the tracker alone (Host)

Table 4.6: Experimental setup of SCL.

Item	Setting
Detection sources	YOLOX-X (published MOT17 detections, floors 0.01 and 0.1); detections released with PD-SORT and Hybrid-SORT; YOLOv8n; RT-DETR-L
Trackers (hosts)	ByteTrack (official and library settings), BoT-SORT (library), OC-SORT, SparseTrack, BoostTrack (online and with GBI), PD-SORT, Hybrid-SORT
Score transforms	s^3, $0.5\,s$, logit temperature $T = 2$ (MOT17)
Development data	MOT17 val-half with four trackers; KITTI tracking training; score transforms
External runs	PD-SORT and Hybrid-SORT on MOT17 val-half (predeclared, run once); UAVDT test (written plan, run once)
Baseline	host alone with its published configuration and the same detections
Scoring	TrackEval (commit 12c8791), official ground truth; KITTI averaged over cars and pedestrians; UAVDT one vehicle class
Statistics	paired sequence-level bootstrap, 10,000 resamples, seed 42
Runtime	4-vCPU Intel Xeon at 2.10 GHz, no GPU; YOLOv8n and ByteTrack, 2,279 timed frames

and with SCL (Host+SCL), then the Change (Host+SCL minus Host) with its 95% paired-bootstrap interval in brackets (Section 2.3.8). A positive change in HOTA, MOTA, or IDF1 and a negative change in IDS mean that SCL helped, and a bold change is reliable (its interval does not include zero); "identical" means an unchanged output. For example, in Table 4.7, OC-SORT at floor 0.01 has a MOTA of 74.67 alone and 75.94 with SCL: the change is +1.27 points, and its interval [+0.17, +3.01] lies above zero, so the gain is reliable. Values are rounded separately, so Host+SCL minus Host can differ from Change in the last digit.

4.6.1 MOT17 Evaluation across Trackers

Table 4.7 shows the development trackers on MOT17 val-half. The two-stage trackers with well-matched detections kept the accuracy of their hosts: BoostTrack (online and with GBI) and ByteTrack at floor 0.1 gave exactly the same output through the self-limiting clean mode. The single-stage OC-SORT improved at both floors (HOTA from 66.43 to 67.04 and from 66.44 to 66.91, MOTA +1.27 and +1.26), with 8 fewer identity switches at floor 0.01 and higher HOTA on 7 of 7 sequences, through the continuation rescue.

Table 4.7: MOT17 val-half, published YOLOX-X detections: each development tracker alone (Host) and with SCL (Host+SCL). Change: Host+SCL minus Host, with its 95% paired-bootstrap interval (10,000 resamples, seed 42); bold: a reliable change. Paper: the value reported by the tracker's authors (–: not reported). Floor: the lowest emitted detector score. IDS: identity switches, a count (lower is better).

Metric	Paper	Host	Host+SCL	Change [95% CI]
BoostTrack, online [90]				
HOTA	68.37	68.49	68.49	0 (identical)
MOTA	75.56	75.50	75.50	0 (identical)
IDF1	81.35	81.41	81.41	0 (identical)
IDS	118	113	113	0 (identical)
BoostTrack, + GBI [90]				
HOTA	71.33	71.73	71.73	0 (identical)
MOTA	80.55	81.03	81.03	0 (identical)
IDF1	83.84	84.16	84.16	0 (identical)
IDS	106	97	97	0 (identical)
ByteTrack, floor 0.1 [13]				
HOTA	–	67.70	67.70	0 (identical)
MOTA	76.6	77.60	77.60	0 (identical)
IDF1	79.3	79.47	79.47	0 (identical)
IDS	–	214	214	0 (identical)
OC-SORT, floor 0.01 [81]				
HOTA	66.5	66.43	67.04	+0.61 [+0.36, +1.24]
MOTA	74.9	74.67	75.94	+1.27 [+0.17, +3.01]
IDS	–	211	203	−8 [−15, −2]
OC-SORT, floor 0.1 [81]				
HOTA	66.5	66.44	66.91	+0.46 [+0.27, +1.09]
MOTA	74.9	74.67	75.93	+1.26 [+0.37, +3.34]
IDF1	77.7	78.05	78.33	+0.287 [+0.003, +0.838]

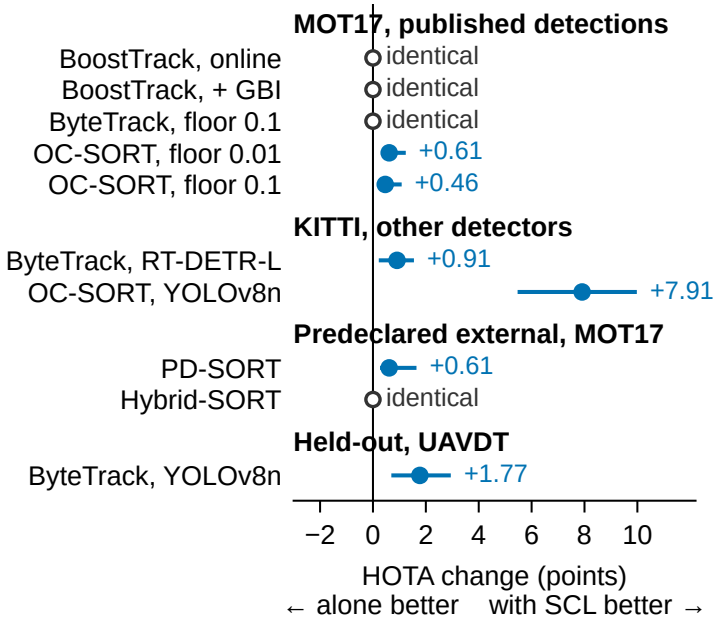


Figure 4.3: HOTA change with SCL (tracker with SCL minus tracker alone, in points) on MOT17, KITTI, and UAVDT. Dot and number: the change; line: its 95% bootstrap interval; right of zero, SCL is better. Filled dot: a reliable gain; open dot: identical output.

4.6.2 KITTI Evaluation with Other Detectors

Table 4.8 shows what happens when the detector is replaced. With the noisy RT-DETR-L, SCL raised ByteTrack by 5.98 MOTA, 3.87 IDF1, and 0.91 HOTA points and BoT-SORT by 7.86 MOTA and 3.10 IDF1 points, with 297 and 226 fewer identity switches. With YOLOv8n, OC-SORT rose from 36.27 to 44.18 HOTA (+7.91 points, [+5.47, +9.98]), 9.40 MOTA points, and 9.60 IDF1 points, as its missed objects (FN) fell by 5,310. Figure 4.4 follows this cell over three frames: each object keeps its ID, and at frame 102 OC-SORT alone tracked 6 of the 10 labeled objects, and with SCL it tracked 9.

4.6.3 Evaluation under Shifted Detector Scores

Table 4.9 shows the trackers under the score transforms. Halving the scores reduced ByteTrack (official setting) and OC-SORT to 0 HOTA; with the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60; BoostTrack rose from 61.33 to 64.84. With $T = 2$, the layer raised ByteTrack and OC-SORT by 1.11 and 1.24 points. Across the score changes, the layer raised HOTA significantly in 7 conditions. Figure 4.5 shows the same values as bars: with the layer, the HOTA of every tracker lies between 64.84 and 67.59 under every score change, including the two cases in which the halved scores reduced the tracker alone to 0.

Table 4.8: KITTI tracking training with detectors the trackers were not tuned for (official HOTA, averaged over cars and pedestrians); columns as in Table 4.7. BoT-SORT leaves out sequence 0020 in both runs, because its code stops with a numerical error on it; the other 20 sequences are used.

Metric	Host	Host+SCL	Change [95% CI]
ByteTrack [13], *RT-DETR-L*			
HOTA	50.73	51.64	+0.91 [+0.22, +1.55]
MOTA	47.27	53.24	+5.98 [+1.16, +9.94]
IDF1	64.98	68.86	+3.87 [+1.67, +5.06]
IDS	686	389	−297 [−530, −122]
BoT-SORT [83], *RT-DETR-L* (20 sequences)			
MOTA	48.25	56.11	+7.86 [+2.21, +13.44]
IDF1	67.74	70.85	+3.10 [+1.00, +4.57]
IDS	476	250	−226 [−369, −120]
OC-SORT [81], *YOLOv8n*			
HOTA	36.27	44.18	+7.91 [+5.47, +9.98]
MOTA	33.99	43.39	+9.40 [+1.10, +12.30]
IDF1	50.46	60.05	+9.60 [+5.61, +12.16]

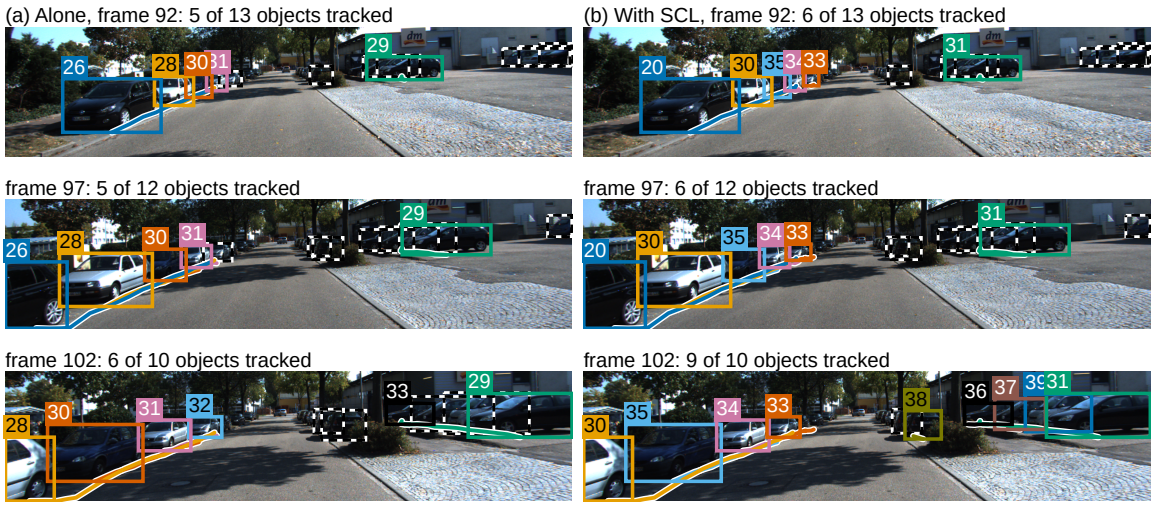


Figure 4.4: KITTI sequence 0001 [8] tracked by OC-SORT with YOLOv8n at frames 92, 97, and 102 (rows): (a) the tracker alone; (b) with SCL. Each color and number is one track; its line shows the object's last 20 frames. Dashed boxes: labeled objects that no track follows. IDs are numbered separately in each run. The sequence has the median HOTA gain of the 21 sequences, and frame 102 is its first frame with the largest gain in tracked objects.

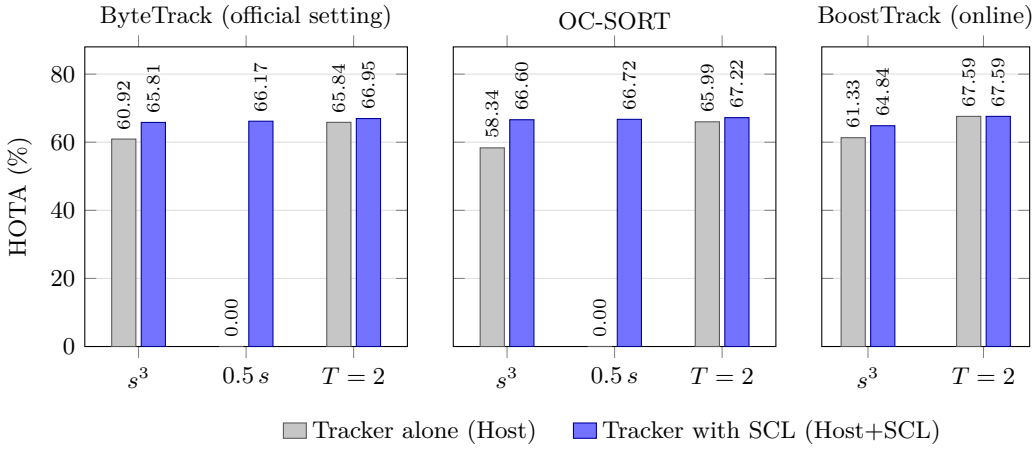


Figure 4.5: HOTA (%) of the tracker alone and with SCL on MOT17 val-half when the detector scores are changed before the tracker (values of Table 4.9): s^3 cubes every score, $0.5\,s$ halves it, and $T = 2$ divides its logit by 2.

Table 4.9: MOT17 val-half with the detector scores changed before the tracker, which keeps its published thresholds: s^3 cubes every score, $0.5\,s$ halves it, and $T = 2$ divides its logit by 2. HOTA (%) of the tracker alone (Host) and with SCL (Host+SCL); 0.00: no object tracked; Change with its 95% paired-bootstrap interval; bold: a reliable change. Lowest emitted score 0.01 (BoostTrack 0.1).

Score change	Host	Host+SCL	Change [95% CI]
ByteTrack, official setting [13]			
s^3	60.92	65.81	+4.89 [+2.60, +13.69]
$0.5\,s$	0.00	66.17	+66.17 [+49.32, +75.40]
$T = 2$	65.84	66.95	+1.11 [+0.35, +3.87]
OC-SORT [81]			
s^3	58.34	66.60	+8.26 [+5.86, +16.81]
$0.5\,s$	0.00	66.72	+66.72 [+50.32, +75.56]
$T = 2$	65.99	67.22	+1.24 [+0.52, +3.42]
BoostTrack, online [90]			
s^3	61.33	64.84	+3.51 [+0.85, +13.28]
$T = 2$	67.59	67.59	0 (identical)

4.6.4 Frozen Evaluation on External Trackers

Table 4.10 gives the two predeclared external systems. PD-SORT [91] was reproduced exactly; with SCL, its HOTA rose from 68.01 to 68.62 (+0.61 points, [+0.27, +1.65]), its MOTA by 1.13 ([+0.27, +3.03]), and its IDF1 by 0.82 ([+0.43, +1.94]), with higher HOTA on 7 of 7 sequences. Like OC-SORT, it is a single-stage tracker, but it was not seen during design. Hybrid-SORT [88], a two-stage tracker, gave identical output, as predicted before the run.

4.6.5 Held-Out UAVDT Evaluation

On the held-out UAVDT test split (Table 4.11), SCL raised the HOTA of the YOLOv8n stream from 29.05 to 30.83 (+1.77 points, [+0.70, +2.94]) and its IDF1 by 3.97 points, and cut identity switches by 45%. With RT-DETR-L, whose host produced many false positives (MOTA 3.29), SCL raised MOTA from 3.29 to 27.99 (+24.70 points, [+10.06, +51.08]) and IDF1 by 5.74 points, and cut false positives by 58% and identity switches by 56%.

4.7 Comparison with Adaptive-Threshold Methods

SCL adapts the thresholds of the tracker, so its gains must be compared with methods that adapt the thresholds alone. This section compares it with two simple adaptive baselines built from the same Otsu splits and with ACT, the closest published method (Section 2.4.3).

4.7.1 Comparison with Adaptive-Threshold Baselines

Two simple baselines set the tracker's matching and birth thresholds in every frame to the first or second Otsu split of the logits of the last 10 frames (t_1 or t_2) and pass the scores unchanged. On KITTI (Table 4.12), SCL reached the highest HOTA in all three cells, 2.2 to 8.6 points above the baselines, with intervals against the t_2 baseline above zero in every cell, so adaptive thresholds alone do not explain the gains.

Logit-to-score conversion. The splits are found on logits and converted to scores with the logistic function; the thresholds recorded in every frame equaled the intended values: $\sigma(t_1)$ or $\sigma(t_2)$ in all eight baseline runs, and $\min(a, \sigma(t_2))$ (Eq. (4.4)) or 0.5 in the four KITTI cells of SCL, so every method received the thresholds it was meant to have.

Table 4.10: The two external trackers, chosen before the run and not used during design (MOT17 val-half, the authors' detections), each run once with the frozen SCL; columns as in Table 4.7. Paper: the authors' number (for PD-SORT, their released output). EXACT: Host within 0.2 HOTA of it; CLOSE: within 1.0.

Metric	Paper	Host	Host+SCL	Change [95% CI]
PD-SORT [91], *EXACT*				
HOTA	68.01	68.01	68.62	+0.61 [+0.27, +1.65]
MOTA	75.19	75.19	76.31	+1.13 [+0.27, +3.03]
IDF1	81.03	81.03	81.85	+0.82 [+0.43, +1.94]
Hybrid-SORT [88], *CLOSE*				
HOTA	67.1	66.70	66.70	0 (identical)
MOTA	75.8	75.52	75.52	0 (identical)
IDF1	78.0	77.56	77.56	0 (identical)
IDS	–	259	259	0 (identical)

Table 4.11: The held-out UAVDT test split (20 drone videos, not used during design), run once with the frozen SCL and ByteTrack with its published setting; columns as in Table 4.8.

Metric	Host	Host+SCL	Change [95% CI]
ByteTrack, RT-DETR-L [13, 71]			
MOTA	3.29	27.99	+24.70 [+10.06, +51.08]
IDF1	48.59	54.33	+5.74 [+0.13, +13.11]
IDS	1041	455	−586 [−951, −248]
ByteTrack, YOLOv8n [13, 66]			
HOTA	29.05	30.83	+1.77 [+0.70, +2.94]
IDF1	34.52	38.49	+3.97 [+1.88, +6.17]
IDS	719	393	−326 [−691, −73]

Table 4.12: KITTI tracking training: HOTA of the tracker alone (Host), of two adaptive baselines that set the tracker's thresholds in every frame to the first (t_1) or second (t_2) Otsu split of the logits of the last 10 frames, and of SCL. Halved: halved detector scores. SCL − t_2: the gain over the stronger baseline, with its 95% paired-bootstrap interval; bold: a reliable gain.

Cell	Host	t_1	t_2	SCL	SCL − t_2 [95% CI]
OC-SORT, YOLOv8n	36.27	39.97	41.09	44.18	+3.09 [+2.00, +4.03]
ByteTrack, RT-DETR-L, halved	49.50	49.58	49.50	51.79	+2.29 [+0.28, +4.96]
OC-SORT, YOLOv8n, halved	0.00	37.68	43.56	46.30	+2.74 [+1.32, +3.34]

4.7.2 Comparison with ACT

ACT [25], the closest published method, sets ByteTrack's high and low thresholds in every frame from the first quartile and the number of the current scores, choosing between preset levels. It was run with its published parameters, without retuning, on the same detections and hosts as SCL (Tables 4.13 and 4.14): its high threshold became the host's matching threshold (with OC-SORT, its single threshold) and its low threshold the lower bound of the second step. With the host's own thresholds, the ACT runner reproduced the host exactly in all 84 KITTI and 40 UAVDT sequence runs. SCL reached significantly higher HOTA than ACT in every row of Table 4.13: +2.01 and +10.06 points under halved scores and +1.82 with YOLOv8n on UAVDT. On UAVDT with RT-DETR-L (Table 4.14), SCL reached a MOTA of 27.99 against 5.34 for ACT (+22.65, [+9.03, +47.21]) and had 418 fewer identity switches, because the fixed score levels of ACT removed few of the host's false positives (178,219 against 78,975).

4.8 Component Ablation and Parameter Sensitivity

Table 4.15 reports the effect of removing one part of the frozen SCL, or changing one of its values, at a time on the three KITTI cells of Table 4.12. Each part matters where its step acts. Without the weak detection recovery, the single-stage OC-SORT lost up to 3.35 HOTA points, as it has no low-score step of its own; without the noisy mode (always clean), the halved-score cells lost up to 2.44 points, without the remap up to 1.67, and without the motion tolerance up to 0.14. The clean mode, in turn, keeps well-matched trackers at their own accuracy (Table 4.4).

The sensitivity analysis, planned before the runs, changed the window W, the frames whose median decides the mode, the IoU of the duplicate and recovery rules, and the motion history. For every value and cell, HOTA stayed within 0.62 points of the frozen SCL, and within 0.24 points on a fourth cell (ByteTrack with the original

Table 4.13: ACT [25], with its published parameters, and SCL on the same detections and tracker: HOTA with each method, and SCL − ACT with its 95% paired-bootstrap interval; bold: a reliable difference. KITTI: official HOTA, averaged over cars and pedestrians; UAVDT: test split.

Data	Detector	Tracker	ACT	SCL	SCL − ACT [95% CI]
KITTI, halved	RT-DETR-L	ByteTrack	49.77	51.79	+2.01 [+0.01, +4.76]
KITTI, halved	YOLOv8n	OC-SORT	36.24	46.30	+10.06 [+6.88, +12.01]
UAVDT test	YOLOv8n	ByteTrack	29.01	30.83	+1.82 [+0.65, +3.00]

Table 4.14: MOTA and identity switches of ACT and SCL on the UAVDT test split with ByteTrack and RT-DETR-L. A positive MOTA difference and a negative IDS difference favor SCL; bold: a reliable difference.

Metric	ACT	SCL	SCL − ACT [95% CI]
MOTA	5.34	27.99	+22.65 [+9.03, +47.21]
IDS	873	455	−418 [−718, −123]

RT-DETR-L scores). Thus, none of the values needs tuning for these detectors and trackers.

4.9 Runtime

SCL needs no training, no GPU, and no access to the inside of the detector or the tracker. On the processor of Table 4.6 it takes 2.95 ms per frame, small compared with the 42 ms of the detector, and the tracker itself ran faster with it (1.43 against 2.04 ms per frame); the end-to-end time rose by only 4.5%. A new tracker is added through its four published contract values and an adapter, without changes to the rules.

4.10 Discussion

The score scale is part of the MOT system. Halving the scores reduced ByteTrack and OC-SORT from 67.70 and 66.43 HOTA to 0, and with RT-DETR-L on UAVDT ByteTrack reached a MOTA of only 3.29. A tracker's fixed thresholds hold only for the detector they were tuned for, so the score scale deserves the same attention as the association method. SCL treats the compatibility between a detector's score scale and a tracker's thresholds as a property that can be checked and restored online, which answers RQ2.

Selective by design. In the clean mode SCL keeps the tracker's thresholds and scores, so a well-matched tracker receives its normal input: BoostTrack, ByteTrack at floor 0.1, and Hybrid-SORT gave exactly the output of their hosts. This is the main difference from the earlier design AOB, which acted in every frame and lowered the HOTA of the same strong trackers by 4.15 to 5.89 points (Table 4.4). The layer therefore does not need to know in advance whether a tracker needs help; it reads the answer from the scores.

Host-aware adaptation. Single-stage trackers gained mainly through the continuation rescue, which gives them the step that two-stage trackers already have: OC-SORT and PD-SORT (a tracker not seen during design) on MOT17, and OC-SORT with YOLOv8n on KITTI, where removing the rescue cost up to 3.35 HOTA points (Table 4.15). Two-stage trackers gained when the detector was noisy or its scores shifted,

Table 4.15: What each part and each fixed value of SCL contributes on the three KITTI cells of Table 4.12 (HOTA): (a) OC-SORT, YOLOv8n; (b) ByteTrack, RT-DETR-L, halved scores; (c) OC-SORT, YOLOv8n, halved scores. Upper part: one part removed at a time, with the largest HOTA loss and its cell. Lower part: one fixed value changed at a time (frozen value in brackets), with the largest HOTA change over the tested values and cells. Frozen SCL, HOTA: (a) 44.18, (b) 51.79, (c) 46.30.

Part removed	Largest loss	Cell
Weak detection recovery	3.35	(a)
Noisy mode (always clean)	2.44	(c)
Remap (raw scores)	1.67	(b)
Motion tolerance	0.14	(c)
Value changed (frozen)	**Largest change**	**Tested**
Window W (10 frames)	0.56	5, 20
Mode median frames (all)	0.36	last 100, 300
Duplicate/recovery IoU (0.5)	0.62	0.4, 0.6
Motion history (100 frames)	0.09	50, 200

as did ByteTrack and BoT-SORT with RT-DETR-L. The same frozen rules served both kinds of tracker through their published contract values.

Wide applicability. One frozen configuration served seven trackers of both kinds, four detection sources, and three datasets of people and vehicles filmed from the ground, from a car, and from drones (RQ3). The splits move exactly with a stretch or shift of the logits, and the layer also restored the trackers under cubed scores (Section 4.6.3), so it follows the common ways in which a detector's score scale changes.

Deployment. The layer runs on an ordinary processor in a few milliseconds per frame (Section 4.9), and a new tracker is added through its four published contract values and an adapter, without changes to the rules. The label-free mode and the confident share ρ can also be logged as a running check that a new or retrained detector still fits its tracker.

4.11 Summary

This chapter treated the compatibility between a detector's score scale and a tracker's thresholds as a property that can be checked and restored online, without labels and without changing either component. SCL checks it with nested Otsu splits of recent logits and a whole-stream majority rule, leaves compatible pairs as they are, and restores incompatible ones through the tracker's own published settings, with a track-guided recovery of weak detections that gives single-stage trackers the continuation step they lack. The layer was developed step by step from an earlier always-on design and frozen before any external test. The frozen layer was tested on seven trackers and four detec-

tion sources. Under halved scores, it restored two trackers from 0 to about 66 HOTA; with a mismatched detector on held-out aerial data, it raised MOTA by up to 24.70 points; well-matched trackers kept their own accuracy; it reached higher HOTA than simple adaptive thresholds and than the published ACT method; and it took 2.95 ms per frame on average on a CPU. Detector–tracker compatibility can therefore be managed as part of the tracking system, so a detector can be replaced or retrained without retuning the tracker. The next chapter draws the conclusions of the thesis as a whole.

Chapter 5

Conclusions and Future Work

5.1 Conclusions

This thesis started from a simple observation about tracking-by-detection systems: the settings that connect the detector and the tracker—the detector operating point and the score thresholds of the tracker—are usually fixed once for one detector and one kind of scene, although the scene changes within a video and the detector changes during the life of a system. The research problem was to adapt these settings online, from information that is available while the video is processed, without labels, without training, and with the detector and the tracker left unchanged (Section 1.3).

The published survey [5] provided the starting point. It reviewed the detectors, trackers, datasets, and metrics of real-time detection and tracking, showed that tracking by detection remains the dominant paradigm, and pointed to the dependence between detection accuracy and association reliability under real-time constraints. Together with the review of related work in Chapter 2, it identified the gap addressed by the thesis: existing adaptive methods either learn a policy for one detector, adapt one tracker from inside, or require labeled data.

The first controller, AC-MOT (Chapter 3), answered RQ1 for UAV video. It adapts the confidence threshold, the NMS threshold, and the input resolution of the detector every ten frames. A scene complexity index formed from the crowd cue, checks from four further inexpensive scene cues, and a three-state controller with recovery, hysteresis, and dwell rules choose one of three operating points; the values of the controller were selected by cross-validation on separate calibration sequences and then frozen. On seven VisDrone2019-MOT sequences it improved HOTA by 2.937 points with ByteTrack and 1.325 points with OATrack, and MOTA by 17.995 and 6.318 points, with fewer identity switches and false positives; paired bootstrap analysis supports these gains. Qualitative experiments on the tracked frames of a VisDrone video made this effect visible: far fewer false tracks, fewer identities per object, and a stable operating

point in a crowded scene. On UAVDT it improved MOTA by 17.601 and 3.267 points without retuning, and it adds about 4.2 ms per frame on a Tesla T4.

The second controller, SCL (Chapter 4), answered RQ2. It reads the score distribution of recent frames with nested Otsu splits and a whole-stream majority rule. On clean scores, it keeps the tracker's scores and, where they fit, its thresholds; on noisy scores, it chooses the candidates, rescores them, and sets the thresholds. A track-guided recovery gives single-stage trackers the continuation step they lack, and a host contract of four published settings lets one configuration serve different trackers. One frozen configuration restored two trackers from 0 to about 66 HOTA under halved scores, improved both single-stage trackers, including one not seen during design, on every MOT17 sequence, raised MOTA by up to 24.70 points with a noisy detector on held-out aerial data, cut the identity switches of two-stage trackers by 43% to 56% with replacement detectors, and kept well-matched trackers at their own accuracy. It reached higher HOTA than simple adaptive thresholds (by 2.2 to 8.6 points) and than the published ACT method, and it takes 2.95 ms per frame on an ordinary processor.

Both controllers kept their effect beyond the data used to design them, at a cost of a few milliseconds per frame, which answers RQ3: AC-MOT on UAVDT without retuning, and SCL on two predeclared external trackers and on the held-out UAVDT test split, with detectors for which the trackers were not tuned. Table 5.1 summarizes the two controllers and their main results, and Table 5.2 relates each research question to its principal evidence and main finding.

Taken together, the two studies advance the understanding of robust tracking in two ways. First, they show that the operating point of the detector and the score scale that the tracker receives are parts of the tracking system that deserve the same attention as the association method: with the detector and the tracker unchanged, each of them moved MOTA or HOTA by many points. Second, they show that both can be controlled from outside the detector and the tracker, with rules whose values were fixed before the final evaluation: AC-MOT selects the detector settings that suit the current aerial scene, and SCL makes the scores of a given detector usable by different trackers and leaves a well-matched pair unchanged. The two controllers are distinct methods that answer different questions: AC-MOT acts before the detector and does not change the thresholds of the tracker, and SCL acts between the detector and the tracker and does not choose the detector operating point. Each was developed, frozen, and evaluated on its own, AC-MOT in Chapter 3 and SCL in Chapter 4.

Practical implications. Neither controller needs training, labels, or access to the inside of the detector or the tracker. AC-MOT changes three arguments of the detector call, and SCL sets the published thresholds and the input of the tracker for each frame. A system integrator can therefore keep a tuned tracker when the scenes change or when the detector is replaced or retrained, and the label-free mode of SCL can be logged as

Table 5.1: Summary of the two proposed controllers and their main results; all changes are relative to the same pipeline without the controller.

Aspect	AC-MOT	SCL
Question	RQ1: adapt the detector operating point to the scene	RQ2: restore the compatibility of score scale and tracker thresholds
Acts on	detector operating point	tracker input and thresholds
Evaluated on	VisDrone2019-MOT and UAVDT with ByteTrack and OATrack	MOT17, KITTI, and UAVDT with seven published trackers
Main results	VisDrone: HOTA +2.937 and +1.325, MOTA +17.995 and +6.318; UAVDT: MOTA +17.601 and +3.267	halved scores: 0 to 66.17 and 66.72 HOTA; KITTI: MOTA +5.98 and +7.86; UAVDT: MOTA +24.70
Cost	4.26 and 4.20 ms per frame (Tesla T4)	2.95 ms per frame (CPU)

Table 5.2: Research questions of the thesis, the principal experiments that answer them, and the main findings.

Research question	Principal evidence	Main finding
RQ1: online control of the detector operating point	AC-MOT on seven VisDrone2019-MOT sequences with ByteTrack and OATrack, and frozen transfer to UAVDT (Sections 3.6.1–3.6.3)	over the fixed operating point: HOTA +2.937 and +1.325 and MOTA +17.995 and +6.318 on VisDrone; MOTA +17.601 and +3.267 on UAVDT without retuning
RQ2: label-free compatibility of scores and tracker thresholds	SCL under halved scores, with replacement detectors, and on MOT17, KITTI, and UAVDT, compared with adaptive thresholds and ACT (Sections 4.6 and 4.7)	two trackers restored from 0 to about 66 HOTA; well-matched trackers kept at their own accuracy; higher HOTA than simple adaptive thresholds (by 2.2 to 8.6 points) and than ACT
RQ3: effect on unseen data, trackers, and detectors, and cost	AC-MOT on UAVDT without retuning; SCL on two predeclared external trackers and the held-out UAVDT test split; runtime (Sections 3.6.3, 3.6.4, 4.6.4, 4.6.5, and 4.9)	effects kept on held-out data and on trackers not used in design; 4.2 ms per frame for AC-MOT (Tesla T4) and 2.95 ms per frame for SCL (CPU)

a running check that a new detector still fits its tracker.

5.2 Research Limitations

The results of Chapters 3 and 4 hold for the conditions under which they were obtained. This section states five limitations that bound their interpretation: two concern the signals from which the controllers decide, two concern the range of detectors and trackers that the evidence covers, and one concerns deployment hardware. Each limitation is stated together with what the experiments did show, so that the demonstrated behavior is not confused with untested conditions.

5.2.1 Dependence on Reliable Scene Cues

AC-MOT decides from inexpensive scene cues: the crowd cue and the tiny-object ratio, both computed from the boxes that the tracker reported in the previous frame, and the edge density, brightness, and sharpness of the current image (Section 3.3.1). Its state is therefore only as good as these cues. Because the crowd cue counts tracked boxes, it measures the scene through the output of the tracker: a tracker that reports many false tracks makes a scene look more crowded than it is, and a tracker that misses most objects, for example at night or under heavy blur, makes it look sparser. The scene label uses fixed thresholds on brightness and sharpness, and these thresholds were chosen for the cameras and conditions of the development data. In such cases the controller may select an operating point that does not suit the actual scene. The design reduces the effect of short disturbances, since the SCI averages seven analysis steps, the hysteresis and dwell rules prevent rapid switching, and the recovery probe reacts to a sudden collapse of the object count (Section 3.3.3). On the evaluated sequences the controller used all three states and changed state rarely (Section 3.6.2). The experiments did not, however, test sequences in which the cues are deliberately corrupted, and the behavior of the controller under strongly misleading cues remains an open question rather than a demonstrated failure or success.

5.2.2 Sensitivity to Detection-Score Distributions

SCL reads the distribution of recent detection scores and assumes that this distribution carries information about the objects: that the order of the scores separates objects from clutter well enough for the nested Otsu splits to form meaningful background, uncertain, and confident groups (Sections 4.3.3 and 4.3.4). This assumption held for the detectors, score transforms, and datasets of Chapter 4, including halved, cubed, and temperature-scaled scores. The splits are always computed, however, even when the scores are poorly separated, for example when the scores of objects and clutter

overlap strongly, when a detector emits only a narrow range of scores, or when a frame window contains very few candidates. In such cases the groups may not correspond to objects and clutter, and the confident share ρ and the clean or noisy decision become less informative. The whole-stream median keeps the mode stable against short disturbances, and the layer falls back to the tracker's own thresholds when no valid split exists (the cold mode, Algorithm 4.1), but a stream whose score distribution changes its character permanently in the middle of a video would be described by a median that adapts only slowly. These are assumptions of the score-based design; the experiments did not include detectors whose scores are uninformative, and no failure of this kind was observed in the evaluated settings.

5.2.3 Limited Detector Coverage of AC-MOT

The frozen AC-MOT controller was evaluated with one detector, YOLO11m trained on VisDrone2019-DET, on two aerial datasets, VisDrone2019-MOT and UAVDT, and with two trackers, ByteTrack and OATrack (Section 3.5). The earlier development stages used YOLOv8n with ByteTrack, the YOLOX-X detector of U2MOT, and SparseTrack on MOT17 (Section 3.4), but with earlier versions of the controller and other protocols, so they do not extend the evidence for the frozen controller. Its action map is also bound to the operating points for which detector outputs were cached: input resolutions of 1088, 1280, and 1536 pixels, NMS thresholds of 0.45, 0.60, and 0.70, and score thresholds of 0.10, 0.25, and 0.40. The improvements reported in Chapter 3 therefore establish the benefit of the frozen controller for this detector and these operating points. They do not by themselves establish the same benefit for detectors of other families, such as two-stage or transformer-based detectors, whose score distributions and sensitivity to input resolution differ, or for detectors trained on other data. For a new detector, the controller would at least need to be checked, and possibly recalibrated with the same cross-validation procedure.

5.2.4 The Host Contract of SCL

SCL serves a tracker through a host contract: the tracker must publish four values, the matching threshold a, the birth threshold b, the lowest score ℓ that it uses, and the matching tolerance m_0, and it must accept, for every frame, a list of candidate boxes with scores together with new values of these thresholds and of the tolerance (Section 4.3.8). The architectural requirements behind this contract are that the tracker separates detection from association, reads the detector scores through explicit thresholds, and matches tracks and boxes with an overlap-based tolerance that can be set from outside. Trackers that meet these requirements were served by one fixed configuration in this thesis: seven published tracking-by-detection trackers, including two

external trackers chosen before the run. Other parts of the integration are implementation choices: each tracker family needed a small adapter that writes the values of the layer into its existing parameters before each update, so a new tracker requires a new adapter even when it satisfies the contract. Trackers that do not expose such control points, for example joint detection-and-tracking and transformer-based trackers whose thresholds are internal to a network (Section 2.3.4), or trackers that associate mainly by appearance without score thresholds, cannot be served directly. SCL is therefore applicable to trackers with an explicit score-threshold interface, not to every tracker.

5.2.5 Incomplete Embedded-Hardware Validation

The computational cost of both controllers was measured, but not on embedded hardware. For AC-MOT, the time spent in the controller was measured on a Tesla T4 GPU over 200 frames (Section 3.6.4); for SCL, the time of the layer and the end-to-end time of a YOLOv8n and ByteTrack pipeline were measured on a 4-vCPU server processor without a GPU (Section 4.9). These measurements show that the controllers themselves are inexpensive compared with the detector, but they are not an end-to-end validation on the resource-constrained processors carried by UAVs. On such platforms, the detector at each operating point determines most of the latency and the throughput, and AC-MOT changes the input resolution, so the end-to-end frame rate depends on the hardware. Memory use, power consumption, thermal limits during long flights, and the effect of hardware-specific optimizations such as reduced numerical precision were not measured, and the sustained performance of the complete pipeline over long operation was not evaluated. The real-time suitability shown in this thesis therefore refers to the measured cost of the controllers, not to a demonstrated deployment on embedded hardware.

5.3 Future Work

The limitations above point to the following directions, which build on the training-free and modular design of the two controllers. None of these studies has been carried out in this thesis.

1. *Robust scene cues and controller reliability* (Section 5.2.1). The scene cues of AC-MOT can be made less dependent on the output of the tracker, for example by combining the count of tracked boxes with cues that the detector provides directly, and the reliability of the state decisions can be studied on sequences with abrupt changes of illumination, blur, and crowd density, by recording the state changes per 100 frames together with HOTA, MOTA, and IDF1.

2. *Score-distribution robustness of SCL* (Section 5.2.2). A label-free measure of how well the score distribution is separated could tell the layer when its splits are reliable and when it should keep the tracker's own thresholds, and alternative label-free partitions of the scores, such as mixture models, can be compared with the nested Otsu splits on detectors with compressed or poorly separated score distributions.

3. *More detectors and datasets for AC-MOT* (Section 5.2.3). Because AC-MOT needs no training and changes only three arguments of the detector call, the same calibration procedure can be applied to other detector families, including two-stage and transformer-based detectors, and to further aerial datasets, and its action map can include further operating points. Each new detector is to be evaluated in two separate conditions, the frozen controller transferred without retuning and the controller recalibrated with the same cross-validation procedure, by HOTA, MOTA, and IDF1 against the detector at its fixed operating point.

4. *Portability of SCL to other tracker interfaces* (Section 5.2.4). The host contract can be extended to trackers that learn detection and association jointly or that rely mainly on appearance, by identifying the score thresholds or confidence gates that such trackers use internally, and a generic adapter interface would reduce the work of adding a new tracker. As in Chapter 4, each extension is to be judged by HOTA and IDS against the same tracker alone under score shifts and detector changes.

5. *Embedded-hardware evaluation* (Section 5.2.5). The complete pipeline, with each controller, is to be evaluated on embedded processors of the kind carried on UAVs, such as an NVIDIA Jetson module, with a target of 30 FPS, that is, at most 33.3 ms per frame for the detector, the controller, and the tracker together, and with latency, throughput, memory use, and power consumption measured over long runs.

6. *Deployment optimizations.* On such hardware, the cost of the detector at each operating point of AC-MOT, the use of reduced numerical precision, and the scheduling of scene analysis and tracking can be optimized for latency, memory, throughput, and energy, and the effect of each optimization on the decisions of the controllers and on tracking accuracy is to be measured.

7. *Monitoring of detector updates.* The label-free mode and the confident share of SCL can serve as a running check when the detector of a deployed system is updated, retrained, or compressed; the objective is to log the mode and ρ before and after each update and to relate them to the tracking accuracy measured on labeled test sequences.

8. *A combined pipeline.* Since the two controllers act on different parts of the pipeline, they can be run together, with AC-MOT setting the detector operating point and SCL adapting the tracker input to the resulting scores, to evaluate whether their improvements are complementary. The planned evaluation compares four configurations on the same sequences—the original pipeline, AC-MOT alone, SCL alone, and both controllers together—by tracking accuracy (HOTA and MOTA), identity preservation (IDF1 and IDS), the stability of the controller states and of the SCL modes (changes per 100 frames), and the processing time per frame.

Bibliography

[1] S. Li, H. Ren, X. Xie, and Y. Cao, "A Review of Multi-Object Tracking in Recent Times," *IET Computer Vision*, vol. 19, no. 1, 2025.

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, IEEE, 2016.

[4] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, pp. 213–229, Springer International Publishing, 2020.

[5] A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, "Real-Time Object Detection and Tracking: Challenges, Innovations and Future Directions," in *2026 IEEE 3rd International Conference on Machine Intelligence and Smart Innovation (ICMISI)*, (Alexandria, Egypt), pp. 119–125, IEEE, 2026.

[6] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and Tracking Meet Drones Challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, 2022.

[7] X. Wu, W. Li, D. Hong, R. Tao, and Q. Du, "Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A Survey," *IEEE Geoscience and Remote Sensing Magazine*, vol. 10, no. 1, pp. 91–124, 2022.

[8] A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3354–3361, IEEE, 2012.

[9] D. Du, Y. Qi, H. Yu, Y. Yang, K. Duan, G. Li, W. Zhang, Q. Huang, and Q. Tian, "The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking," in

Computer Vision – ECCV 2018, Lecture Notes in Computer Science, pp. 375–391, Springer International Publishing, 2018.

[10] G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng, X. Xie, and J. Han, "Towards Large-Scale Small Object Detection: Survey and Benchmarks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–20, 2023.

[11] M. Adžemović, "Deep Learning-Based Multi-Object Tracking: A Comprehensive Survey from Foundations to State-of-the-Art," *The Visual Computer*, vol. 42, no. 9, p. 368, 2026.

[12] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*, (Phoenix, AZ, USA), pp. 3464–3468, IEEE, 2016.

[13] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, pp. 1–21, Springer Nature Switzerland, 2022.

[14] S. Liu, X. Li, H. Lu, and Y. He, "Multi-Object Tracking Meets Moving UAV," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8866–8875, IEEE, 2022.

[15] Y. Chae, H. Kim, and K.-J. Yoon, "Towards Robust 3D Object Detection with LiDAR and 4D Radar Fusion in Various Weather Conditions," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15162–15172, IEEE, 2024.

[16] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO Series in 2021." arXiv preprint arXiv:2107.08430, 2021.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks." arXiv preprint arXiv:1706.04599, 2017.

[18] F. Kuppers, J. Kronenberger, A. Shantia, and A. Haselhoff, "Multivariate Confidence Calibration for Object Detection," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 1322–1330, IEEE, 2020.

[19] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking," *International Journal of Computer Vision*, vol. 129, no. 2, pp. 548–578, 2021.

[20] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic Neural Networks: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2022.

[21] T.-W. Chin, R. Ding, and D. Marculescu, "AdaScale: Towards Real-Time Video Object Detection Using Adaptive Scaling," in *Proceedings of Machine Learning and Systems*, vol. 1, pp. 431–441, 2019.

[22] R. Xu, C.-L. Zhang, P. Wang, J. Lee, S. Mitra, S. Chaterji, Y. Li, and S. Bagchi, "ApproxDet: Content and Contention-Aware Approximate Object Detection for Mobiles," in *Proceedings of the 18th Conference on Embedded Networked Sensor Systems (SenSys)*, pp. 449–462, ACM, 2020.

[23] L. V. Ma, M. I. Hussain, J. Park, J. Kim, and M. Jeon, "Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking," in *2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS)*, pp. 370–374, IEEE, 2023.

[24] K. Shim, K. Ko, J. Hwang, and C. Kim, "AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking," in *2024 IEEE International Conference on Image Processing (ICIP)*, pp. 2222–2228, IEEE, 2024.

[25] D. U. Gwon and W. I. Lee, "Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking," *Electronics*, vol. 15, no. 17, p. 4047, 2026.

[26] T. Y. Goh, S. N. Basah, H. Yazid, M. J. Aziz Safar, and F. S. Ahmad Saad, "Performance Analysis of Image Thresholding: Otsu Technique," *Measurement*, vol. 114, pp. 298–307, 2018.

[27] A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, "Optimization-Guided Adaptive Detector Operating-Point Control for Multi-Object Aerial Tracking," *Journal of Aerospace Information Systems*, 2026. Submitted (under review).

[28] A. G. Ismail, T. A. Mahmoud, and M. S. Mohamed, "Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers," *The Visual Computer*, 2026. Submitted (under review).

[29] W. Sultani, C. Chen, and M. Shah, "Real-World Anomaly Detection in Surveillance Videos," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6479–6488, 2018.

[30] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, "Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking," in *Computer*

Vision – ECCV 2016 Workshops, Lecture Notes in Computer Science, pp. 17–35, Springer International Publishing, 2016.

[31] E. Ristani and C. Tomasi, "Features for Multi-Target Multi-Camera Tracking and Re-Identification," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6036–6046, 2018.

[32] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A Multimodal Dataset for Autonomous Driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11618–11628, 2020.

[33] X. Weng, J. Wang, D. Held, and K. Kitani, "3D Multi-Object Tracking: A Baseline and New Evaluation Metrics," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10359–10366, 2020.

[34] C. Kanellakis and G. Nikolakopoulos, "Survey on Computer Vision for UAVs: Current Developments and Trends," *Journal of Intelligent & Robotic Systems*, vol. 87, no. 1, pp. 141–168, 2017.

[35] M. Bakirci, "Performance Evaluation of Low-Power and Lightweight Object Detectors for Real-Time Monitoring in Resource-Constrained Drone Systems," *Engineering Applications of Artificial Intelligence*, vol. 159, p. 111775, 2025.

[36] N. Miller and J. Rogers, "Simultaneous Tracking of Multiple Ground Targets from a Multirotor Unmanned Aerial Vehicle," *Journal of Aerospace Information Systems*, vol. 12, no. 3, pp. 345–364, 2015.

[37] A. Desai and D.-J. Lee, "Efficient Feature Descriptor for Unmanned Aerial Vehicle Ground Moving Object Tracking," *Journal of Aerospace Information Systems*, vol. 14, no. 6, pp. 345–350, 2017.

[38] Y.-R. Lu and D. Sun, "Development of Real-Time Unmanned Aerial Vehicle Urban Object Detection System with Federated Learning," *Journal of Aerospace Information Systems*, vol. 21, no. 7, pp. 547–553, 2024.

[39] A. J. Choi, J. Park, and J.-H. Han, "Automated Aerial Docking System Using Onboard Vision-Based Deep Learning," *Journal of Aerospace Information Systems*, vol. 19, no. 6, pp. 421–436, 2022.

[40] C. Parsons, Z. Paulson, S. Nykl, W. Dallman, B. G. Woolley, and J. Pecarina, "Analysis of Simulated Imagery for Real-Time Vision-Based Automated Aerial Refueling," *Journal of Aerospace Information Systems*, vol. 16, no. 3, pp. 77–93, 2019.

[41] P. Dai, L. Chen, Y. Lian, and L. Xie, "LSO-YOLO: A Lightweight Real-Time Object Detection Network for UAV Small-Object Detection," *Journal of Real-Time Image Processing*, vol. 23, no. 5, p. 176, 2026.

[42] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. Aparicio, and K. Goldberg, "Dex-Net 2.0: Deep Learning to Plan Robust Grasps with Synthetic Point Clouds and Analytic Grasp Metrics," in *Robotics: Science and Systems XIII (RSS)*, 2017.

[43] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[44] G. Litjens, T. Kooi, B. E. Bejnordi, A. A. A. Setio, F. Ciompi, M. Ghafoorian, J. A. W. M. van der Laak, B. van Ginneken, and C. I. Sánchez, "A Survey on Deep Learning in Medical Image Analysis," *Medical Image Analysis*, vol. 42, pp. 60–88, 2017.

[45] A. P. Twinanda, S. Shehata, D. Mutter, J. Marescaux, M. de Mathelin, and N. Padoy, "EndoNet: A Deep Architecture for Recognition Tasks on Laparoscopic Videos," *IEEE Transactions on Medical Imaging*, vol. 36, no. 1, pp. 86–97, 2017.

[46] M. Mubashir, L. Shao, and L. Seed, "A Survey on Fall Detection: Principles and Approaches," *Neurocomputing*, vol. 100, pp. 144–152, 2013.

[47] E. Goldman, R. Herzig, A. Eisenschtat, J. Goldberger, and T. Hassner, "Precise Detection in Densely Packed Scenes," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5222–5231, 2019.

[48] Y. Wei, S. Tran, S. Xu, B. Kang, and M. Springer, "Deep Learning for Retail Product Recognition: Challenges and Techniques," *Computational Intelligence and Neuroscience*, vol. 2020, p. 8875910, 2020.

[49] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, pp. 886–893, IEEE, 2005.

[50] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[51] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[52] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015.

[53] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 580–587, IEEE, 2014.

[54] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, pp. 21–37, Springer International Publishing, 2016.

[55] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, IEEE, 2016.

[56] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need." arXiv preprint arXiv:1706.03762, 2017.

[57] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." arXiv preprint arXiv:2010.11929, 2020.

[58] P. Sun, J. Cao, Y. Jiang, R. Zhang, E. Xie, Z. Yuan, C. Wang, and P. Luo, "TransTrack: Multiple Object Tracking with Transformer." arXiv preprint arXiv:2012.15460, 2020.

[59] T. Meinhardt, A. Kirillov, L. Leal-Taixé, and C. Feichtenhofer, "TrackFormer: Multi-Object Tracking with Transformers," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8834–8844, IEEE, 2022.

[60] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang, and Y. Wei, "MOTR: End-to-End Multiple-Object Tracking with Transformer," in *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, pp. 659–675, Springer Nature Switzerland, 2022.

[61] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *Computer Vision – ECCV 2014*, Lecture Notes in Computer Science, pp. 740–755, Springer International Publishing, 2014.

[62] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, IEEE, 2017.

[63] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement." arXiv preprint arXiv:1804.02767, 2018.

[64] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection." arXiv preprint arXiv:2004.10934, 2020.

[65] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," in *Computer Vision – ECCV 2024*, Lecture Notes in Computer Science, pp. 1–21, 2024.

[66] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8." Software package, Ver. 8.0.0, Ultralytics, `https://github.com/ultralytics/ultralytics`, 2023.

[67] G. Jocher and J. Qiu, "Ultralytics YOLO11." Software package, Ver. 11.0.0, Ultralytics, `https://github.com/ultralytics/ultralytics`, 2024.

[68] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 2999–3007, IEEE, 2017.

[69] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10778–10787, IEEE, 2020.

[70] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 936–944, IEEE, 2017.

[71] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs Beat YOLOs on Real-Time Object Detection," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16965–16974, IEEE, 2024.

[72] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, "RF-DETR: Neural Architecture Search for Real-Time Detection Transformers." arXiv preprint arXiv:2511.09554, 2025.

[73] H. W. Kuhn, "The Hungarian Method for the Assignment Problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[74] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[75] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in *2017 IEEE International Conference on Image Processing (ICIP)*, pp. 3645–3649, IEEE, 2017.

[76] Z. Wang, L. Zheng, Y. Liu, Y. Li, and S. Wang, "Towards Real-Time Multi-Object Tracking," in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, pp. 107–122, Springer International Publishing, 2020.

[77] Y. Zhang, C. Wang, X. Wang, W. Zeng, and W. Liu, "FairMOT: On the Fairness of Detection and Re-Identification in Multiple Object Tracking," *International Journal of Computer Vision*, vol. 129, no. 11, pp. 3069–3087, 2021.

[78] X. Zhou, V. Koltun, and P. Krähenbühl, "Tracking Objects as Points," in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, pp. 474–490, Springer International Publishing, 2020.

[79] R. Gao, J. Qi, and L. Wang, "Multiple Object Tracking as ID Prediction," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 27883–27893, IEEE, 2025.

[80] E. Bochinski, V. Eiselein, and T. Sikora, "High-Speed Tracking-by-Detection without Using Image Information," in *2017 14th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, pp. 1–6, IEEE, 2017.

[81] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9686–9696, IEEE, 2023.

[82] F. Yang, S. Odashima, S. Masui, and S. Jiang, "Hard to Track Objects with Irregular Motions and Similar Appearances? Make It Easier by Buffering the Matching Space," in *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 4788–4797, IEEE, 2023.

[83] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust Associations Multi-Pedestrian Tracking." arXiv preprint arXiv:2206.14651, 2022.

[84] G. Evangelidis and E. Psarakis, "Parametric Image Alignment Using Enhanced Correlation Coefficient Maximization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 10, pp. 1858–1865, 2008.

[85] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, "StrongSORT: Make DeepSORT Great Again," *IEEE Transactions on Multimedia*, vol. 25, pp. 8725–8737, 2023.

[86] G. Maggiolino, A. Ahmad, J. Cao, and K. Kitani, "Deep OC-SORT: Multi-Pedestrian Tracking by Adaptive Re-Identification," in *2023 IEEE International Conference on Image Processing (ICIP)*, pp. 3025–3029, IEEE, 2023.

[87] Y.-H. Wang, J.-W. Hsieh, P.-Y. Chen, M.-C. Chang, H.-H. So, and X. Li, "SMILEtrack: SiMIlarity LEarning for Occlusion-Aware Multiple Object Tracking," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 6, pp. 5740–5748, 2024.

[88] M. Yang, G. Han, B. Yan, W. Zhang, J. Qi, H. Lu, and D. Wang, "Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 7, pp. 6504–6512, 2024.

[89] Z. Liu, X. Wang, C. Wang, W. Liu, and X. Bai, "SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 35, no. 5, pp. 4870–4882, 2025.

[90] V. D. Stanojevic and B. T. Todorovic, "BoostTrack: Boosting the Similarity Measure and Detection Confidence for Improved Multiple Object Tracking," *Machine Vision and Applications*, vol. 35, no. 3, 2024.

[91] Y. Wang, D. Zhang, R. Li, Z. Zheng, and M. Li, "PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 1, pp. 165–177, 2025.

[92] K. Liu, S. Jin, Z. Fu, Z. Chen, R. Jiang, and J. Ye, "Uncertainty-Aware Unsupervised Multi-Object Tracking," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9996–10005, 2023.

[93] L. Qi, L. Wang, and S. Jiang, "OATrack: A Quality-Gated Progressive Association Framework with a YOLO Detection Cache for UAV Small-Object Multi-Object Tracking," *Sensors*, vol. 26, no. 16, p. 5222, 2026.

[94] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, "Soft-NMS — Improving Object Detection with One Line of Code," in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 5562–5570, IEEE, 2017.

[95] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully Test-Time Adaptation by Entropy Minimization." arXiv preprint arXiv:2006.10726, 2020.

[96] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, "MOT16: A Benchmark for Multi-Object Tracking." arXiv preprint arXiv:1603.00831, 2016.

[97] P. Dendorfer, A. Ošep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixé, "MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking," *International Journal of Computer Vision*, vol. 129, no. 4, pp. 845–881, 2021.

[98] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé, "MOT20: A Benchmark for Multi Object Tracking in Crowded Scenes." arXiv preprint arXiv:2003.09003, 2020.

[99] P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani, and P. Luo, "DanceTrack: Multi-Object Tracking in Uniform Appearance and Diverse Motion," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20961–20970, IEEE, 2022.

[100] K. Bernardin and R. Stiefelhagen, "Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics," *EURASIP Journal on Image and Video Processing*, vol. 2008, pp. 1–10, 2008.

[101] B. Efron and T. Hastie, *Computer Age Statistical Inference: Algorithms, Evidence, and Data Science*. Cambridge: Cambridge University Press, 2016.

[102] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-Generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, (Anchorage, AK, USA), pp. 2623–2631, ACM, 2019.

[103] Y. Li, Y. Liu, C. Zhou, D. Xu, and W. Tao, "A Lightweight Scheme of Deep Appearance Extraction for Robust Online Multi-Object Tracking," *The Visual Computer*, vol. 40, no. 3, pp. 2049–2065, 2024.

[104] Y. Bao, Y. Yu, Y. Qi, and Z. Wang, "Multiple Object Tracking with Adaptive Multi-Features Fusion and Improved Learnable Graph Matching," *The Visual Computer*, vol. 40, no. 4, pp. 2279–2292, 2024.

[105] Y. Zhang, Y. Liang, J. Wang, H. Zhu, and Z. Wang, "Enhanced Multi-Object Tracking via Embedded Graph Matching and Differentiable Sinkhorn Assign-

ment: Addressing Challenges in Occlusion and Varying Object Appearances," *The Visual Computer*, vol. 41, no. 8, pp. 6067–6085, 2025.

[106] Y. Ding, Y. Wang, L. Wang, B. Zhang, J. Li, L. Guo, Z. Yang, and W. Liang, "Enhancing Multi-Object Tracking Efficiency through Result-Guided Feature Extraction and Query Filtering," *The Visual Computer*, vol. 41, no. 11, pp. 8153–8167, 2025.

[107] Z. Liang, G. Xiao, J. Hu, J. Wang, and C. Ding, "MotionTrack: Rethinking the Motion Cue for Multiple Object Tracking in USV Videos," *The Visual Computer*, vol. 40, no. 4, pp. 2761–2773, 2024.

[108] Z. Sun, Z. Lv, M. Xiong, W. Hu, G. Wei, J. Zhou, T. He, and Q. Meng, "Online Visual Multi-Object Tracking via Blind Super-Resolution and Clustering Learning," *The Visual Computer*, vol. 42, no. 4, p. 189, 2026.

[109] S. Kim, I. Petrunin, and H.-S. Shin, "AFJPDA: A Multiclass Multi-Object Tracking with Appearance Feature-Aided Joint Probabilistic Data Association," *Journal of Aerospace Information Systems*, vol. 21, no. 4, pp. 294–304, 2024.

[110] G. Huang, X. Zhang, R. Zhao, W. Li, B. Liang, and J. Xie, "Efficient Small-Object Detection in Airport Surface Based on Maintain Feature High Resolution," *Journal of Aerospace Information Systems*, vol. 19, no. 4, pp. 305–316, 2022.

[111] M. Fathy, M. K. Elhadad, and M. A. Elshafey, "TOReN: Tiny Object Recognition Network in Remote Sensing Imagery," *Journal of Aerospace Information Systems*, vol. 23, no. 8, pp. 784–803, 2026.

[112] L. Yang, Y. Han, X. Chen, S. Song, J. Dai, and G. Huang, "Resolution Adaptive Networks for Efficient Inference," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2369–2378, 2020.

[113] J. Jiang, G. Ananthanarayanan, P. Bodik, S. Sen, and I. Stoica, "Chameleon: Scalable Adaptation of Video Analytics," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, (New York, NY, USA), pp. 253–266, ACM, 2018.

[114] H. Jung, S. Kang, T. Kim, and H. Kim, "ConfTrack: Kalman Filter-Based Multi-Person Tracking by Utilizing Confidence Score of Detection Box," in *2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 6569–6578, 2024.

[115] E. Solano-Carrillo, F. Sattler, A. Alex, A. Klein, B. P. Costa, A. B. Rodriguez, and J. Stoppe, "UTrack: Multi-Object Tracking with Uncertain Detections," in *Computer Vision – ECCV 2024 Workshops*, Lecture Notes in Computer Science, pp. 219–236, Springer Nature Switzerland, 2025.

[116] S. Kim, S. Lee, and M. Cho, "PlugTrack: Multi-Perceptive Motion Analysis for Adaptive Fusion in Multi-Object Tracking," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, pp. 5709–5716, 2026.

[117] J. Pech-Pacheco, G. Cristobal, J. Chamorro-Martinez, and J. Fernandez-Valdivia, "Diatom Autofocusing in Brightfield Microscopy: A Comparative Study," in *Proceedings of the 15th International Conference on Pattern Recognition (ICPR)*, pp. 314–317, IEEE Computer Society, 2000.

[118] X. Tong, Z. Ye, Y. Xu, S. Gao, H. Xie, Q. Du, S. Liu, X. Xu, S. Liu, K. Luan, and U. Stilla, "Image Registration with Fourier-Based Image Correlation: A Comprehensive Review of Developments and Applications," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 10, pp. 4062–4081, 2019.

Publications

1. A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, "Real-Time Object Detection and Tracking: Challenges, Innovations and Future Directions," in *2026 IEEE 3rd International Conference on Machine Intelligence and Smart Innovation (ICMISI)*, Alexandria, Egypt, 2026, pp. 119–125, doi: 10.1109/ICMISI69868.2026.11583994.

2. A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, "Optimization-Guided Adaptive Detector Operating-Point Control for Multi-Object Aerial Tracking," *Journal of Aerospace Information Systems*, submitted (under review).

3. A. G. Ismail, T. A. Mahmoud, and M. S. Mohamed, "Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers," *The Visual Computer*, submitted (under review).

The first publication forms the basis of Chapter 2, the second of Chapter 3, and the third of Chapter 4.